Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Aryan Patankar** of **D20A** semester **VIII,** have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026.**

Subject Teacher
Name:Mrs. Kajal Joseph
Signature:

Head of Department
Name: Dr. Mrs. Shalu Chopra
Signature:

Lab Teacher
Name: Kajal Joseph
Signature:

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	**Computing & Emerging Technology:** Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	**Professionalism & Innovation :** Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	**PO**											**PSO**	
	PO 1	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PSO 1**	**PSO 2**
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.1

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

1. Cryptographic Hash Functions in Blockchain

Cryptographic hash functions are mathematical algorithms that take an input of any length and produce a fixed-length output string known as a hash value. In blockchain technology, they play a crucial role in ensuring data integrity, security, and immutability. These functions must be cryptographically secure, meaning they exhibit specific properties that make them resistant to attacks.

Properties of Cryptographic Hash Functions

- **Collision Resistant**: It is computationally infeasible to find two different inputs (m1 and m2) that produce the same hash output.
- **Preimage Resistant**: Given a hash value h, it is difficult to find the original input m that produced h.
- **Second Preimage Resistant**: Given an input m1, it is hard to find another input m2 that produces the same hash as m1.
- **Large Output Space**: Brute-force attacks require checking an enormous number of possibilities.
- **Deterministic**: The same input always produces the same output.
- **Avalanche Effect**: A small change in the input causes a significant change in the output.

- **Puzzle Friendliness**: Even if part of the output is known, predicting the rest is impossible.
- **Fixed-Length Mapping**: Outputs are always of a consistent length regardless of input size.

How Hash Functions Work in Blockchain

Hash functions convert variable-length data (e.g., transactions) into fixed-size outputs. In blockchain, this process involves encryption (generating the hash) and potential decryption for verification. Common algorithms include SHA-256, which always outputs 256 bits (32 bytes).

Properties of SHA-256

SHA-256 (Secure Hash Algorithm – 256-bit) belongs to the SHA-2 family and is one of the most commonly used hashing algorithms in blockchain technology. It converts any input data into a secure 256-bit hash value, regardless of the input's original size.

- **Fixed 256-bit Output:** SHA-256 always produces a hash of exactly 256 bits, usually shown as 64 hexadecimal characters, no matter how large the input is.
- **High Security Strength:** It is widely trusted for modern cryptography and is a standard choice in blockchain networks for strong protection.
- **Collision Resistance:** The chance that two different pieces of data generate the same hash is extremely rare, ensuring reliable data integrity.
- **Avalanche Property:** Even the smallest change in input data results in a completely different hash output, making unauthorized modifications easy to detect.
- **Irreversible (One-Way Function):** It is practically impossible to reverse the hash and recover the original input data.

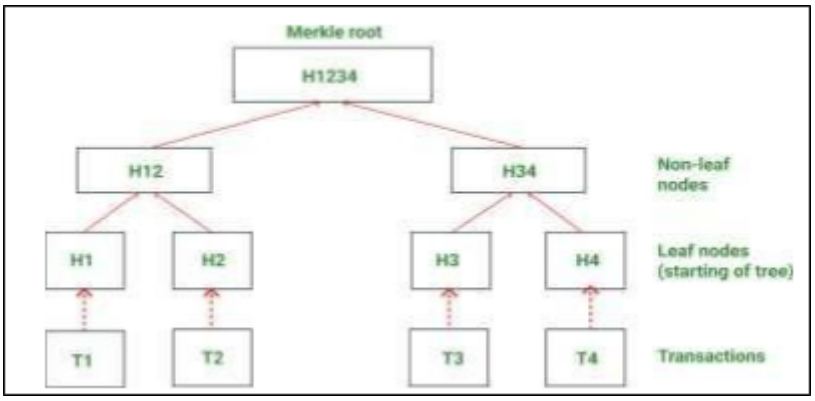
2. What is a Merkle Tree?

A Merkle Tree, also known as a hash tree, is a binary tree data structure used for efficient data verification and synchronization. Each leaf node represents the hash of a data block (e.g., a transaction), while each non-leaf node is the hash of its child nodes. The tree culminates in a single root hash (Merkle root), which serves as a digital fingerprint of the entire dataset.

In blockchain, Merkle Trees organize transactions per block, replacing less efficient structures like linked lists. They leverage cryptographic hashes and hash pointers (pointers that include both data location and its hash for verification).

Structure of a Merkle Tree

A Merkle Tree follows a layered hierarchical arrangement. It begins with individual transaction hashes at the bottom level. These hashes are merged pairwise to form parent nodes, continuing upward through multiple levels until reaching the topmost hash, the **Merkle Root**, which summarizes and secures all transactions within the block.



1. **Leaf Nodes:** These are the hashes of individual transactions

in a block. Each transaction is first hashed to form a leaf node.

2. **Intermediate (Parent) Nodes:** Pairs of leaf nodes are combined and hashed again to form parent nodes. This process continues upward, combining child nodes at each level.
3. **Root Node (Merkle Root):** The topmost node of the tree, representing all transactions in the block. If any transaction changes, the Merkle Root changes, making tampering detectable.

2. **<u>Merkle Root</u>**

   The Merkle Root is the **topmost hash** of the Merkle Tree. It acts as a summary of all transactions in a block.
   If any transaction is changed, its hash changes, which affects the Merkle Root. Since the Merkle Root is stored in the block header, this makes tampering easy to detect.

3. What is a Cryptographic Puzzle and Explanation of the Golden Nonce

A cryptographic puzzle in blockchain refers to a complex mathematical problem that miners must solve to validate transactions and add new blocks. It is central to the Proof of Work (PoW) consensus mechanism, particularly in networks like Bitcoin. The puzzle involves finding a specific hash value that meets the network's difficulty criteria, such as being below a certain target threshold. This requires intensive computations, making it resource-heavy to solve but easy to verify.

Key Aspects of Cryptographic Puzzles

- **Purpose**: Secures the network by preventing easy tampering, double-spending, or counterfeiting. It ensures consensus without trusting any single party.

- **Process**: Miners hash block data (including transactions and a nonce) repeatedly until the output satisfies conditions. The difficulty adjusts dynamically (e.g., every 2016 blocks in Bitcoin) to maintain a ~10-minute block time.
- **Computational Intensity**: Involves brute-force trials; with millions of miners, it takes significant power and time.
- **Security**: Limits total coin supply (e.g., 21 million Bitcoins) and makes illicit changes expensive.

The "Golden Nonce" refers to the specific nonce value that, when combined with the block data and hashed, produces a valid hash solving the puzzle. It is the "winning" nonce that allows a miner to claim the block reward (e.g., 6.25 Bitcoins). Miners iterate through nonce values (a 32-bit number) until this golden nonce is found, after which the block is broadcast for verification

4. Working of Merkle Tree

The Merkle Tree works by organizing and summarizing all transactions in a block into a single hash (the Merkle Root) so that data can be verified efficiently and securely.

The process follows these steps:

- **Hashing Transactions:** Each transaction in the block is first hashed using a cryptographic hash function, usually SHA-256. These hashes form the leaf nodes of the Merkle Tree.
  Example:
  Transactions: T1, T2,
  T3, T4 Hashes: H(T1),
  H(T2), H(T3), H(T4)

- **Pairing and Hashing:** The leaf nodes are grouped in pairs. Each pair of hashes is concatenated (joined together) and then hashed again to create a parent node.
  If the number of transactions is odd, the last hash is duplicated to form a pair. This ensures that every level of the tree has an even number of nodes.
  Example:
  - Pair 1: H(T1) + H(T2) → Hash = H12
  - Pair 2: H(T3) + H(T4) → Hash = H34

- **Building the Tree:** The pairing and hashing process continues up the tree, combining parent nodes to create new higher-level nodes.
  Example:
  H12 + H34 → Hash = H123

- **Creating the Merkle Root:** This process repeats until only one hash remains at the top of the tree. This top-level hash is called the Merkle Root, which represents all transactions in the block.

- **Verification:** To verify that a transaction exists in a block, a Merkle Proof is used. Instead of checking every transaction, only a small number of hashes along the path from the transaction leaf to the Merkle Root are needed. This makes verification fast and efficient, even for large blocks of data.
- **Example:**
  To verify T1: Use H(T2) and H34 along with H(T1) to recalculate H1234. If it matches the Merkle Root, T1 is valid.

- **Tamper Detection:** If any transaction is modified, its hash changes. This change propagates up the tree and alters the

Merkle Root. Since the Merkle Root is stored in the block header, any tampering becomes immediately obvious.

Example:

If T3 changes → H(T3) changes → H34 changes → H1234 changes → Merkle Root

mismatch.

5. Benefits of Merkle Tree

- **Efficient Verification:** Only a small number of hashes are needed to verify a transaction using a Merkle Proof, so checking data is fast even for large blocks.
- **Data Integrity:** Any change in a transaction alters the Merkle Root, making it easy to detect tampering.
- **Reduced Storage:** Instead of storing all transaction data in the block header, only the Merkle Root is stored, saving space.
- **Scalability:** Merkle Trees allow blockchains to handle a large number of transactions without slowing down verification.
- **Security:** The structure ensures that transactions cannot be altered without being noticed, keeping the blockchain secure.

6. Use Cases of Merkle Tree

Merkle Trees are versatile, with primary applications in blockchain and beyond:

- **Blockchain Transactions**: Organize and verify transactions in blocks; Merkle root in headers ensures integrity.
- **Proof of Membership**: Miners or users prove a transaction's inclusion using a Merkle proof (path of hashes), requiring only O(log n) data.
- **Simple Payment Verification (SPV)**: Lightweight clients verify transactions without the full blockchain.

- **Coinbase Transactions**: Included in every block's tree for new coin creation.
- **Distributed Systems**: Ensure data consistency across nodes; detect inconsistencies in replicas (e.g., Apache Cassandra databases).
- **File Systems**: Verify content in systems like IPFS or BitTorrent.
- **Inconsistency Detection**: Compare root hashes to pinpoint data differences quickly.

In Bitcoin and other blockchains, they enable trustless, efficient operations in peer-to-peer networks.

Colab Notebook:
Blockchain Exp1

Code & Output:

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```python
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()

    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

```
Enter a string: aryanpatankar
Hash: a224b5173a85e4214fbf6d88600d3b4dca7b9523032095bbdb0584ed4e73fa87
```

2.**Target Hash Generation with Nonce**: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

```python
# Program 2: Generating Target Hash with Input String and Nonce
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

```
Enter a string: Aryan
Enter the nonce: 1
Hash Code: 52d2de03394c757ee98f7cd8b73a72c99bfa8d953cbfde1905f948b6fc51c61d
```

3.**Proof-of-Work Puzzle Solving**: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```python
# Program 3: Solving Cryptographic Puzzle for Leading Zeros
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = "Aryan"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

```
Hash: 043a2485b4ad07329119b33b48423d5477bf17a2afab3fe342b67cd0a727dde5
Input String: Aryan
Leading Zeros: 1
Expected Nonce: 20
```

4.**Merkle Tree Constructio**n: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```
import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Hash each transaction initially
    hashed_transactions = [
        hashlib.sha256(t.encode('utf-8')).hexdigest()
        for t in transactions
    ]
    print("Initial Hashed Transactions:", hashed_transactions)

    # If only one transaction, it is the Merkle Root
    if len(hashed_transactions) == 1:
        return hashed_transactions[0]

    # Iterative construction of the Merkle Tree
    current_level = hashed_transactions
    level = 1
```

```
while len(current_level) > 1:
    print(f"\nLevel {level} Hashes:")

    # If odd number of hashes, duplicate the last one
    if len(current_level) % 2 != 0:
        current_level.append(current_level[-1])

    new_level = []

    for i in range(0, len(current_level), 2):
        combined = current_level[i] + current_level[i + 1]
        hash_combined = hashlib.sha256(
            combined.encode('utf-8')
        ).hexdigest()

        new_level.append(hash_combined)

        print(
            f"  Combining {current_level[i][:6]}... and "
            f"{current_level[i + 1][:6]}... to get "
            f"{hash_combined[:6]}..."
        )

    current_level = new_level
    level += 1

# Final remaining hash is the Merkle Root
return current_level[0]
```

```
Initial Hashed Transactions: ['4d5a82f66fccc5af28031a82a5ceb39dc3fe23097c56c8dff7720841a02aef1f', 'fd0b9eafb0c7f54035c2f21ff45dbbf165440d59

Level 1 Hashes:
  Combining 4d5a82... and fd0b9e... to get 9cd41e...
  Combining 43dd57... and 2ed5c8... to get 9f37c0...
  Combining 4313e3... and 4313e3... to get bc2172...

Level 2 Hashes:
  Combining 9cd41e... and 9f37c0... to get 6fa7f1...
  Combining bc2172... and bc2172... to get ba86e9...

Level 3 Hashes:
  Combining 6fa7f1... and ba86e9... to get c5db5f...

Merkle Root: c5db5fb4092cf71773baf667101b0158168f44d9a99579c5385353c80ef38aff
```

Conclusion:-

Cryptographic hash functions like **SHA-256** help keep blockchain data safe, secure, and unchangeable. **Merkle Trees** organize transactions in a way that makes it easy to verify them quickly and detect any tampering. Together, they make blockchain trustworthy, capable of handling large amounts of data without compromising security. These concepts help us understand how blockchain maintains security, efficiency even with large amounts of data.

ITC801: Blockchain & DLT Lab
Experiment 02

Aim:Create a Blockchain using Python

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.2

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim: Create a Blockchain using Python

1. What is Blockchain

Blockchain is a revolutionary technology that functions as a shared, immutable digital ledger, often described as a decentralized database or distributed ledger technology. It records transactions across a network of computers in a way that ensures security, transparency, and resistance to tampering. The term "blockchain" derives from its structure: data is organized into blocks, each linked to the previous one through cryptographic hashes, forming a continuous chain. This decentralized nature eliminates the need for intermediaries, allowing multiple parties to perform transactions without third-party intervention.

Key characteristics include:

- **Decentralization**: No single entity controls the network; it's maintained by nodes (computers) that validate and record transactions.
- **Immutability**: Once data is added, it cannot be altered without consensus from the network, making it tamper-proof.
- **Transparency**: All participants can view the ledger, but privacy is maintained through cryptography.
- **Security**: Uses cryptographic techniques to protect data.

Blockchain stores various types of information, but it's most commonly used for transactions in cryptocurrencies like Bitcoin. It operates on principles

such as consensus mechanisms to agree on the state of the ledger. Unlike traditional databases that structure data in tables, blockchain uses blocks chained together, ensuring that changes to one block would require altering all subsequent blocks, which is computationally infeasible

2. What is a Block?

A block is the fundamental building block of a blockchain, acting as a container for data such as transactions. It serves as a record in the digital ledger, similar to a page in a book, where each block holds a complete record of transactions or other information. Blocks are linked together in a chain, with each new block referencing the previous one via a cryptographic hash, creating an unbreakable sequence.

In essence, a block is a node in the blockchain network that stores encrypted data, including a list of transactions, a timestamp, and a unique identifier (hash). The first block in any blockchain is called the Genesis Block, which has no previous block reference and is hardcoded into the protocol. This structure ensures the integrity of the entire chain, as altering any block would invalidate the hashes of all following blocks

3. Components of a Block

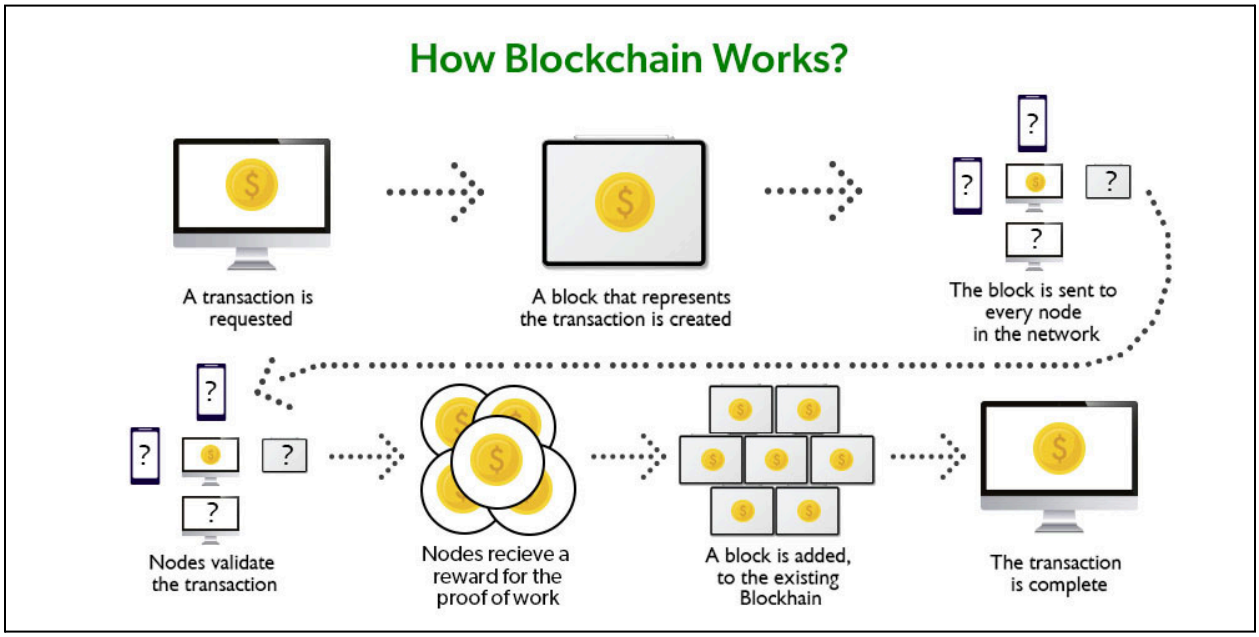
A block in blockchain consists of two main parts: the block header and the block body (containing transactions). The header is crucial for identification and linking, while the body holds the actual data.

Key components include:

Component	Description
Block Header	Identifies the block and includes metadata like version, timestamp, Merkle root, difficulty target, nonce, and previous block hash.
Version	A 4-byte field tracking software or protocol upgrades.
Previous Block Hash	A 32-byte reference to the hash of the prior block, ensuring chain continuity.
Merkle Root	A 32-byte hash of all transactions in the block, derived from a Merkle tree for efficient verification.
Timestamp	A 4-byte record of the block's approximate creation time.
Difficulty Target	A 4-byte value setting the mining difficulty.
Nonce	A 4-byte number used in proof-of-work mining to find a valid hash.

Transactions	The list of verified transactions stored in the block body.

These elements work together to maintain security and enable validation. The header alone is often sufficient for light clients to verify blocks without the full data



4. Process of mining

Mining is the process of validating transactions and adding new blocks to the blockchain. It is primarily associated with **Proof-of-Work (PoW)** consensus mechanisms (e.g., Bitcoin). Miners compete to solve a complex cryptographic puzzle, and the winner gets to add the block and receive a reward.

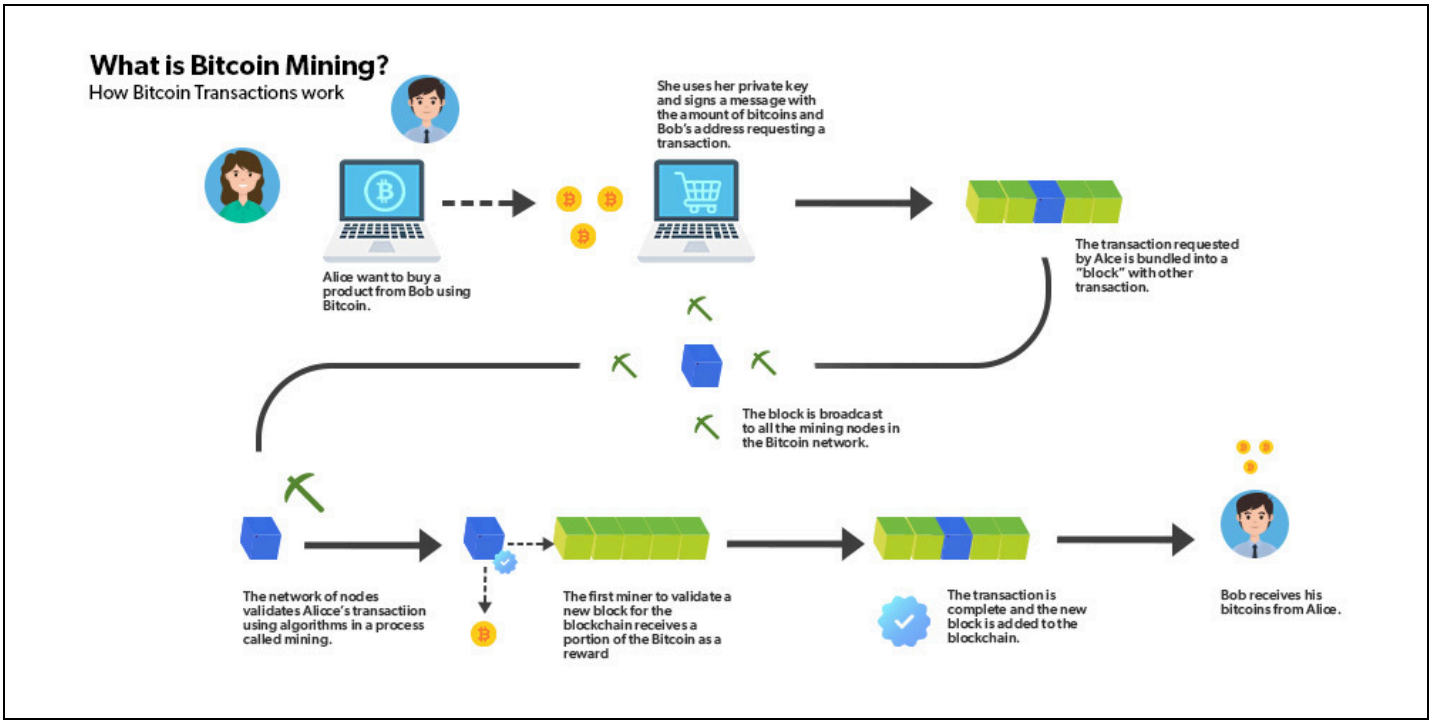
Key steps in the mining process:

2. **Transaction Collection** Unconfirmed transactions are collected from the network and stored in a **mempool** (memory pool). Miners select

transactions (usually prioritizing those with higher fees) to include in a candidate block. A special **coinbase transaction** is added, which rewards the miner with new coins + transaction fees.

3. **Block Header Preparation** The miner constructs the block header, which includes:
    - Previous block hash
    - Merkle root (summary hash of all transactions)
    - Timestamp
    - Difficulty target
    - Nonce (a variable number starting from 0)
4. **Solving the Cryptographic Puzzle (Proof-of-Work)** Miners repeatedly hash the block header (using SHA-256 in Bitcoin) while changing the nonce value. The goal is to find a hash that is **less than or equal to the current difficulty target** (a very small number, often requiring the hash to start with many leading zeros). This is computationally intensive and probabilistic — miners try billions/trillions of nonces per second.
5. **Block Found** The first miner to find a valid nonce (producing a qualifying hash) has "solved" the puzzle. They broadcast the completed block to the network.
6. **Network Acceptance & Reward** Other nodes verify the block. If valid, it is added to their copy of the blockchain. The successful miner receives the **block reward** (e.g., currently 3.125 BTC as of recent halvings) + transaction fees. Difficulty adjusts periodically to keep average block time stable (~10 minutes in Bitcoin).

Mining secures the network by making it extremely expensive to alter past blocks (would require re-mining all subsequent blocks) and prevents double-spending.



5. Checking the validity of blocks in blockchain

Once a mined block is broadcast, every full node independently verifies it before accepting it into their blockchain copy. This decentralized validation is what makes blockchain trustless and secure.

Key validity checks:

- **Block Structure & Syntax**
  - Correct format and size
  - Valid version number
  - Timestamp is reasonable (not too far in future or before previous block)
- **Previous Block Hash Match** The "previous block hash" field must exactly match the hash of the most recent block already in the node's chain.

- **Proof-of-Work Validation** Recompute the block hash → check if it is ≤ current difficulty target (i.e., has enough leading zeros). This confirms the miner did the required work.
- **Merkle Root Verification** Rebuild the Merkle tree from all transactions in the block body → verify the computed Merkle root matches the one in the block header.
- **Transaction Validity**
    - Each transaction has valid signatures
    - Inputs are unspent (no double-spending)
    - `Input amounts ≥ output amounts + fees`
    - Transactions follow protocol rules
- **Consensus Rules Compliance** Block follows all network rules (max block size, valid coinbase, etc.).

If **any** check fails, the block is rejected and not added to the chain. Nodes continue waiting for a valid block. The longest valid chain (most cumulative work) is considered the true chain.

Light clients (SPV wallets) use simplified checks — they rely on **Merkle proofs** to confirm a transaction exists in a block without downloading everything.

Implementation:-

Installing Flask

```
PS C:\Users\Student\Downloads\BCEx2> pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
```

blockchain.py

```
import datetime
import hashlib
import json


class Blockchain:

    def __init__(self):
        self.chain = []
        self.difficulty = 4  # 4 leading zeroes
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = {
            'index': 1,
            'timestamp': str(datetime.datetime.now()),
            'data': 'Genesis Block',
            'nonce': 0,
```

```
        'previous_hash': '0'
    }
    genesis_block['hash'] = self.calculate_hash(genesis_block)
    self.chain.append(genesis_block)

def calculate_hash(self, block):
    block_copy = block.copy()
    block_copy.pop('hash', None)  # Remove hash before recalculating

    encoded = json.dumps(block_copy, sort_keys=True).encode()
    return hashlib.sha256(encoded).hexdigest()

def mine_block(self, data):
    previous_block = self.chain[-1]
    nonce = 0

    while True:
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'data': data,
            'nonce': nonce,
            'previous_hash': previous_block['hash']
        }

        block_hash = self.calculate_hash(block)

        # Check difficulty condition
        if block_hash.startswith('0' * self.difficulty):
            block['hash'] = block_hash
            self.chain.append(block)
            return block
```

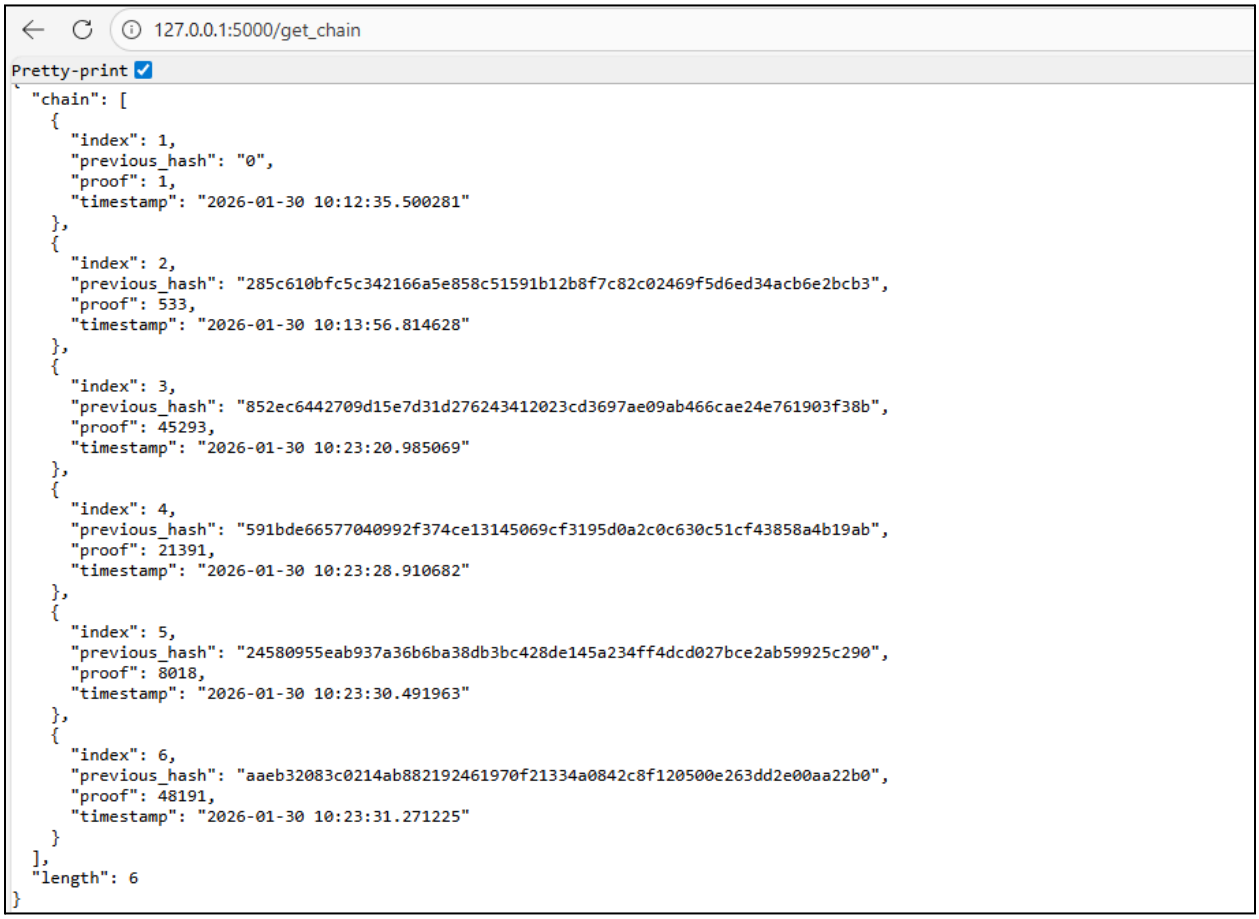
```
            nonce += 1

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]

            # Check previous hash link
            if current['previous_hash'] != previous['hash']:
                return False

            # Recalculate and verify hash
            recalculated_hash = self.calculate_hash(current)
            if recalculated_hash != current['hash']:
                return False

            # Check difficulty rule
            if not current['hash'].startswith('0' * self.difficulty):
                return False

        return True
```

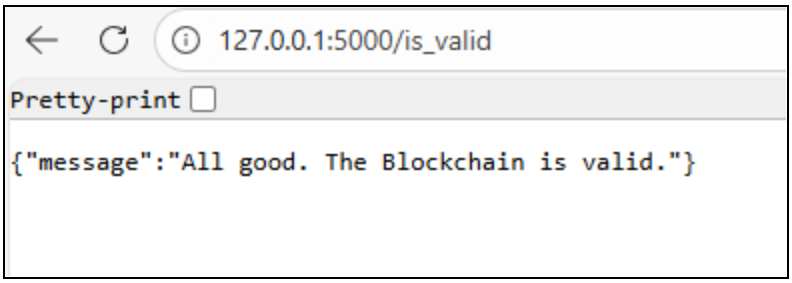
app.py

```
from flask import Flask, jsonify
from blockchain import Blockchain

app = Flask(__name__)
blockchain = Blockchain()


@app.route('/mine_block', methods=['GET'])
```

```
def mine_block():
    block = blockchain.mine_block("Some transaction data")
    return jsonify({
        'message': '⛏️ Block mined successfully!',
        'block': block
    }), 200


@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200


@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'is_valid': blockchain.is_chain_valid()
    }), 200


if __name__ == '__main__':
    print("⛏️ Mining blockchain with 4 leading zeroes...")
    app.run(debug=True, use_reloader=False)
```


127.0.0.1:5000/mine_block
Pretty-print
{
"index": 2,
"message": "Congratulations, you just mined a block!",
"previous_hash": "285c610bfc5c342166a5e858c51591b12b8f7c82c02469f5d6ed34acb6e2bcb3",
"proof": 533,
"timestamp": "2026-01-30 10:13:56.814628"
}


127.0.0.1:5000/get_chain
Pretty-print
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-01-30 10:12:35.500281"
},
{
"index": 2,
"previous_hash": "285c610bfc5c342166a5e858c51591b12b8f7c82c02469f5d6ed34acb6e2bcb3",
"proof": 533,
"timestamp": "2026-01-30 10:13:56.814628"
},
{
"index": 3,
"previous_hash": "852ec6442709d15e7d31d276243412023cd3697ae09ab466cae24e761903f38b",
"proof": 45293,
"timestamp": "2026-01-30 10:23:20.985069"
},
{
"index": 4,
"previous_hash": "591bde66577040992f374ce13145069cf3195d0a2c0c630c51cf43858a4b19ab",
"proof": 21391,
"timestamp": "2026-01-30 10:23:28.910682"
},
{
"index": 5,
"previous_hash": "24580955eab937a36b6ba38db3bc428de145a234ff4dcd027bce2ab59925c290",
"proof": 8018,
"timestamp": "2026-01-30 10:23:30.491963"
},
{
"index": 6,
"previous_hash": "aaeb32083c0214ab882192461970f21334a0842c8f120500e263dd2e00aa22b0",
"proof": 48191,
"timestamp": "2026-01-30 10:23:31.271225"
}
],
"length": 6
}



Conclusion:-

In this experiment, a simple blockchain was successfully built using Python. It clearly illustrated the essential concepts of blockchain, such as block structure, cryptographic hashing, Proof-of-Work mining, and chain validation.Through hash-linked blocks and strict difficulty-based verification rules, the system guarantees data integrity, immutability, and strong resistance to tampering. Even a minor change in any block breaks the entire chain, perfectly showcasing the decentralized and secure foundation of blockchain technology.

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.3

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

1. Blockchain Overview

Blockchain is a **distributed and decentralized digital ledger** that records transactions across a network of computers (nodes) in a secure, transparent, and tamper-resistant manner. It eliminates the need for a central authority by using cryptography and consensus.

Data is stored in a series of linked **blocks** forming a chronological chain. Each block typically contains:

4. A list of **transaction data** (e.g., sender, receiver, amount)
5. A **timestamp** indicating when the block was created
6. The **hash of the previous block** (creating the link)
7. Its own **unique hash** (a cryptographic digital fingerprint generated from the block's contents)

Once a block is added to the chain and confirmed by the network, its data becomes **immutable** — changing any information in a block would alter its hash, breaking the link to the next block and requiring recalculation of all subsequent blocks (which is computationally infeasible in a large network). This structure ensures security, transparency, and trust without intermediaries.

2. Mining

Mining is the process of validating transactions, creating new blocks, and securing the blockchain through computational work, primarily using the **Proof-of-Work (PoW)** consensus mechanism (as in Bitcoin).

Steps in mining:

- Miners collect **pending transactions** from the network (mempool) and group them into a candidate block.
- They perform a **computational puzzle** — repeatedly hashing the block header (including a variable nonce) until the resulting hash meets the network's difficulty target (e.g., starts with a required number of leading zeros).
- The first miner to find a valid hash **adds the new block** to their copy of the blockchain and **broadcasts** it to all peers for verification.
- If accepted, the block becomes part of the official chain.

Successful miners receive a **reward** (newly minted cryptocurrency + transaction fees) as an incentive for their work. Mining prevents double-spending, maintains decentralization, and makes tampering extremely expensive.

3. Multi-Node Blockchain Network

In a real blockchain (and in this lab simulation), the network operates as a **peer-to-peer (P2P)** system with multiple independent **nodes** (computers) instead of a central server.

In the lab setup:

3. Three nodes run on separate ports (e.g., 5001, 5002, 5003).

4. Each node maintains its **own full copy** of the blockchain ledger.
5. Nodes communicate directly with each other (P2P) to share new blocks, transactions, and chain information.
6. This decentralization ensures no single point of failure — if one node goes offline, others continue operating and can sync later.

Nodes validate incoming data and propagate valid information across the network, enabling global consensus without trusting any central entity.

4. Consensus Mechanism

Consensus is the process by which all nodes in a decentralized network agree on the valid state of the blockchain (i.e., which chain and transactions are legitimate).

In this lab (and Bitcoin), the mechanism used is the **Longest Chain Rule** (also called Nakamoto Consensus in PoW systems):

- When multiple valid chains exist (e.g., due to near-simultaneous block mining), nodes always adopt the **longest valid chain** (the one with the most accumulated proof-of-work / blocks).
- This rule resolves conflicts automatically — shorter/forked chains are eventually discarded as miners continue building on the longest one.
- It ensures eventual agreement on a single transaction history across the entire network.

This simple yet powerful rule achieves reliable consensus in a trustless environment.

5. Transactions & Mining Reward

A **transaction** in blockchain is a record of value transfer between parties, typically including:

- **Sender** address
- **Receiver** address
- **Amount** transferred
- Digital signature (to prove ownership)

Transactions are collected in the mempool, verified, and included in blocks during mining.

When a miner successfully creates a new block:

- All selected pending transactions are added to the block body.
- An automatic **reward transaction** (coinbase transaction) is included as the first transaction in the block.
- This coinbase transaction pays the miner a fixed **block reward** (newly created coins) plus all **transaction fees** paid by users.

The reward incentivizes miners to secure the network and introduces new coins into circulation in a controlled manner.

6. Chain Replacement

In a decentralized network, nodes may temporarily have different chain versions due to network delays or simultaneous mining.

The **/replace_chain** endpoint (or equivalent resolve function) implements conflict resolution:

7. The node requests the current blockchain from all connected peers.

8. It compares the lengths (and validity) of received chains against its own.
9. If a **longer and valid chain** is found (following the longest chain rule), the node replaces its local chain with the longer one.
10. The node discards its shorter/forked chain and adopts the majority-accepted version.

This process keeps the blockchain synchronized and consistent across all nodes, ensuring everyone eventually agrees on the same transaction history.

CODE:-

```
import datetime,hashlib,json

from flask import Flask,jsonify,request

import requests

from uuid import uuid4

from urllib.parse import urlparse

class Blockchain:

    def __init__(self):

        self.chain=[]

        self.transactions=[]

        self.create_block(proof=1,previous_hash='0')

        self.nodes=set()


    def create_block(self,proof,previous_hash):
```

```
block={'index':len(self.chain)+1,'timestamp':str(datetime.datetime.now()),'proof':proof,'previous_hash':previous_hash,'transactions':self.transactions}
        self.transactions=[]
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self,previous_proof):
        new_proof=1
        check_proof=False
        while not check_proof:
hash_operation=hashlib.sha256(str(new_proof**2-previous_proof**2).encode()).hexdigest()
            if hash_operation[:4]=='0000':check_proof=True
            else:new_proof+=1
        return new_proof

    def hash(self,block):
        encoded_block=json.dumps(block,sort_keys=True).encode()
        return hashlib.sha256(encoded_block).hexdigest()
```

```
    def is_chain_valid(self,chain):
        previous_block=chain[0]
        block_index=1
        while block_index<len(chain):
            block=chain[block_index]
            if block['previous_hash']!=self.hash(previous_block):return False
            previous_proof=previous_block['proof']
            proof=block['proof']

hash_operation=hashlib.sha256(str(proof**2-previous_proof**2).encode()).hexdigest()
            if hash_operation[:4]!='0000':return False
            previous_block=block
            block_index+=1
        return True

    def add_transaction(self,sender,receiver,amount):
        self.transactions.append({'sender':sender,'receiver':receiver,'amount':amount})
        previous_block=self.get_previous_block()
        return previous_block['index']+1

    def add_node(self,address):
        parsed_url=urlparse(address)
```

```
        self.nodes.add(parsed_url.netloc)

    def replace_chain(self):
        network=self.nodes
        longest_chain=None
        max_length=len(self.chain)
        for node in network:
            response=requests.get(f'http://{node}/get_chain')
            if response.status_code==200:
                length=response.json()['length']
                chain=response.json()['chain']
                if length>max_length and self.is_chain_valid(chain):
                    max_length=length
                    longest_chain=chain
        if longest_chain:
            self.chain=longest_chain
            return True
        return False

app=Flask(__name__)
node_address=str(uuid4()).replace('-','')
blockchain=Blockchain()
```

```
@app.route('/mine_block',methods=['GET'])

def mine_block():

    previous_block=blockchain.get_previous_block()

    previous_proof=previous_block['proof']

    proof=blockchain.proof_of_work(previous_proof)

    previous_hash=blockchain.hash(previous_block)

    blockchain.add_transaction(sender=node_address,receiver='Richard',amount=1)

    block=blockchain.create_block(proof,previous_hash)

    response={'message':'Congratulations,you just mined a
block!','index':block['index'],'timestamp':block['timestamp'],'proof':block['proof'],'previou
s_hash':block['previous_hash'],'transactions':block['transactions']}

    return jsonify(response),200

@app.route('/add_transaction',methods=['POST'])

def add_transaction():

    json_data=request.get_json()

    transaction_keys=['sender','receiver','amount']

    if not all(key in json_data for key in transaction_keys):return 'Some elements of the
transaction are missing',400


index=blockchain.add_transaction(json_data['sender'],json_data['receiver'],json_data['am
ount'])

    response={'message':f'This transaction will be added to Block {index}'}

    return jsonify(response),201

@app.route('/connect_node',methods=['POST'])
```

```
def connect_node():
    json_data=request.get_json()
    nodes=json_data.get('nodes')
    if nodes is None:return "No node",400
    for node in nodes:blockchain.add_node(node)
    response={'message':'All the nodes are now
connected.','total_nodes':list(blockchain.nodes)}
    return jsonify(response),201
@app.route('/replace_chain',methods=['GET'])
def replace_chain():
    is_chain_replaced=blockchain.replace_chain()
    if is_chain_replaced:
        response={'message':'The chain was replaced by the longest
one.','new_chain':blockchain.chain}
    else:
        response={'message':'The chain is already the largest
one.','actual_chain':blockchain.chain}
    return jsonify(response),200
app.run(host='0.0.0.0',port=5000)
```

OUTPUT:-

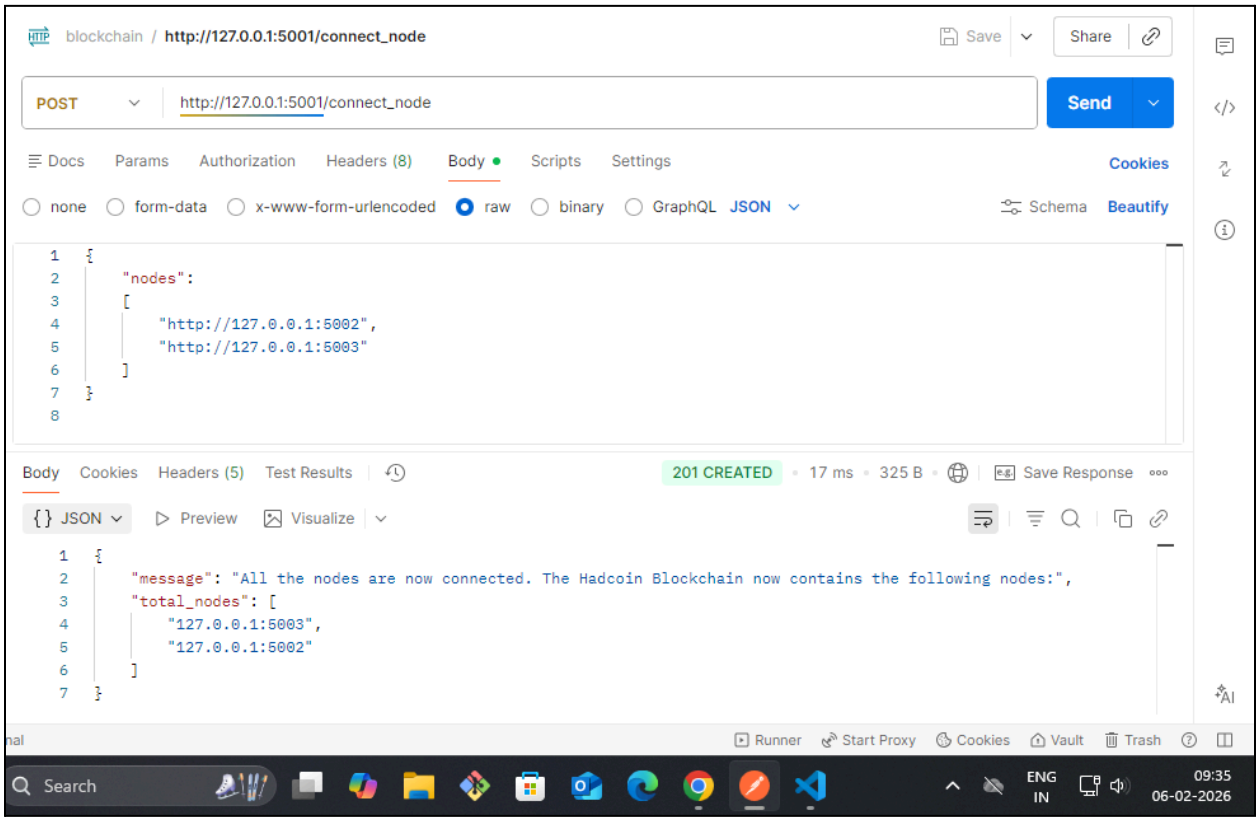
1. Connect Nodes (POST)

URL (from any node):

POST http://127.0.0.1:5001/connect_node

Body → raw → JSON

```
{
    "nodes":
    [
        "http://127.0.0.1:5002",
        "http://127.0.0.1:5003"
    ]
}
```



2. Add Transaction (POST)

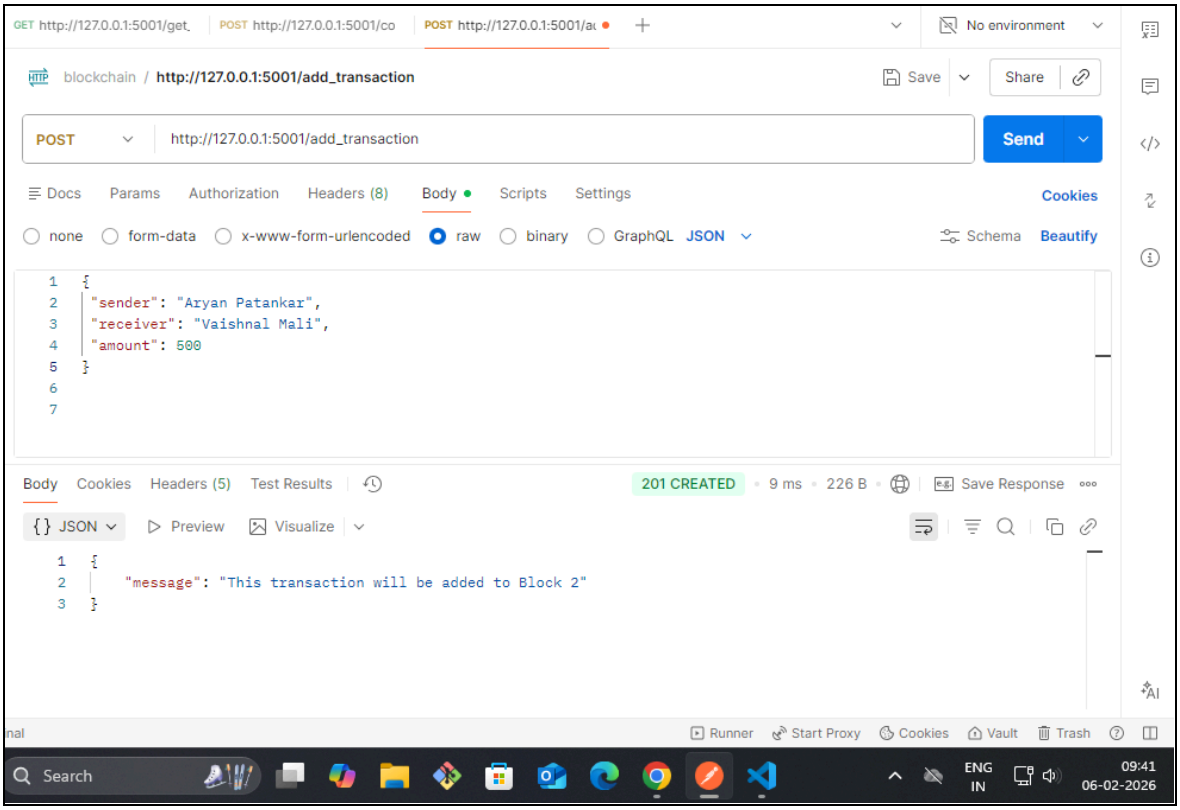
URL

POST http://127.0.0.1:5001/add_transaction

Body

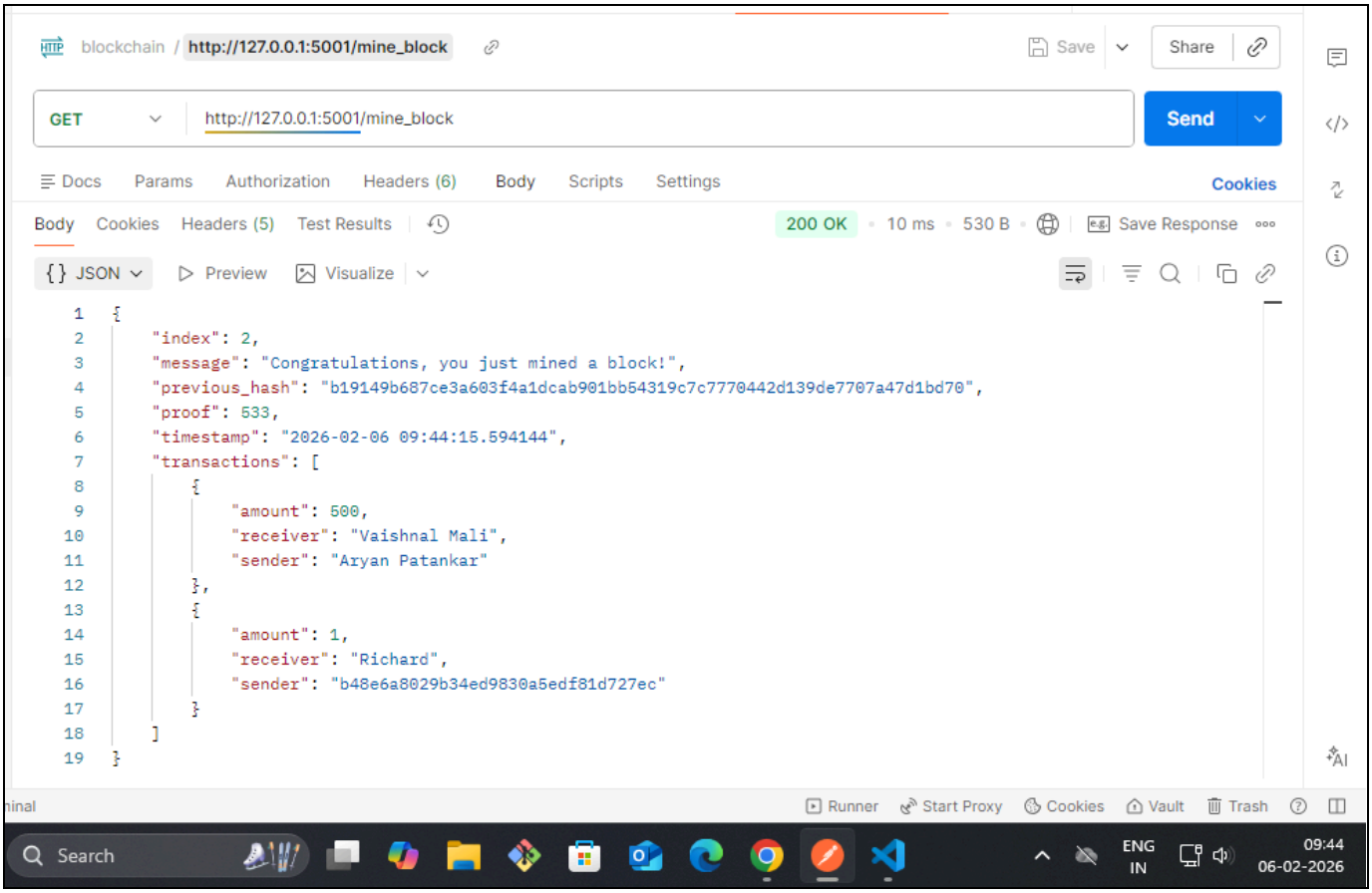

```
{
 "sender": "Aryan Patankar",
 "receiver": "Vaishnal Mali",
 "amount": 500
}
```


Transaction goes into **mempool**, NOT block yet



3. Mine Block (GET)

GET http://127.0.0.1:5001/mine_block

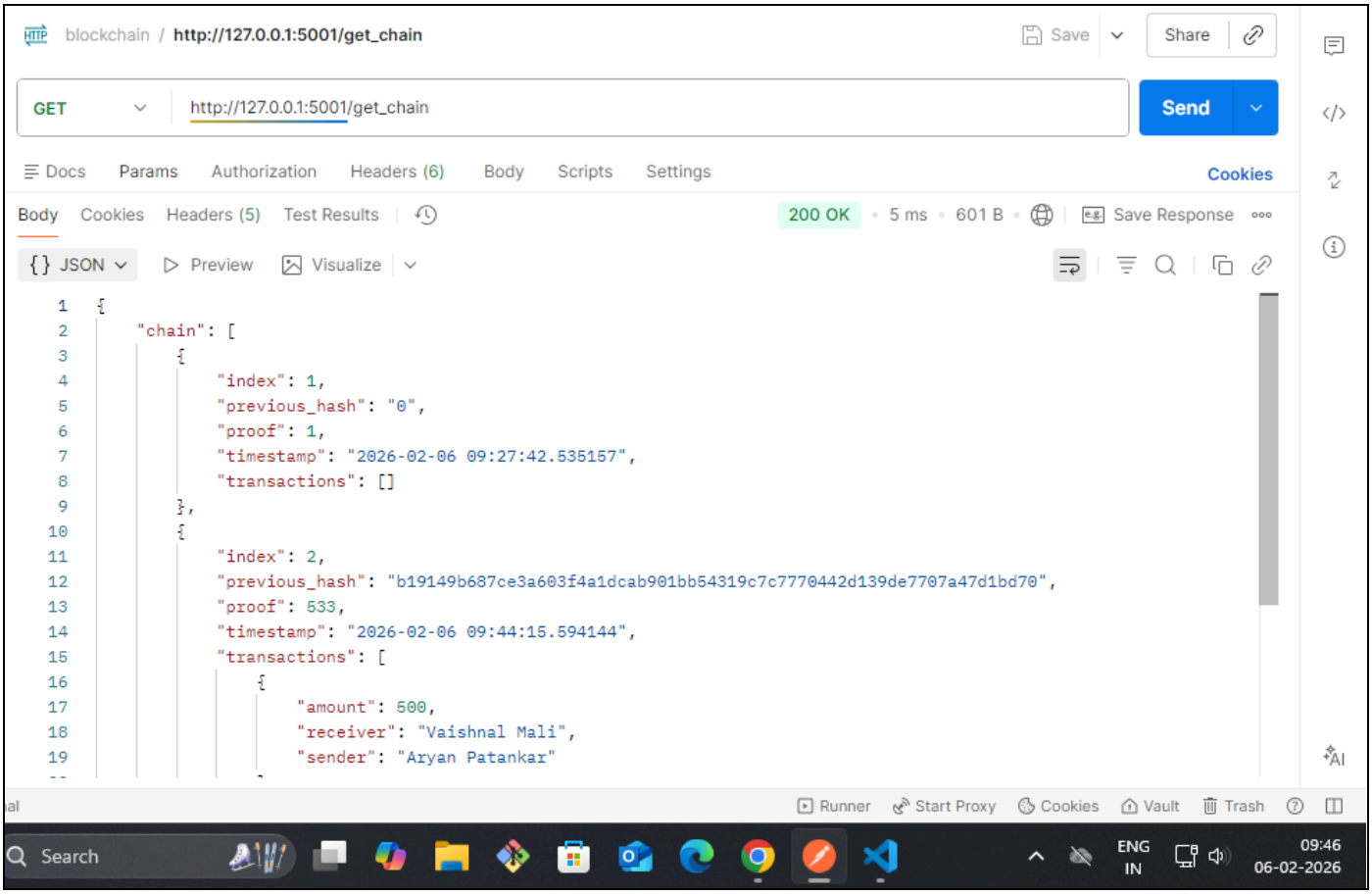


4. Get Blockchain (GET)

GET http://127.0.0.1:5001/get_chain

You'll see:

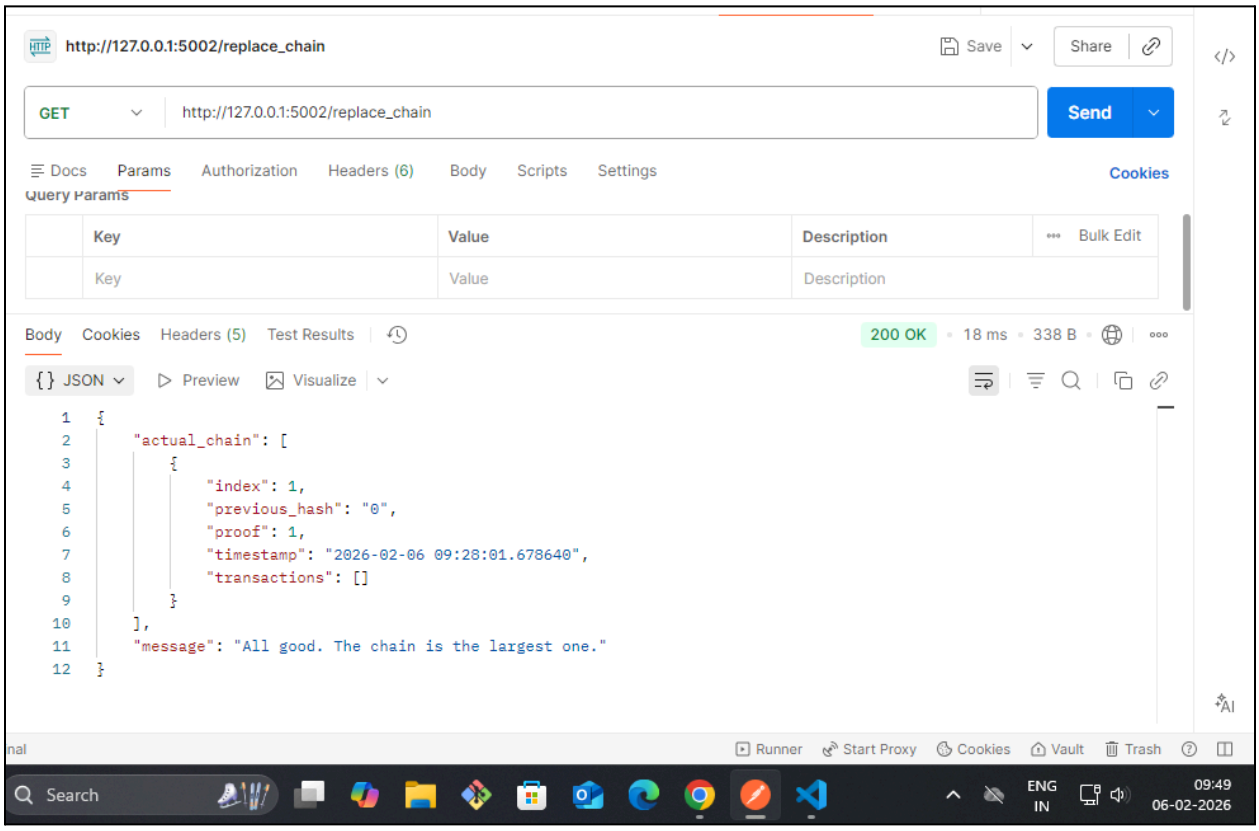
- Transactions inside blocks
- transactions: [] for new pending list


blockchain / http://127.0.0.1:5001/get_chain
Save
Share
GET
http://127.0.0.1:5001/get_chain
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Settings
Cookies
Body
Cookies
Headers (5)
Test Results
200 OK
5 ms
601 B
Save Response
JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-02-06 09:27:42.535157",
"transactions": []
},
{
"index": 2,
"previous_hash": "b19149b687ce3a603f4a1dcab901bb54319c7c7770442d139de7707a47d1bd70",
"proof": 533,
"timestamp": "2026-02-06 09:44:15.594144",
"transactions": [
{
"amount": 500,
"receiver": "Vaishnal Mali",
"sender": "Aryan Patankar"
Runner
Start Proxy
Cookies
Vault
Trash
Search
ENG
IN
09:46
06-02-2026

5. Replace Chain (Consensus)

GET http://127.0.0.1:5002/replace_chain

Shorter chains get replaced by the longest valid one



CONCLUSION:

In this experiment, a simple cryptocurrency was successfully created using Python by implementing core blockchain concepts such as block creation, hashing, Proof-of-Work mining, transactions, and decentralization. The blockchain system was developed using Flask, allowing multiple nodes to communicate and maintain their own copies of the blockchain. Mining was performed by solving the Proof-of-Work algorithm, through which new blocks were added to the blockchain and miners were rewarded with cryptocurrency. The experiment also demonstrated transaction handling, where transactions were verified and included in mined blocks. Overall, this experiment

provided a practical understanding of how blockchain technology works, including mining, decentralization, and consensus, and highlighted how cryptocurrencies operate in real-world distributed systems.

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT NO.4

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim:Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:-

1. Primitive Data Types, Variables, Functions - pure, view

Primitive (Value) Data Types in Solidity include:

bool → true/false
int / uint (signed/unsigned integers, e.g., uint256 is most common)
address → 20-byte Ethereum address
bytes1 to bytes32 → fixed-size byte arrays
string → dynamic UTF-8 encoded text (reference type, but often grouped here)

Variables are declared with a type and can be:

State variables → stored permanently on the blockchain (expensive)
Local variables → exist only during function execution

Functions can be marked as:

pure → does not read or write state (computes only from inputs; cheapest gas)

view → reads state but does not modify it (e.g., getters; no gas when called externally)

Example

```
function getResult() public view returns (uint product, uint sum) {
   product = num1 * num2;  // reads state
   sum = num1 + num2;
}

function pureCalc(uint a, uint b) public pure returns (uint) {
   return a + b;  // no state access
}
```

2. Inputs and Outputs to Functions

Functions in Solidity can take **parameters** (inputs) and return **values** (outputs).

Inputs: Declared in parentheses; can use data locations like memory or calldata for reference types.
Outputs: Declared after returns keyword; can return multiple values.

Example:

```
function add(uint a, uint b) public pure returns (uint sum) {
   sum = a + b;
}
```

```
// Multiple outputs
function getValues() public view returns (uint, string memory) {
   return (age, name);
}
```

Use calldata for external calls (cheaper, read-only).
Use memory for temporary copies inside functions.

3. Visibility, Modifiers and Constructors

Visibility Specifiers (for functions and state variables):

- **public** → anyone can call/read (default for state vars creates getter)
- **private** → only inside current contract
- **internal** → current + derived (child) contracts
- **external** → only external calls (cheaper for large data)

Modifiers → reusable code blocks that run before/after function body (e.g., access control).

Use _; to insert function body.

```
modifier onlyOwner() {
   require(msg.sender == owner, "Not owner");
   _;
}

function restricted() public onlyOwner { ... }
```

Constructors → special function that runs once on deployment (initializes state).

- Syntax: constructor() { ... } (older: same name as contract)

4. Control Flow: if-else, loops

Solidity supports standard control structures:

if-else:

```
if (condition) {
   // true
} else if (another) {
   // else-if
} else {
   // false
}
```

Loops:

```
for (uint i = 0; i < 10; i++) { ... }
while (condition) { ... }
do { ... } while (condition);
```

Avoid unbounded loops (gas limit risk). Use break / continue when needed.

5. Data Structures: Arrays, Mappings, structs, enums

- **Arrays**
    - Fixed: uint[5] arr;

    - Dynamic: uint[] arr; (use .push(), .pop(), .length)
- **Mappings** → key-value store (like hash table)

  mapping(address => uint) public balances;

  Keys can be most types; no iteration possible.

Structs → custom composite types

```
struct User {
   address wallet;
   uint balance;
   bool active;
}
User public owner;
```

Enums → named constants (integers under the hood)

```
enum Status { Pending, Active, Cancelled }
Status public state = Status.Pending;
```

6. Data Locations

Solidity has three main **data locations** for reference types (arrays, structs, mappings, strings):

storage → permanent blockchain storage (persistent, expensive) Default for state variables.

memory → temporary, function lifetime only (deleted after execution) Cheap; used for local variables & function args/returns.

calldata → read-only, non-modifiable area for function call data
Cheapest for external function parameters (immutable copy of tx data).

Rule:

State vars → always storage
Function args → prefer calldata (external) or memory
Local reference vars → must specify location

7. Transactions: Ether and wei, Gas and Gas Price, Sending Transactions

8. **Ether units** → smallest is **wei** (1 ETH = 10^{18} wei) Other: gwei (10^{9} wei), commonly used for gas prices.
9. **Gas** → computational effort unit
    - **Gas Limit** → max gas willing to spend (set by sender)
    - **Gas Price** → price per gas unit (in wei/gwei; set by sender)
    - Total fee = gas used × gas price
10. **msg.value** → amount of wei sent with transaction
11. **Sending Ether** → use .transfer(), .send(), or .call{value: amount}("") (recommended low-level call)

Example:

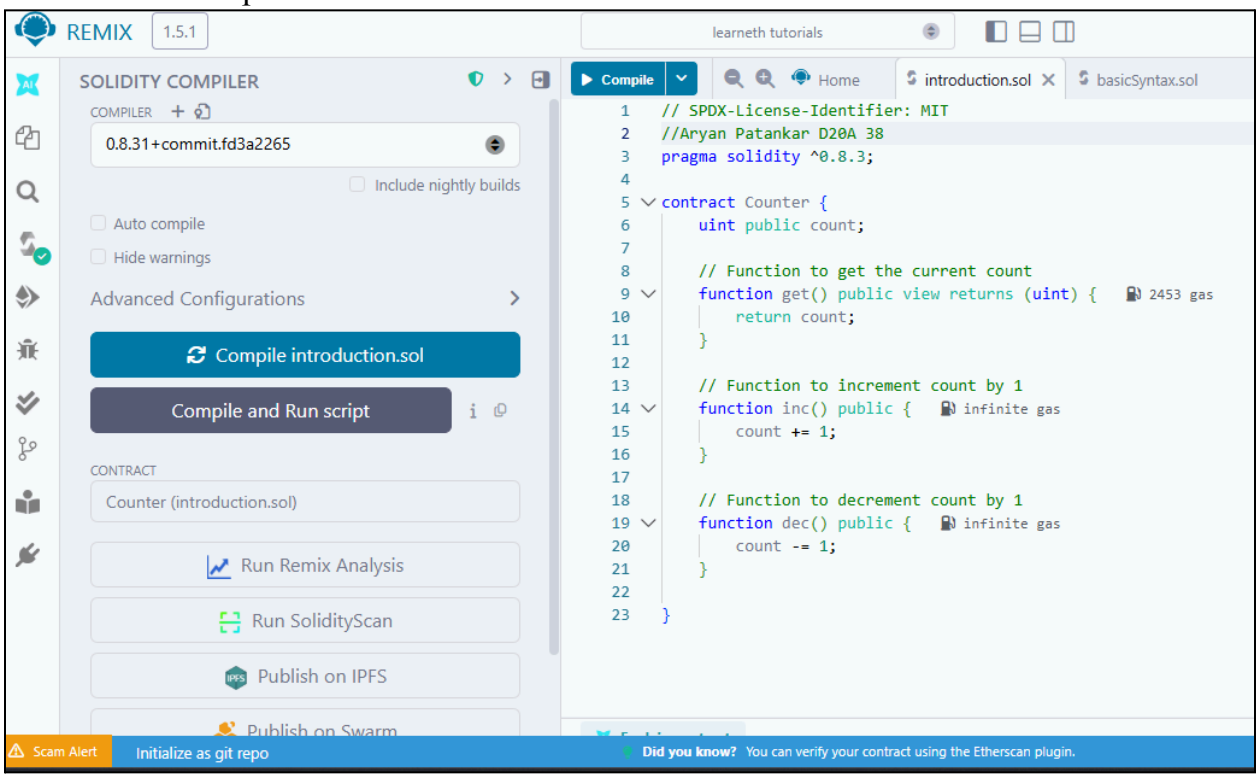
```
function sendEther(address payable recipient) public payable {
   recipient.transfer(msg.value);
}
```

Use payable for addresses/functions that receive Ether.

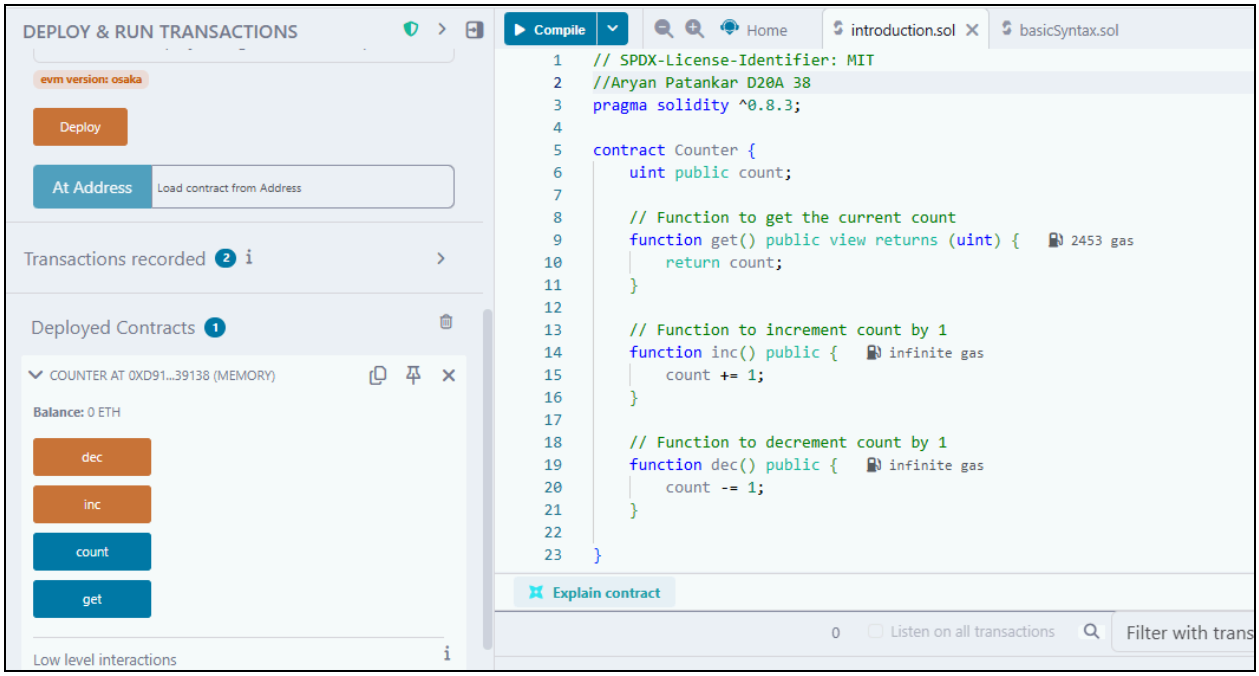
tx.gasprice → current gas price (global var)

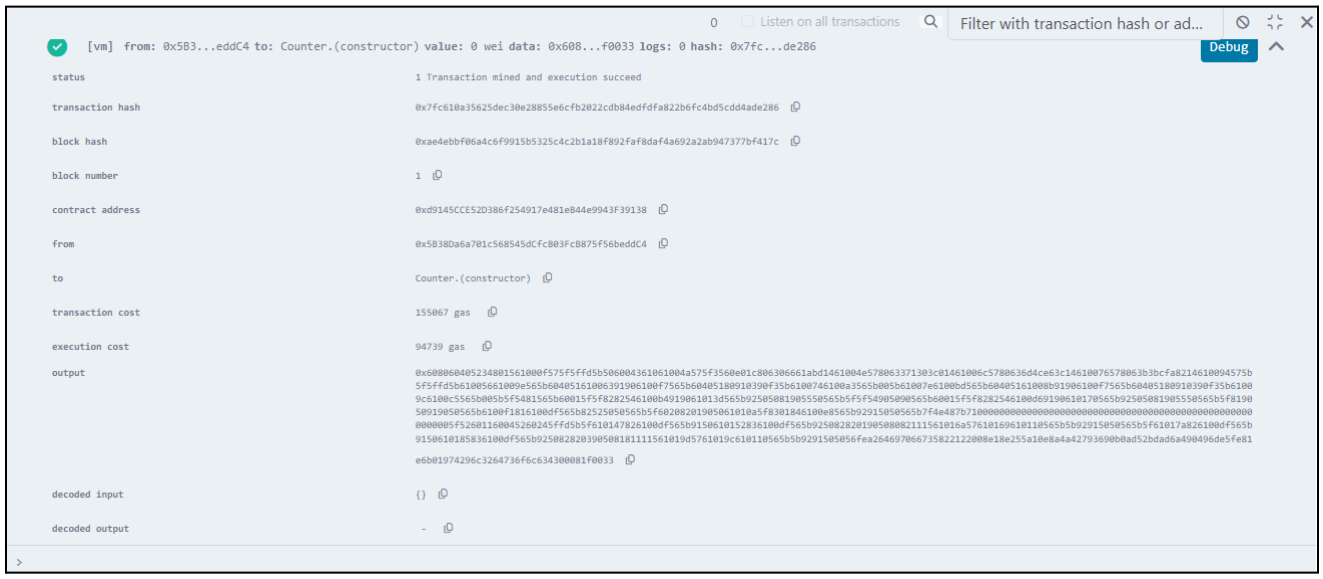
Implementation:-

Tutorial no. 1 – Compile the code

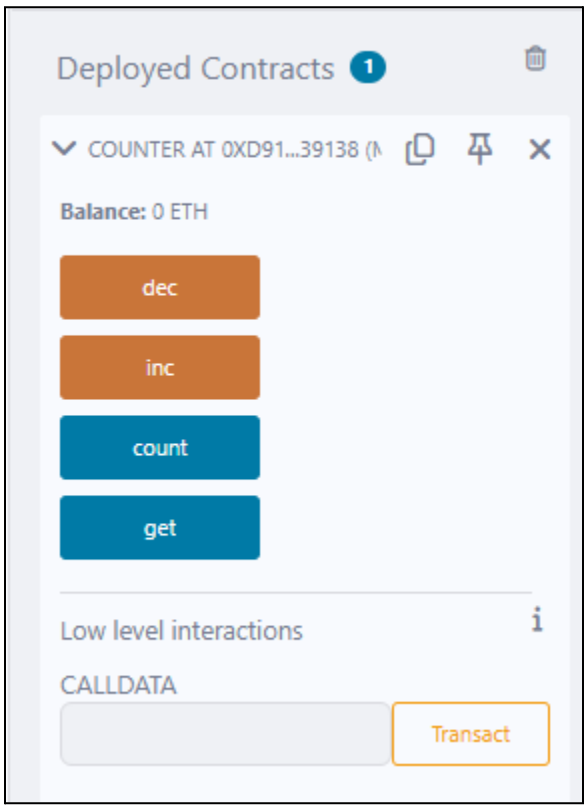


7. Tutorial No.1-Deploy the contract

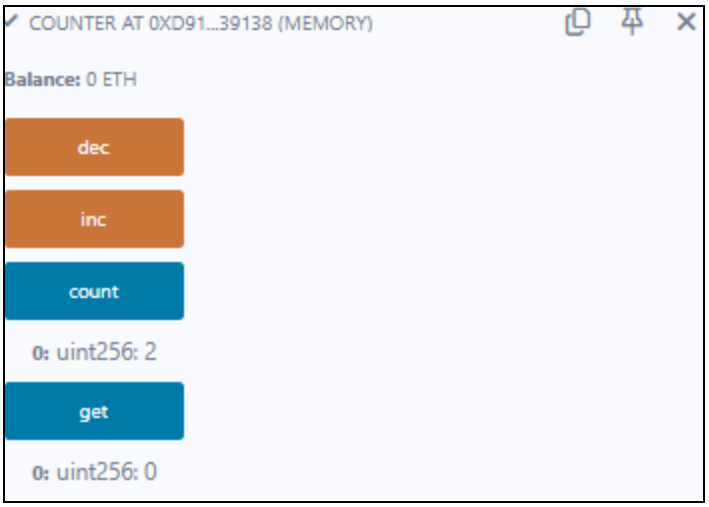




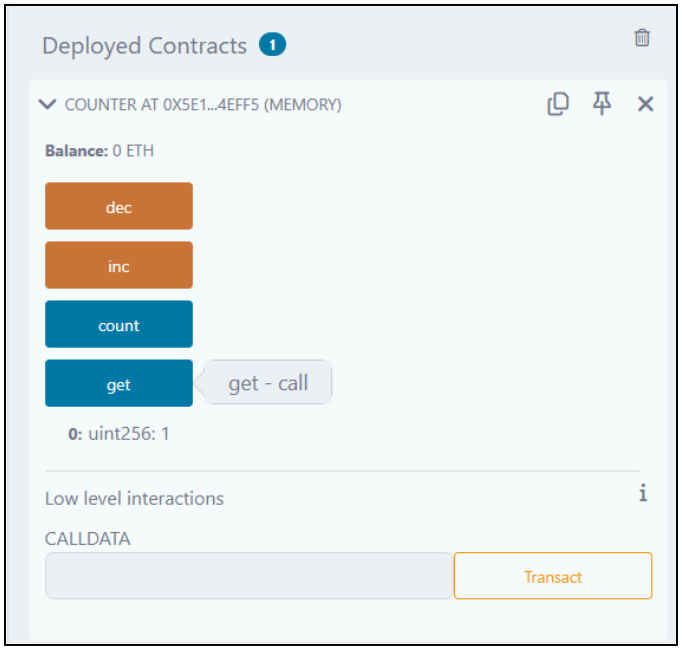
- Tutorial no. 1 – get



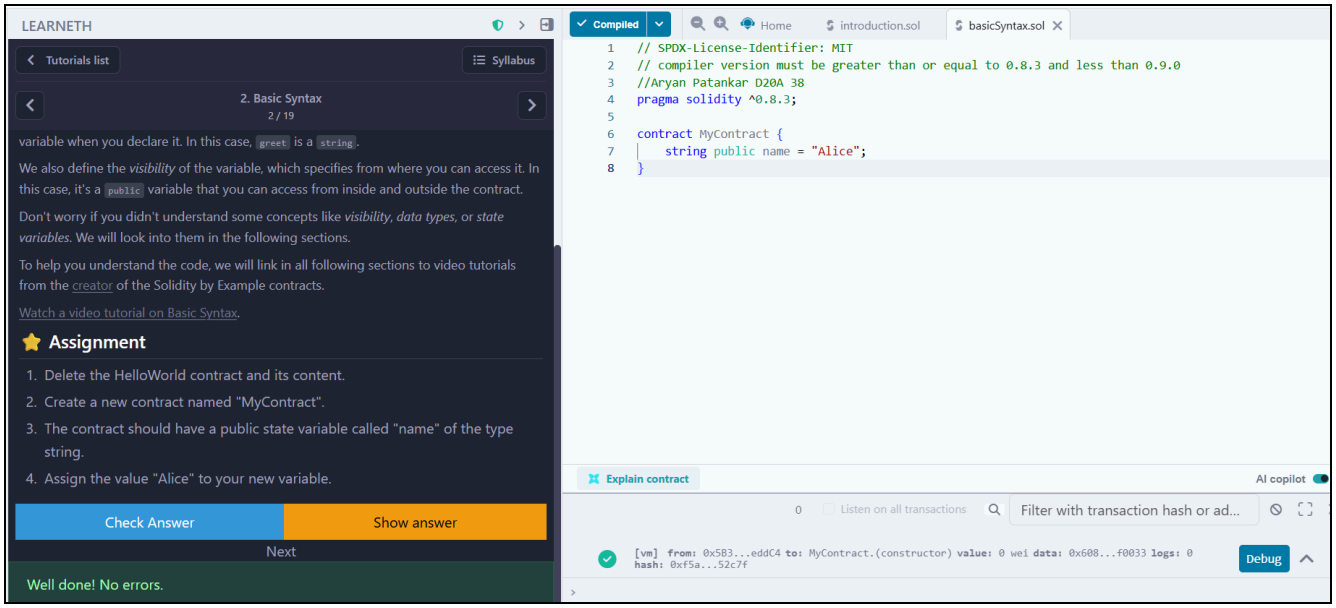
11. Tutorial no. 1 – Increment



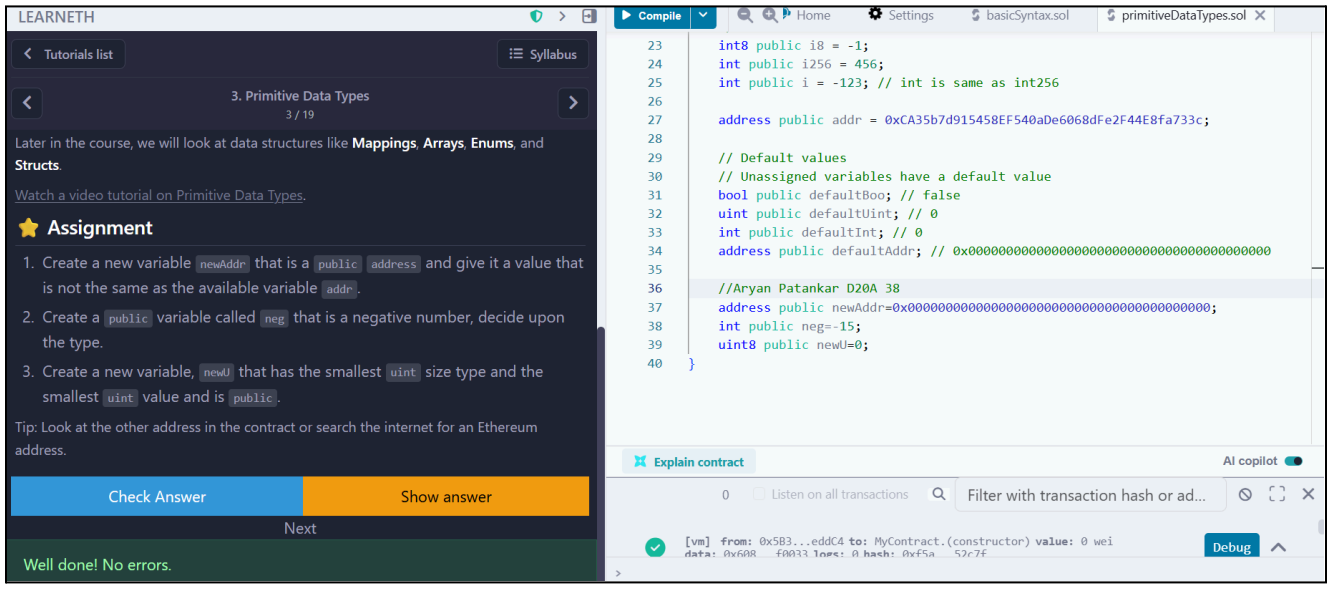
Tutorial no. 1 – Decrement



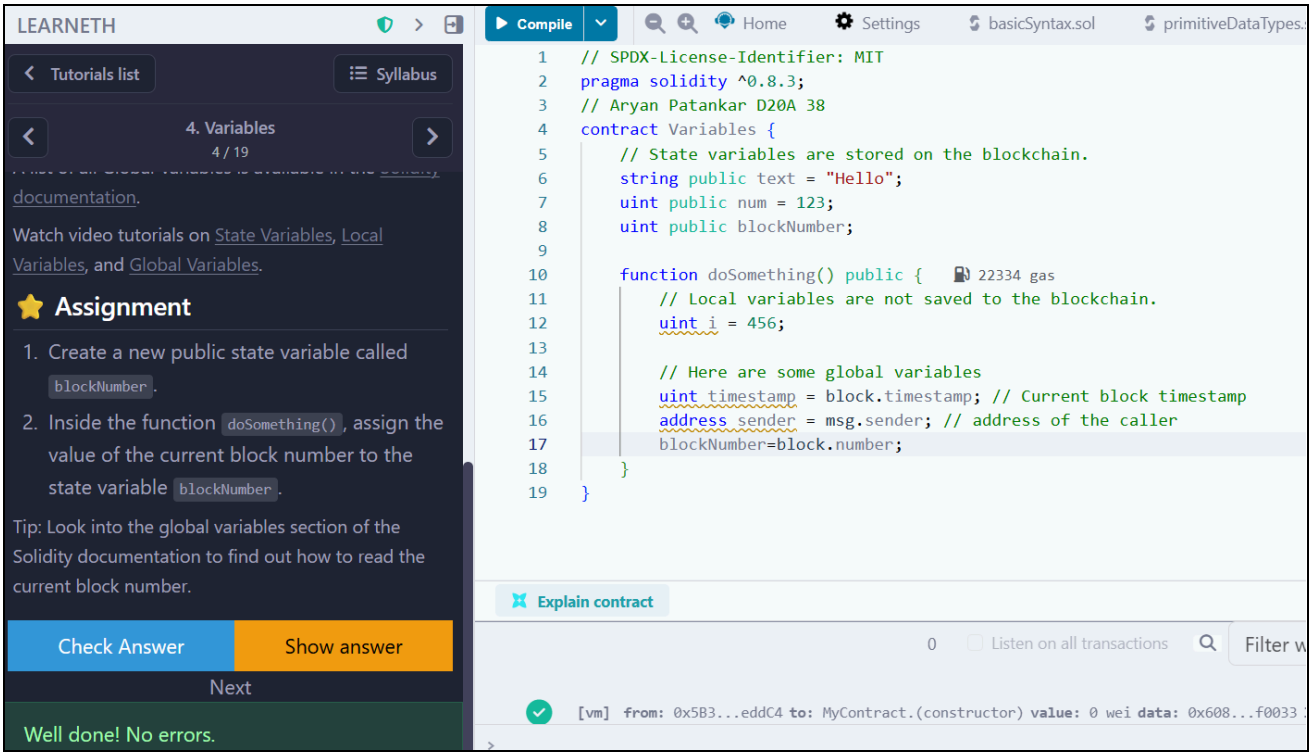
Tutorial no.2



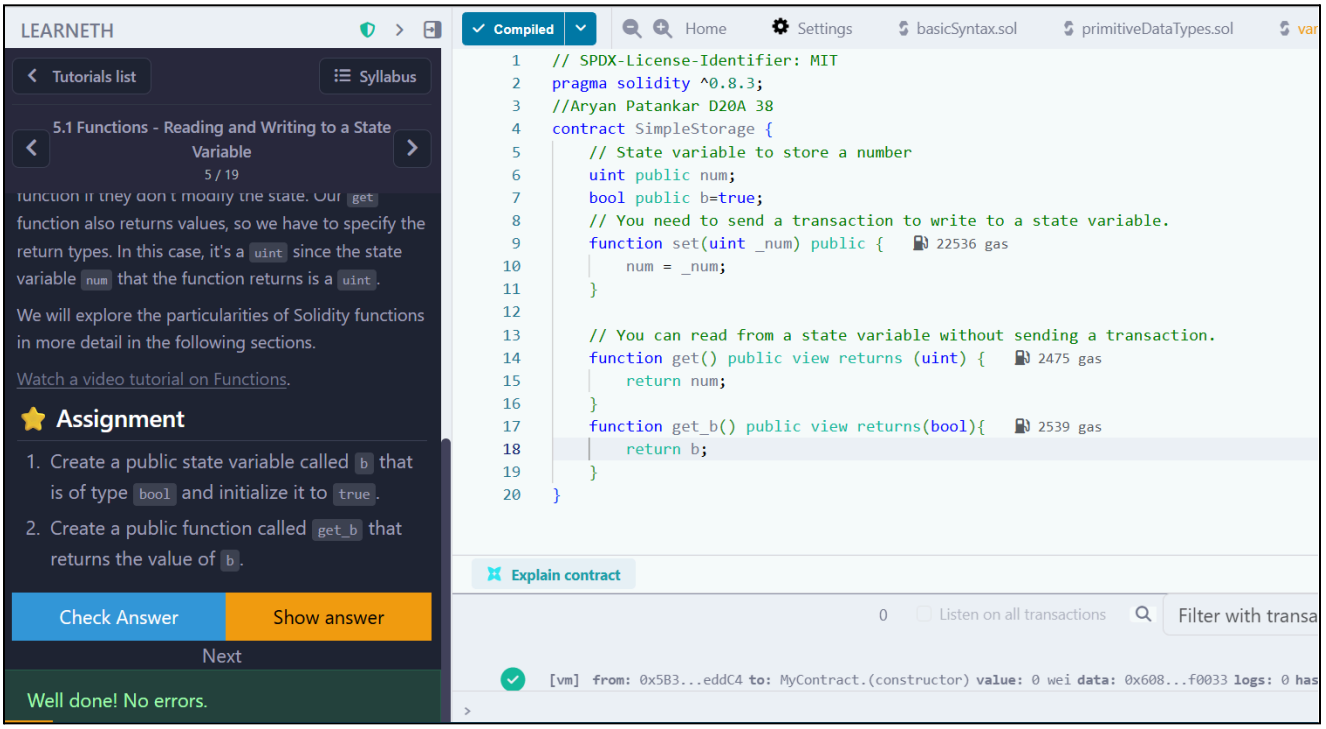
- Tutorial no.3



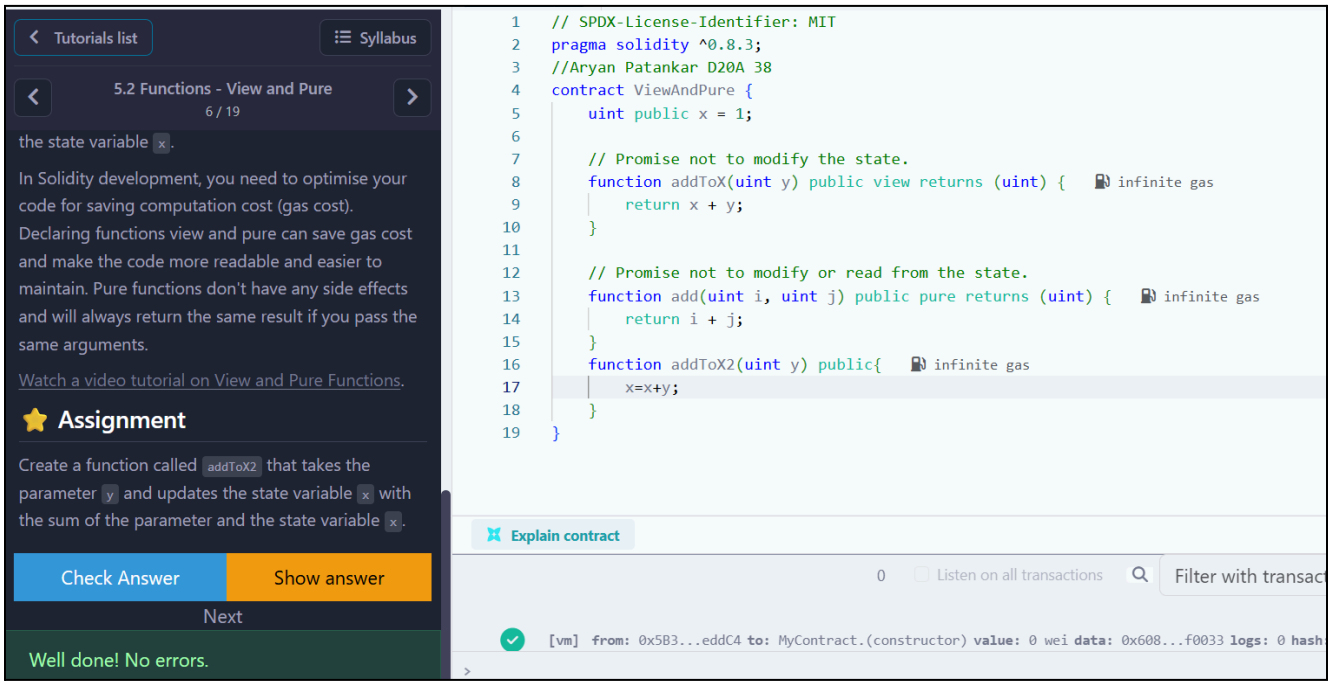
Tutorial no.4



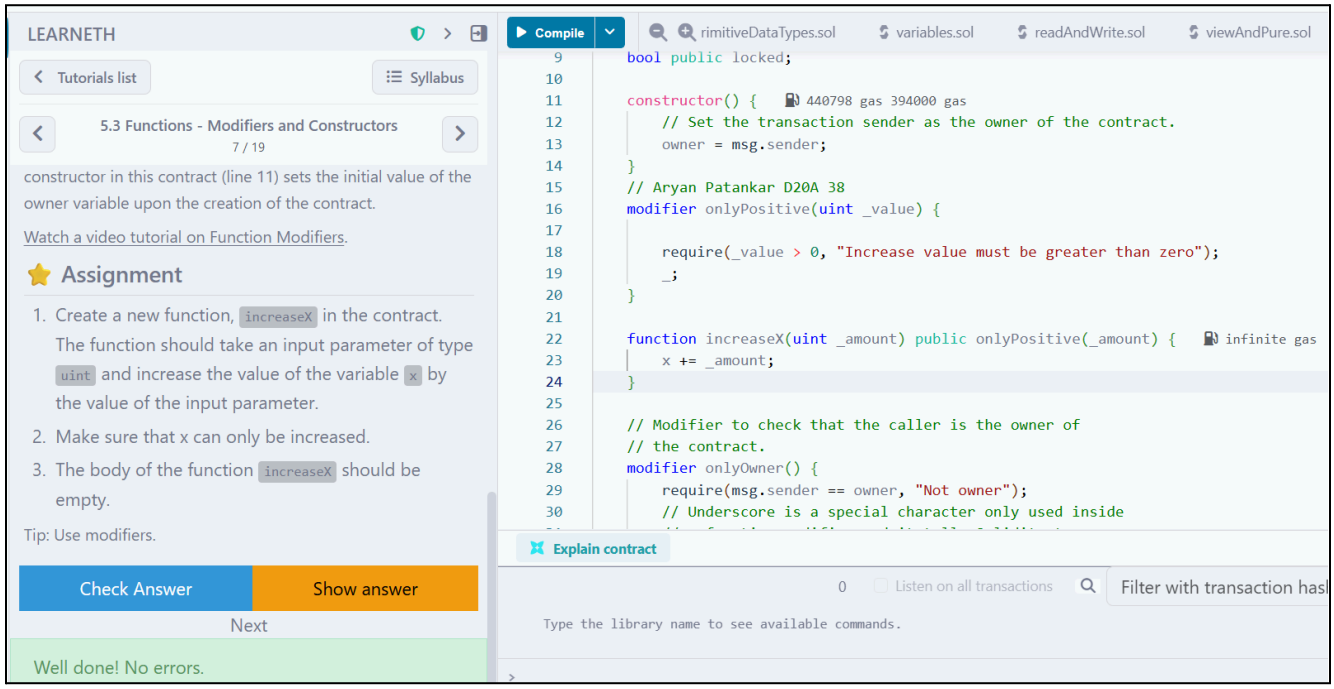
Tutorial no.5



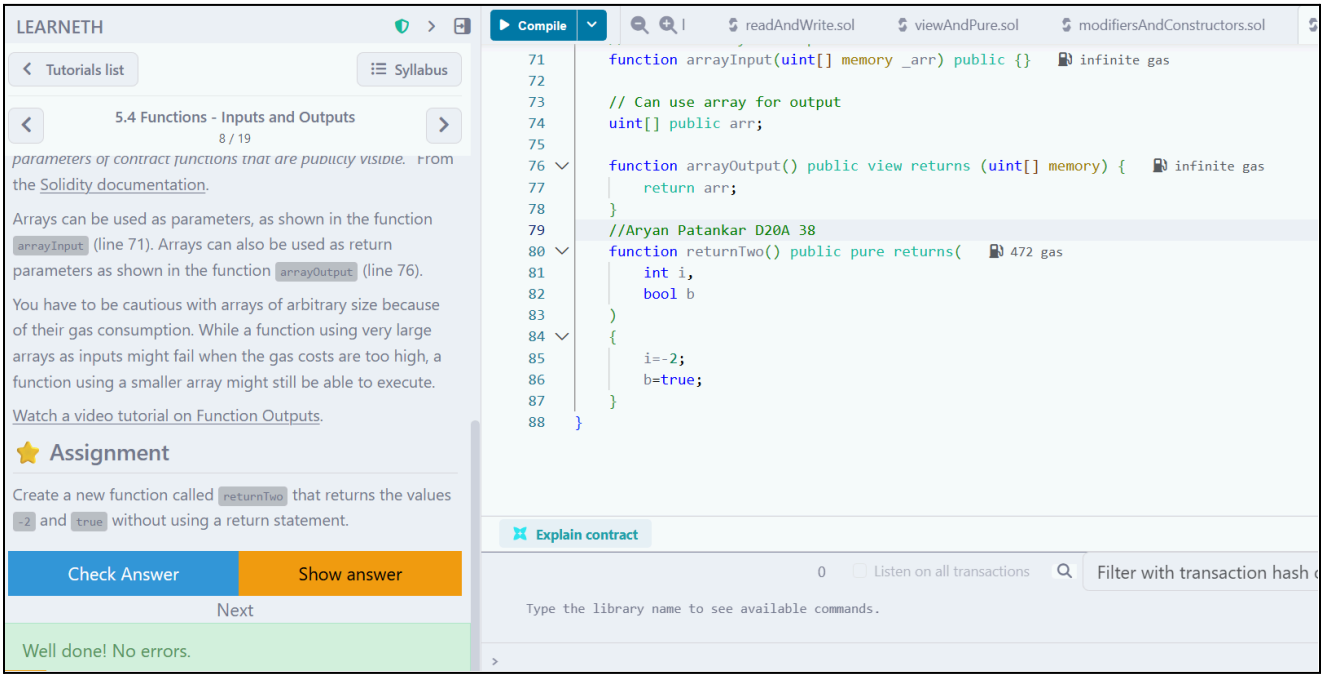
3. Tutorial no.6



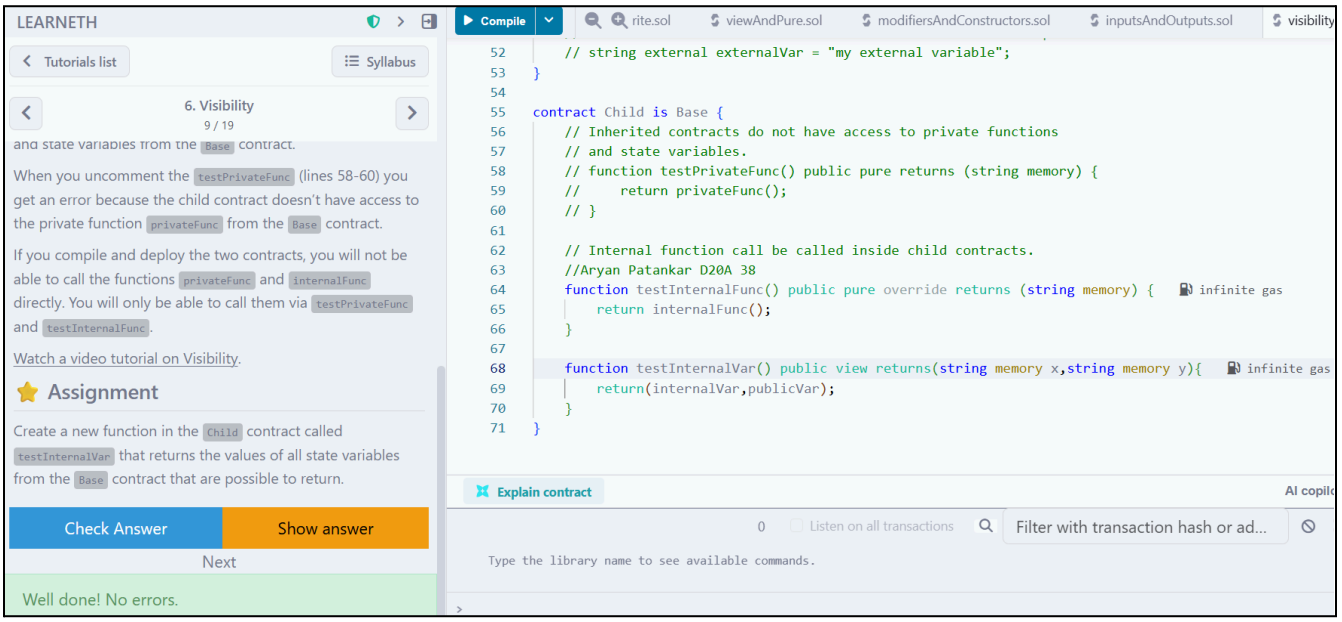
Tutorial no.7



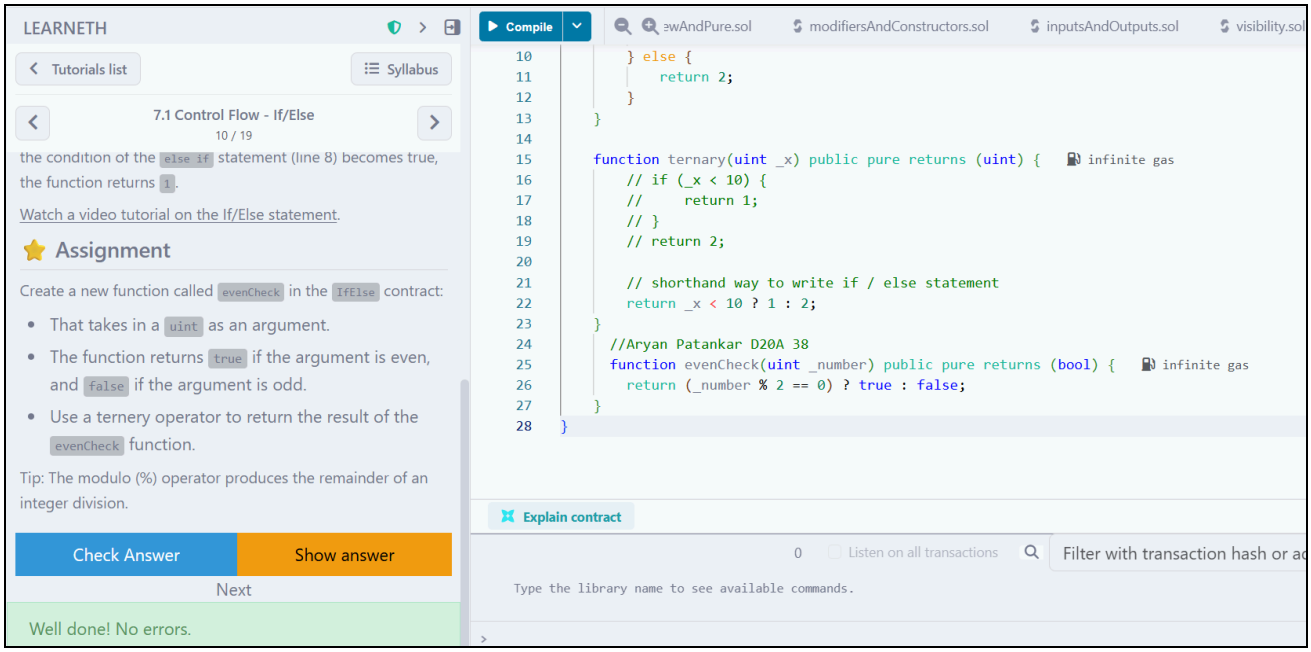
Tutorial no.8



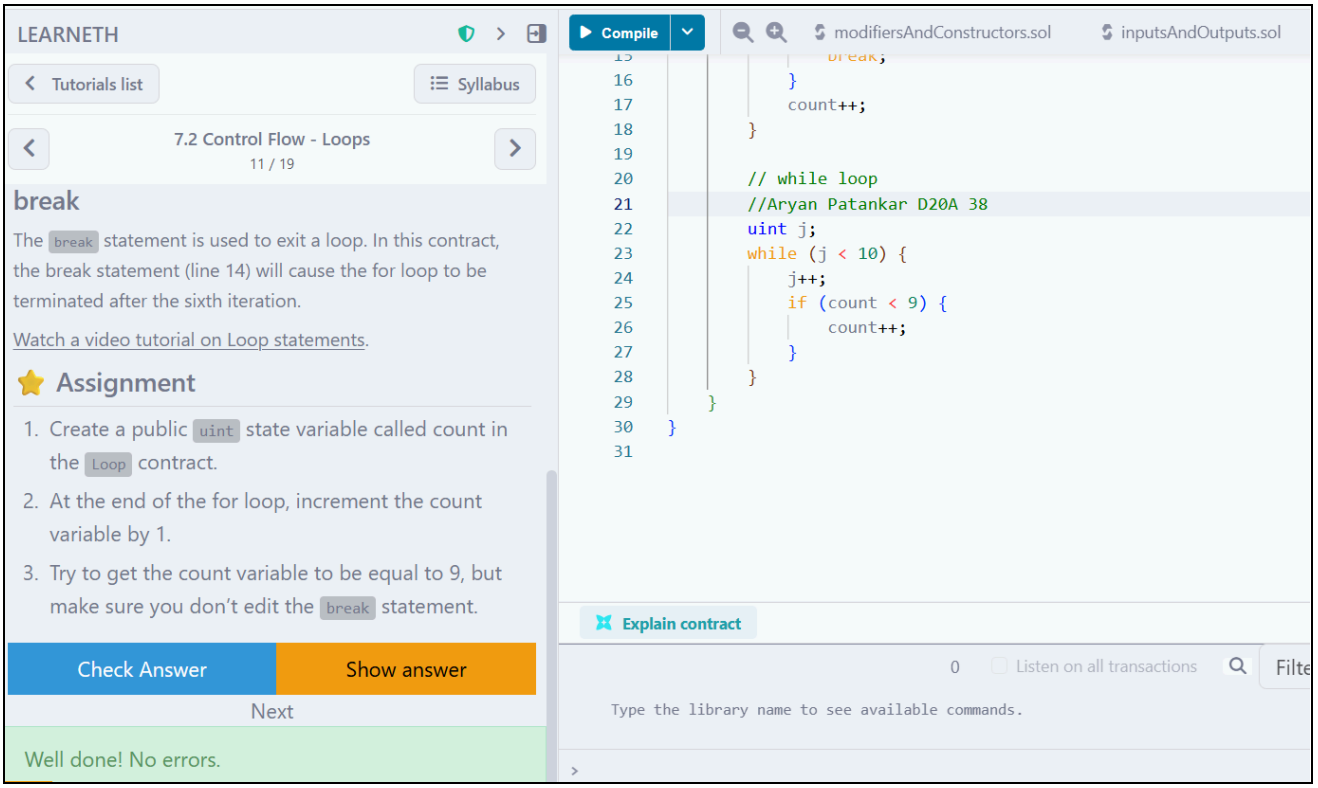
Tutorial no.9



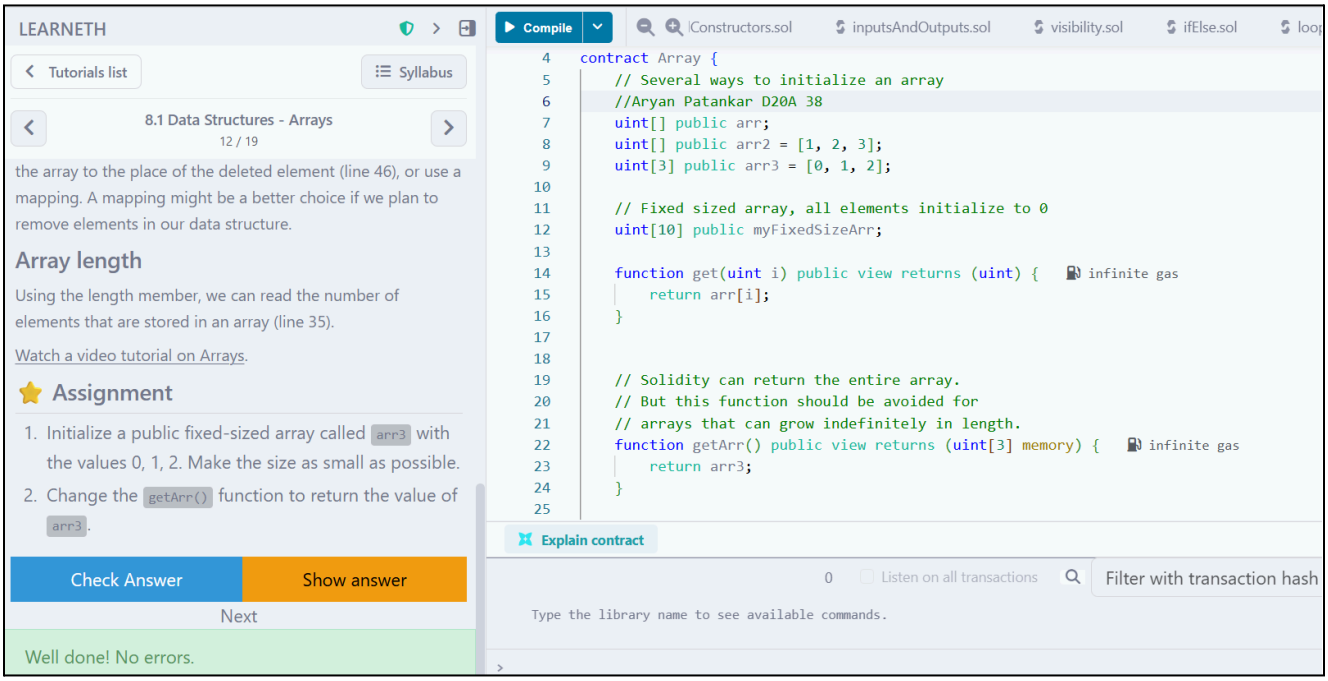
Tutorial no.10



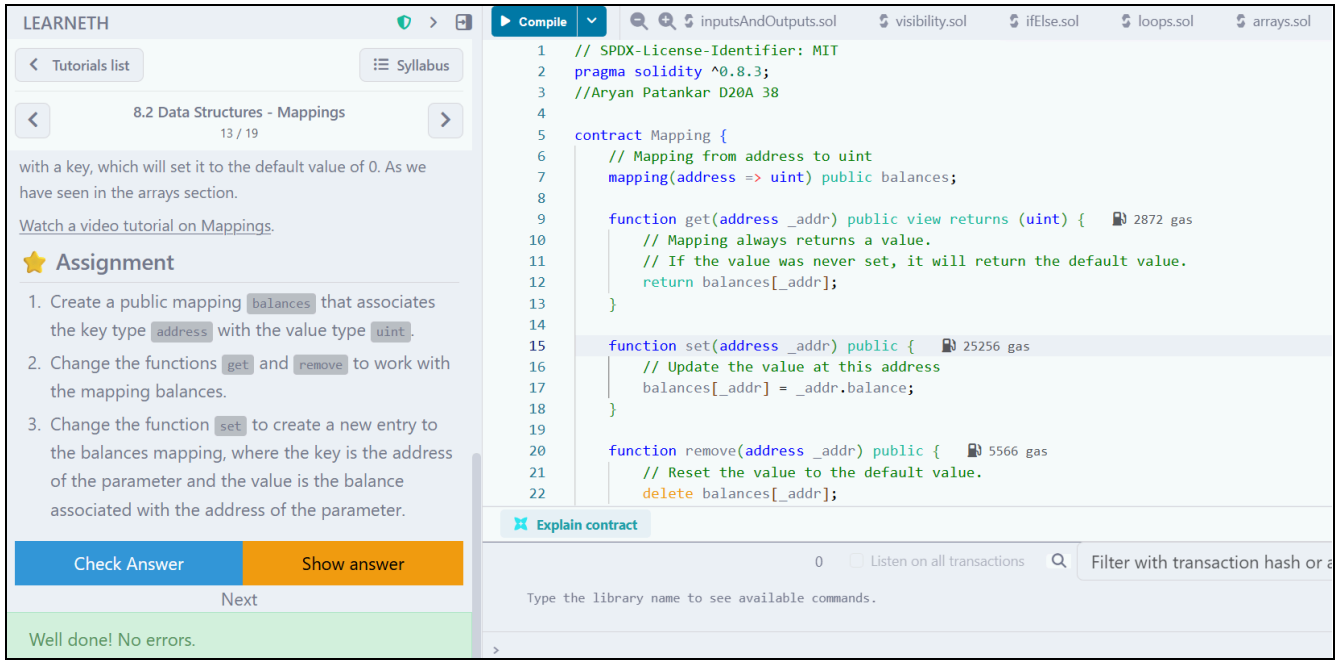
Tutorial no.11



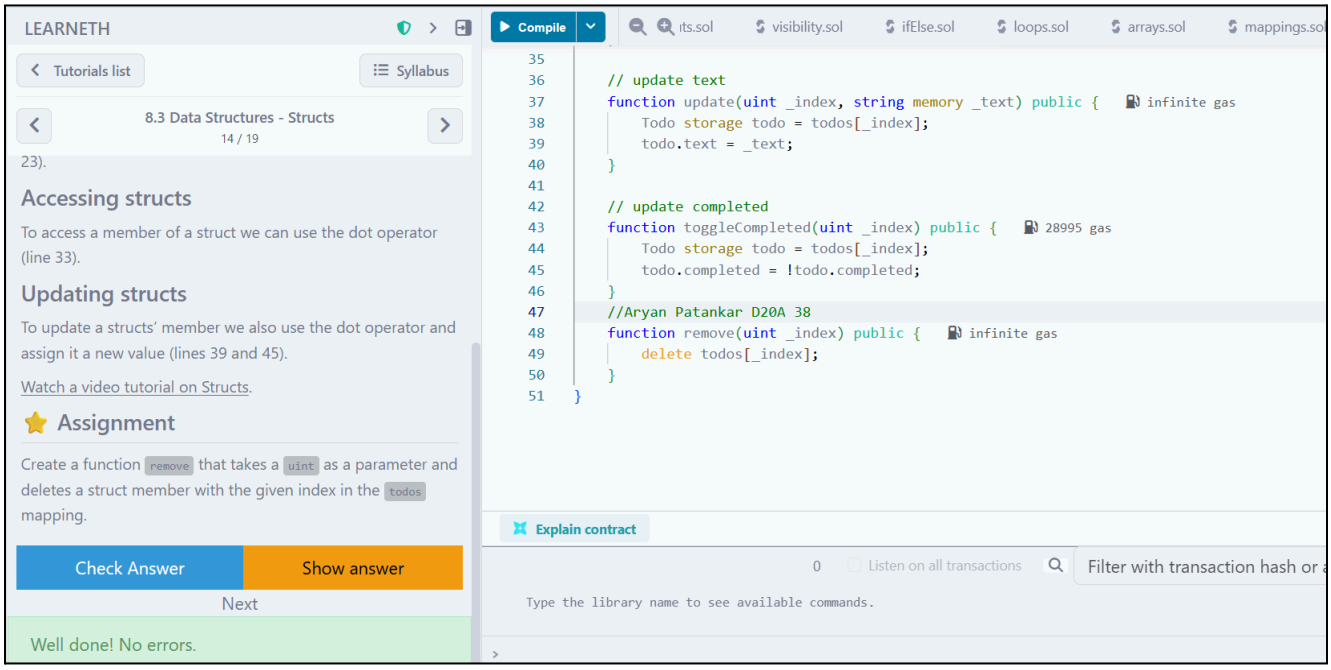
Tutorial no.12



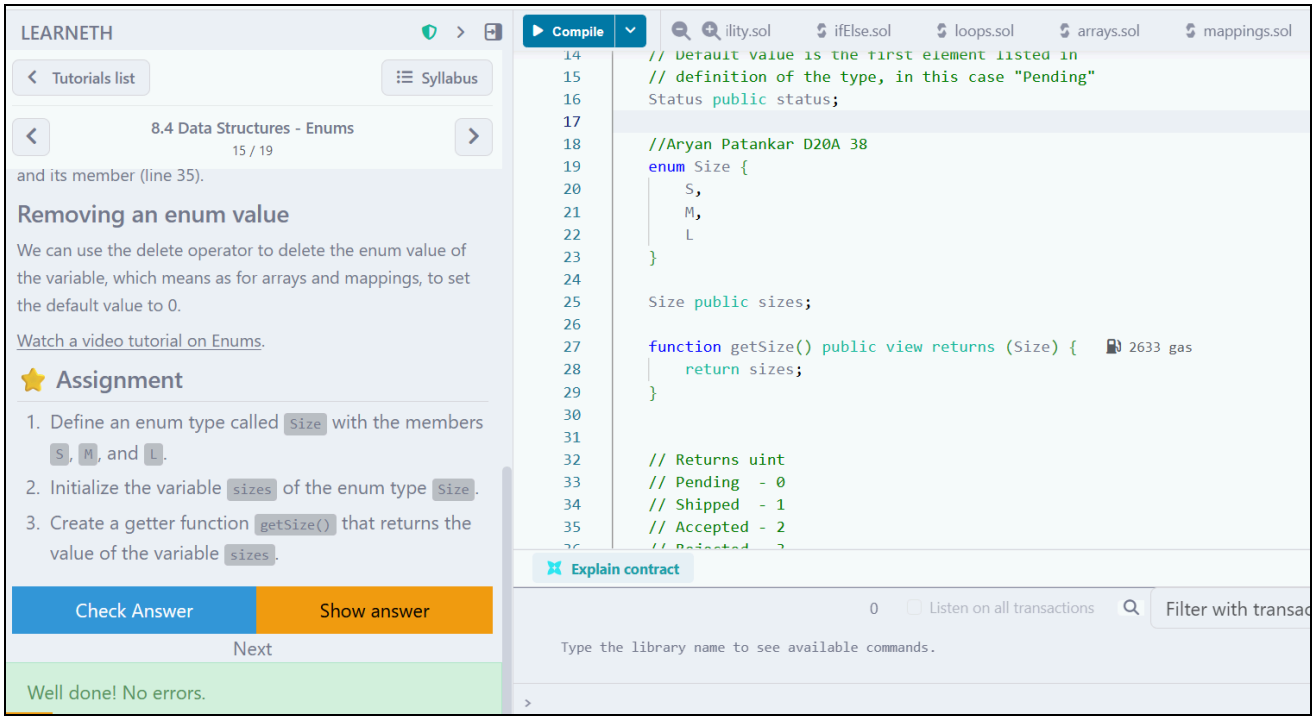
Tutorial no.13



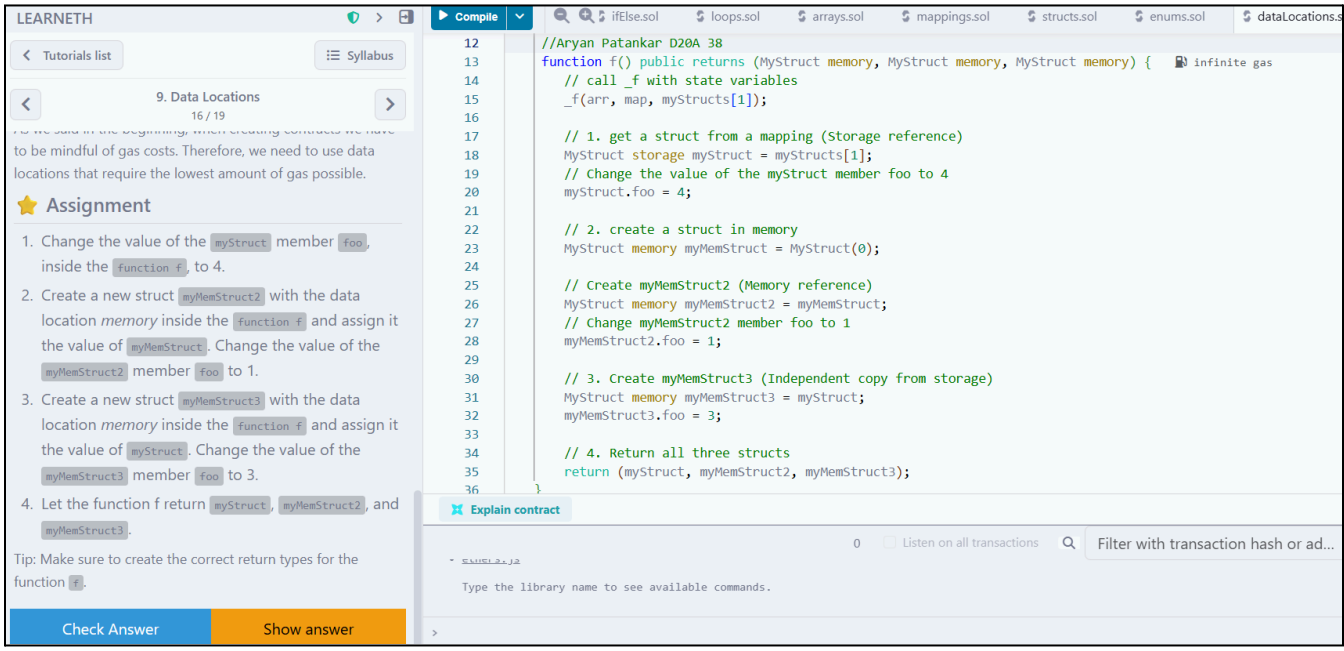
Tutorial no.14



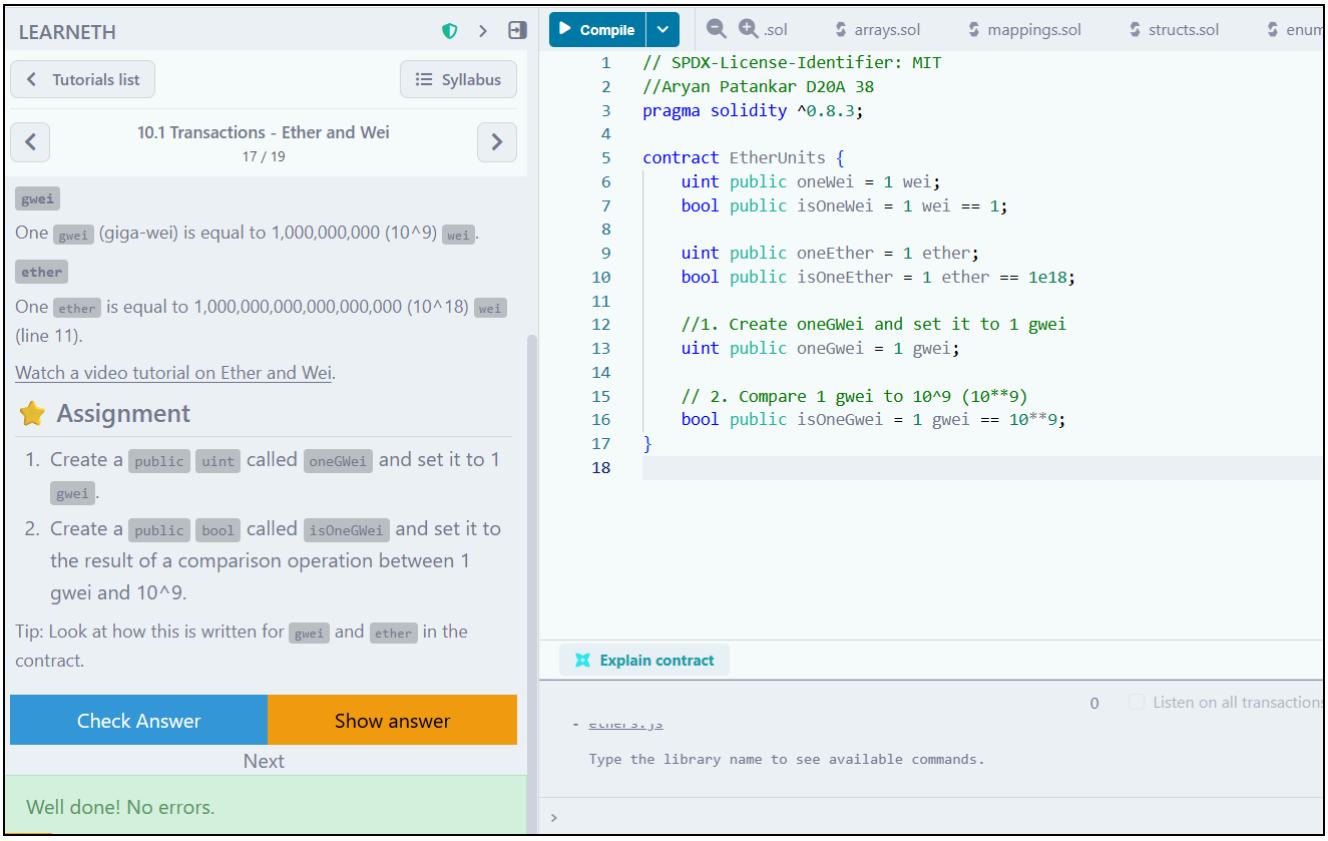
Tutorial no.15



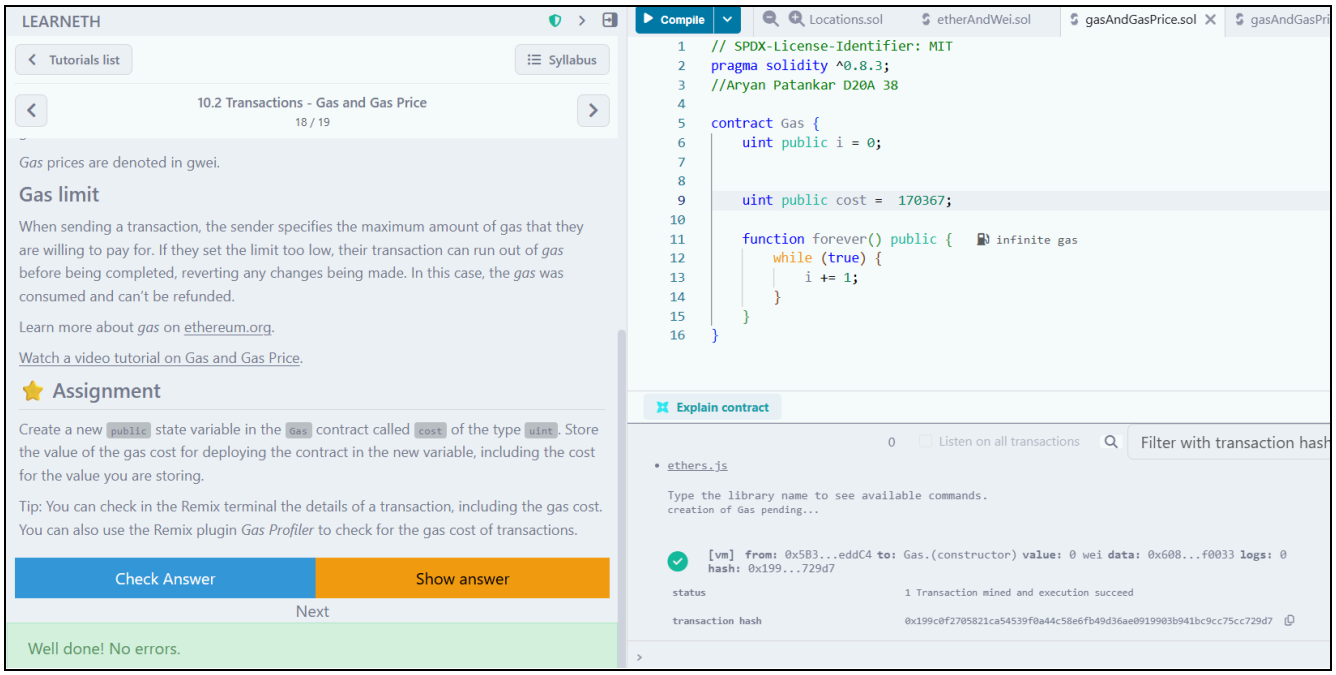
Tutorial no.16



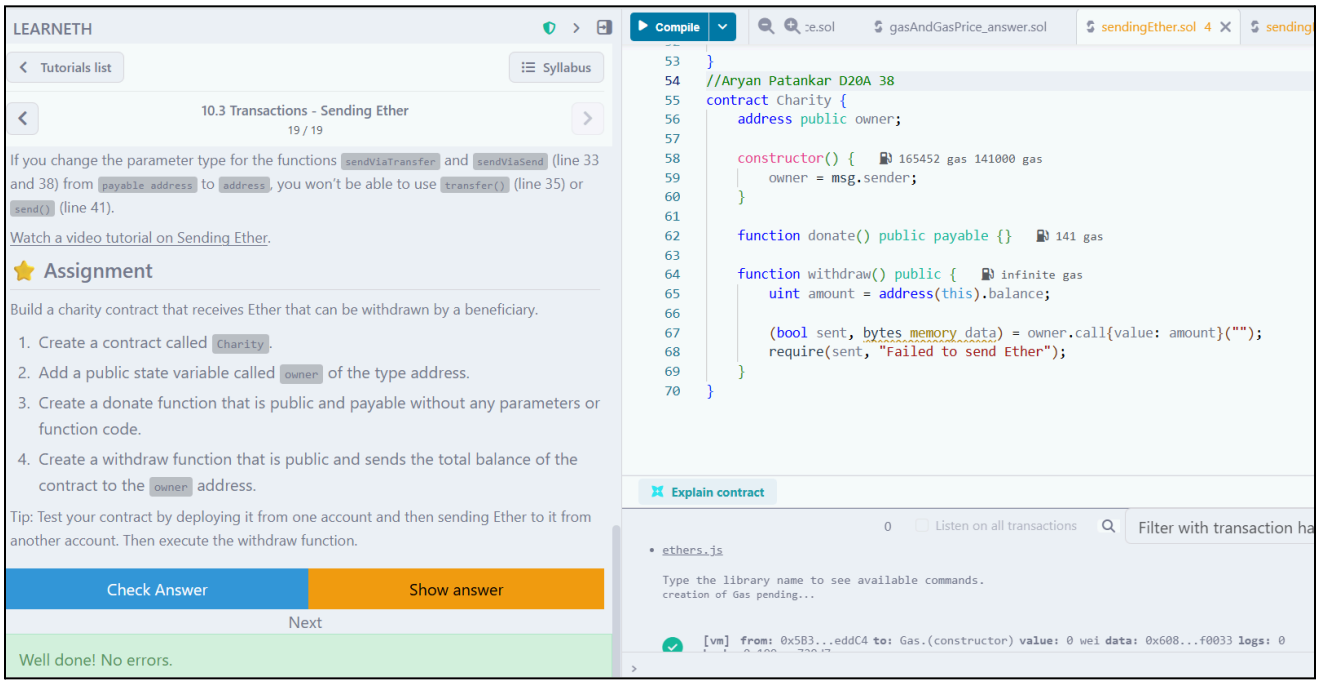
Tutorial no.17



Tutorial no.18



Tutorial no.19



Conclusion:-

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT NO.5

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim:Deploying a Voting/Ballot Smart Contract

Theory:-

- **Relevance of require Statements in Solidity Programs**

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

- **Keywords: mapping, storage, and memory**
  - **mapping**:
    A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage**:

  In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory**:

  In contrast, memory is **temporary storage**, used only for the lifetime of a function

  call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

- **Why bytes32 Instead of string?**

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.

- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type Converter** help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

- **Code**:

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    // This declares a new complex type which will
    // be used for variables later.
    // It will represent a single voter.
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted;  // if true, that person already voted
        address delegate; // person delegated to
        uint vote;   // index of the voted proposal
    }

    // This is a type for a single proposal.
     struct Proposal
    {
    string name;   // Changed from bytes32 to string
    uint voteCount;
     }
    address public chairperson;

    // This declares a state variable that
    // stores a 'Voter' struct for each possible address.
    mapping(address => Voter) public voters;

    // A dynamically-sized array of 'Proposal' structs.
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames) { // Changed bytes32[] to string[]
    chairperson = msg.sender;
    voters[chairperson].weight = 1;
    for (uint i = 0; i < proposalNames.length; i++) {
```

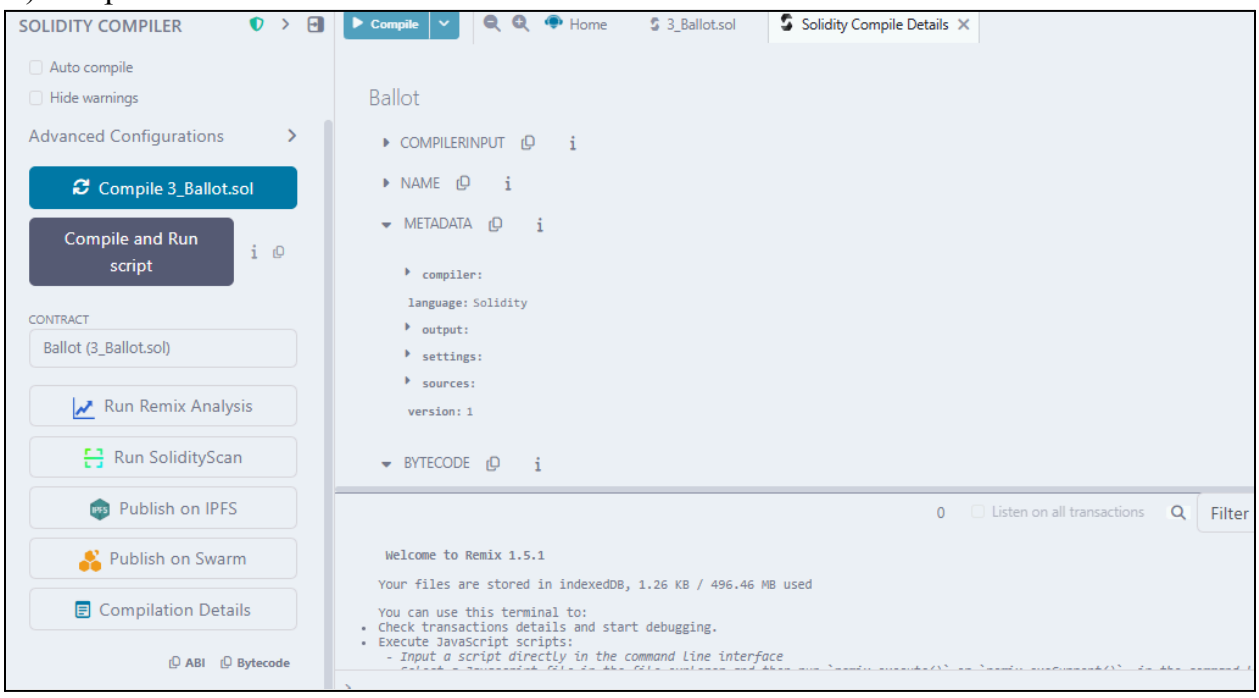
```
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}

     /**
     * @dev Give 'voter' the right to vote on this ballot. May only be called by
'chairperson'.
     * @param voter address of voter
     */
    function giveRightToVote(address voter) external {
        // If the first argument of `require` evaluates
        // to 'false', execution terminates and all
        // changes to the state and to Ether balances
        // are reverted.
        // This used to consume all gas in old EVM versions, but
        // not anymore.
        // It is often a good idea to use 'require' to check if
        // functions are called correctly.
        // As a second argument, you can also provide an
        // explanation about what went wrong.
        require(
            msg.sender == chairperson,
            "Only chairperson can give right to vote."
        );
        require(
            !voters[voter].voted,
            "The voter already voted."
        );
        require(voters[voter].weight == 0, "Voter already has the right to vote.");
        voters[voter].weight = 1;
    }

    /**
     * @dev Delegate your vote to the voter 'to'.
     * @param to address to which vote is delegated
     */
    function delegate(address to) external {
        // assigns reference
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "You have no right to vote");
        require(!sender.voted, "You already voted.");

        require(to != msg.sender, "Self-delegation is disallowed.");

        // Forward the delegation as long as
        // 'to' also delegated.
        // In general, such loops are very dangerous,
        // because if they run too long, they might
        // need more gas than is available in a block.
        // In this case, the delegation will not be executed,
        // but in other situations, such loops might
```

```
        // cause a contract to get "stuck" completely.
        while (voters[to].delegate != address(0)) {
            to = voters[to].delegate;

            // We found a loop in the delegation, not allowed.
            require(to != msg.sender, "Found loop in delegation.");
        }

        Voter storage delegate_ = voters[to];

        // Voters cannot delegate to accounts that cannot vote.
        require(delegate_.weight >= 1);

        // Since 'sender' is a reference, this
        // modifies 'voters[msg.sender]'.
        sender.voted = true;
        sender.delegate = to;

        if (delegate_.voted) {
            // If the delegate already voted,
            // directly add to the number of votes
            proposals[delegate_.vote].voteCount += sender.weight;
        } else {
            // If the delegate did not vote yet,
            // add to her weight.
            delegate_.weight += sender.weight;
        }
    }

    /**
     * @dev Give your vote (including votes delegated to you) to proposal
'proposals[proposal].name'.
     * @param proposal index of proposal in the proposals array
     */
    function vote(uint proposal) external {
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "Has no right to vote");
        require(!sender.voted, "Already voted.");
        sender.voted = true;
        sender.vote = proposal;

        // If 'proposal' is out of the range of the array,
        // this will throw automatically and revert all
        // changes.
        proposals[proposal].voteCount += sender.weight;
    }

    /**
     * @dev Computes the winning proposal taking all previous votes into account.
     * @return winningProposal_ index of winning proposal in the proposals array
     */
    function winningProposal() public view
            returns (uint winningProposal_)
    {
```

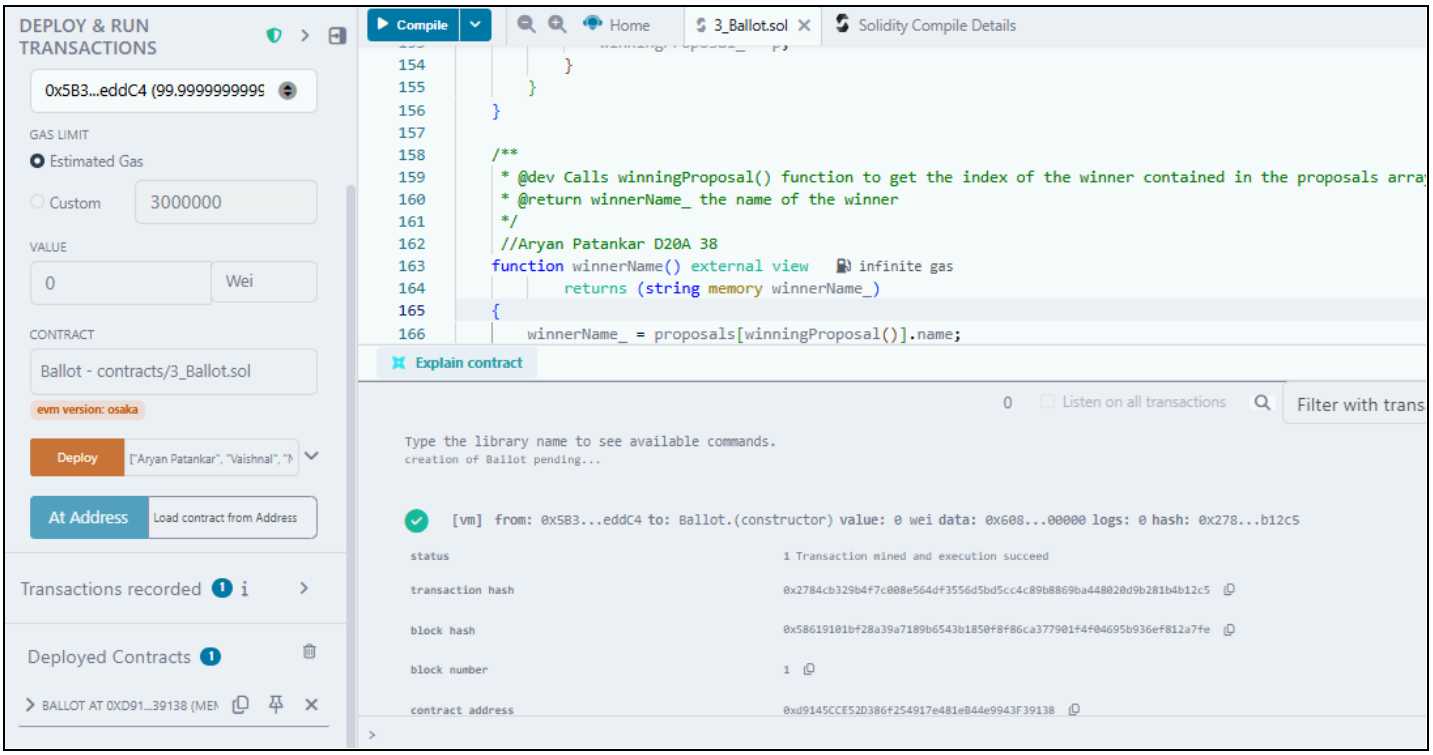
```
        uint winningVoteCount = 0;
        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

    /**
     * @dev Calls winningProposal() function to get the index of the winner
contained in the proposals array and then
     * @return winnerName_ the name of the winner
     */
    function winnerName() external view
            returns (string memory winnerName_)
    {
        winnerName_ = proposals[winningProposal()].name;
    }

}
```

Output:-

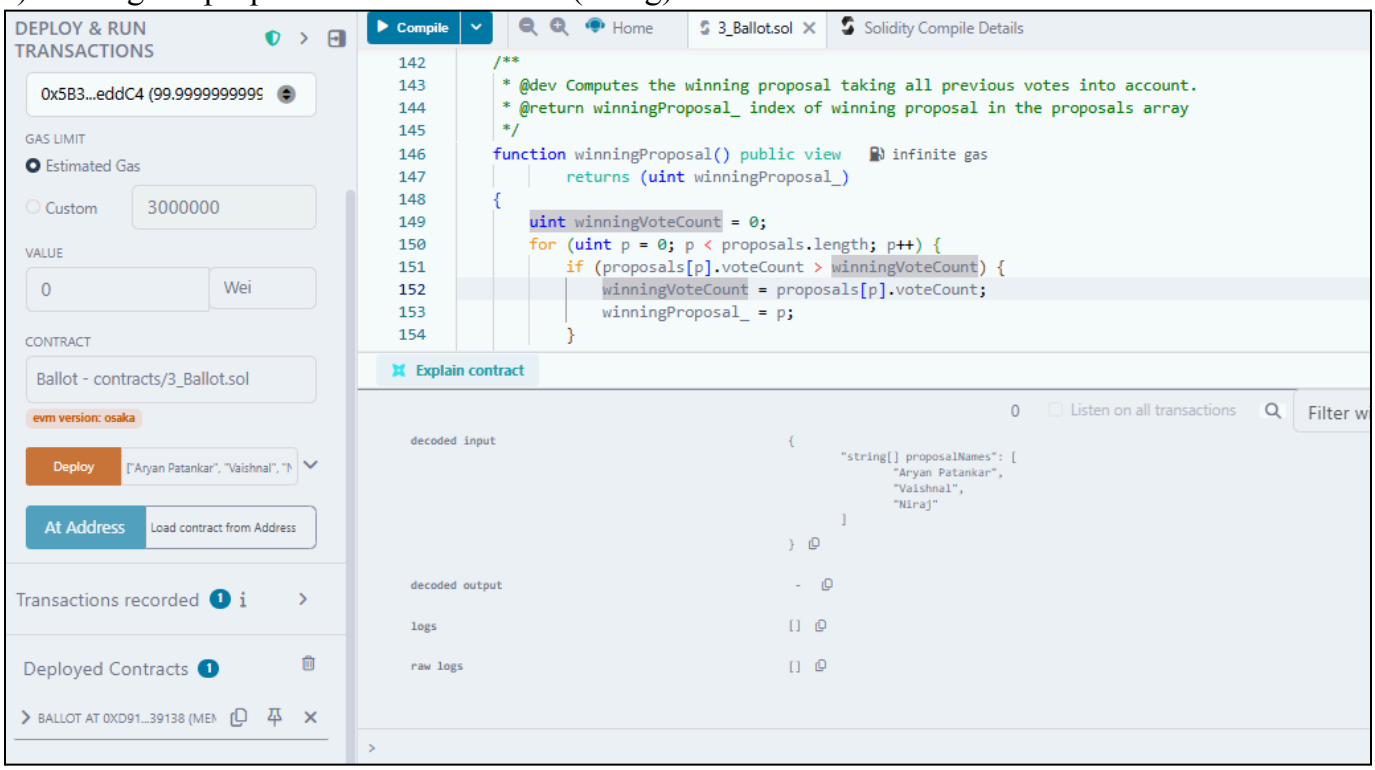
1)Compile the Ballot.sol Contract



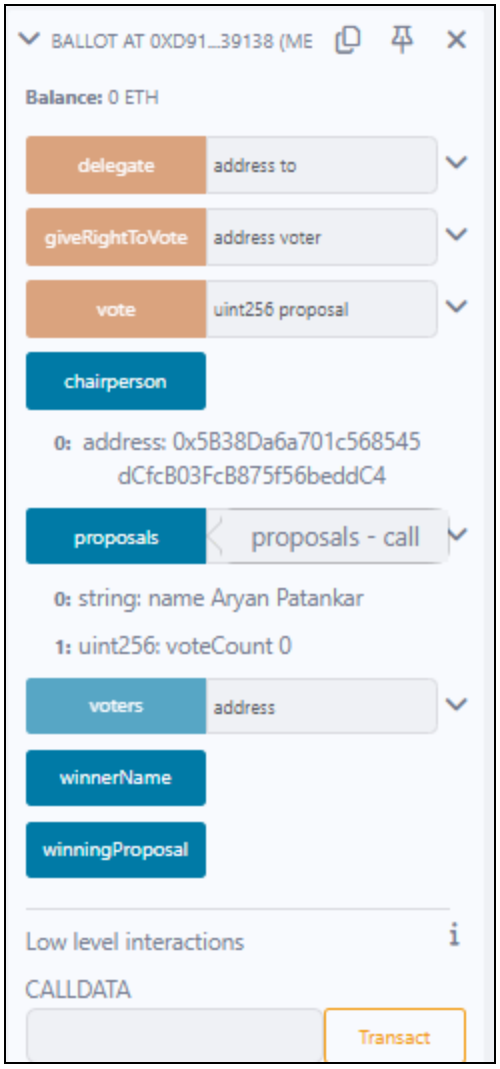
2)Deploying and running of the contract



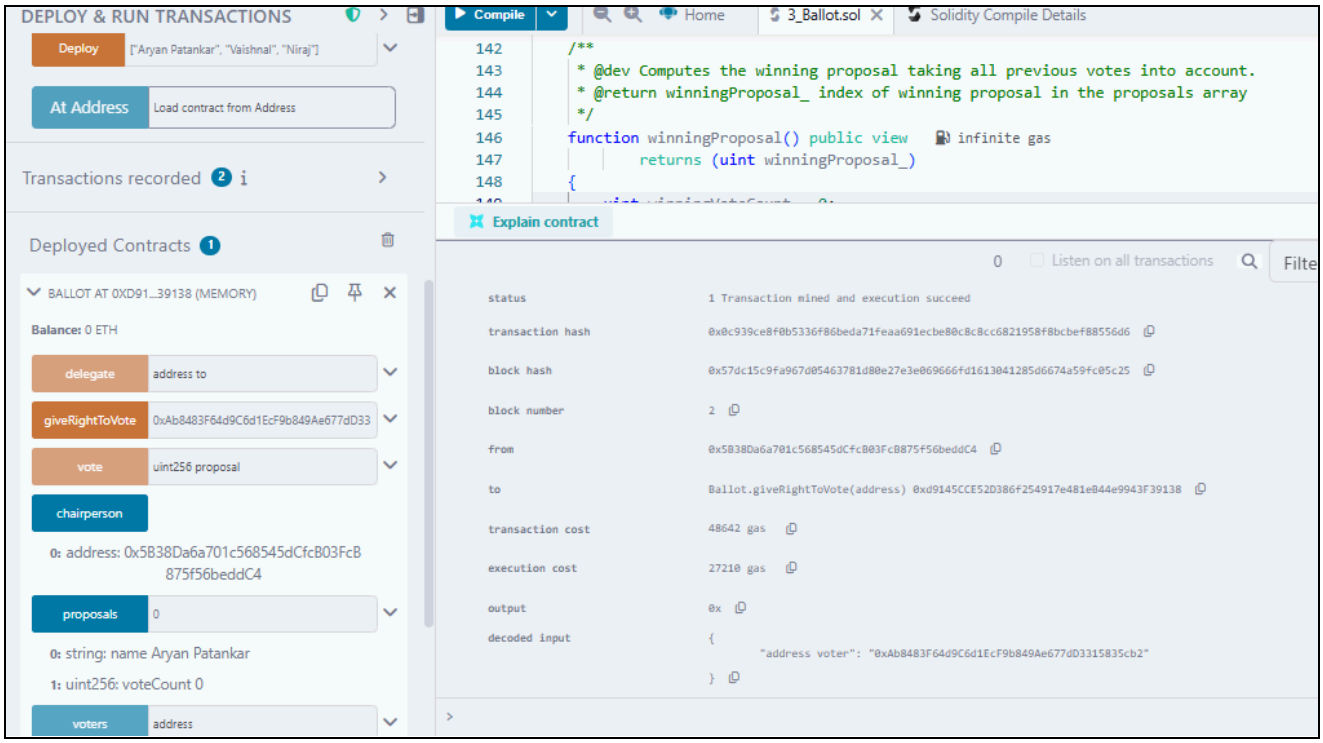
3)Loading the proposal candidate's name(string)



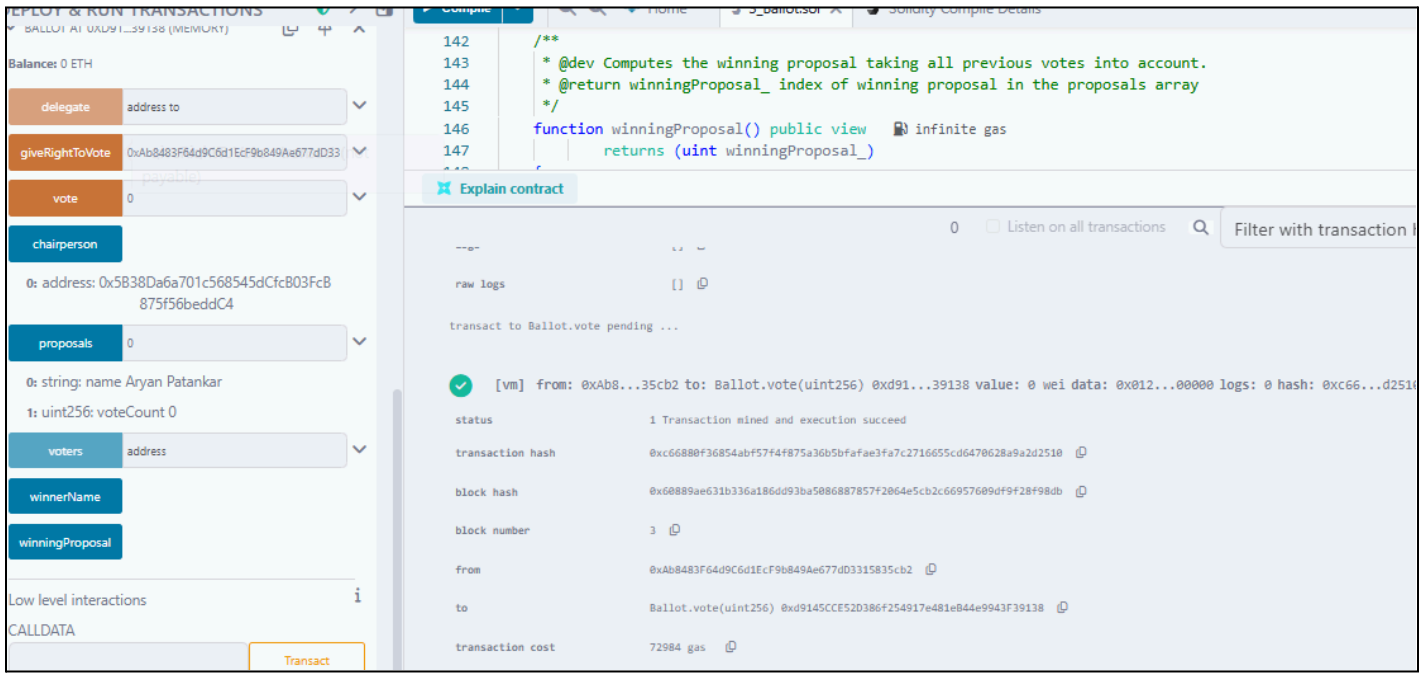
4)Viewing the details of the Proposal Candidate



5)Giving the right to an account other than and by the chairman

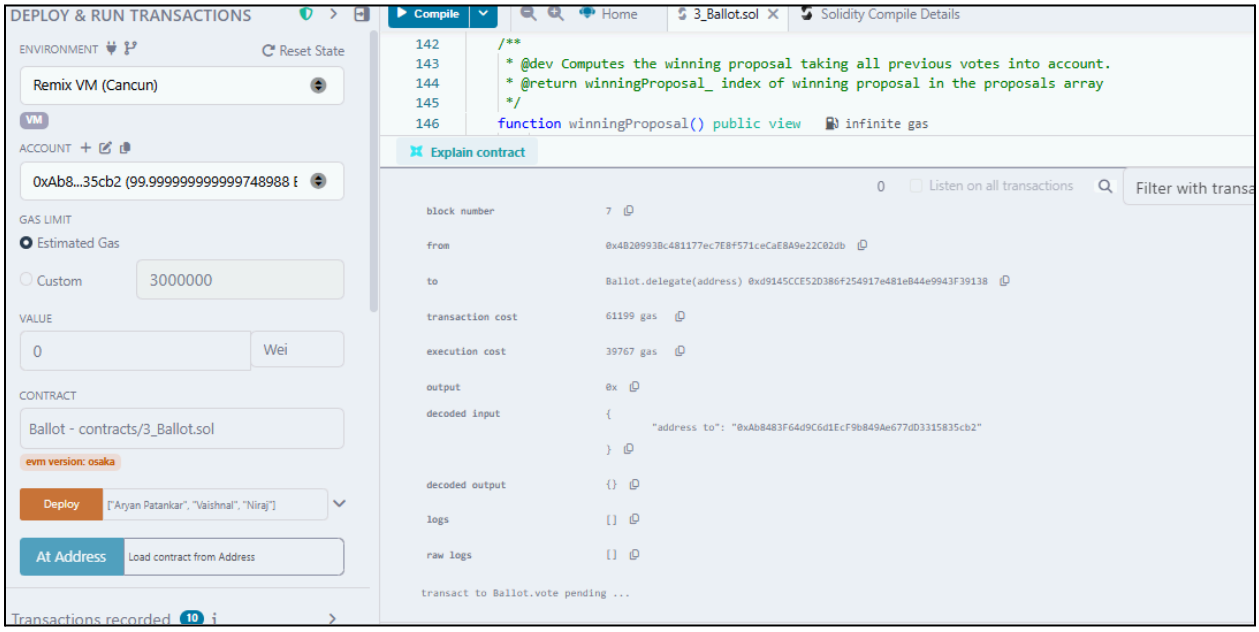


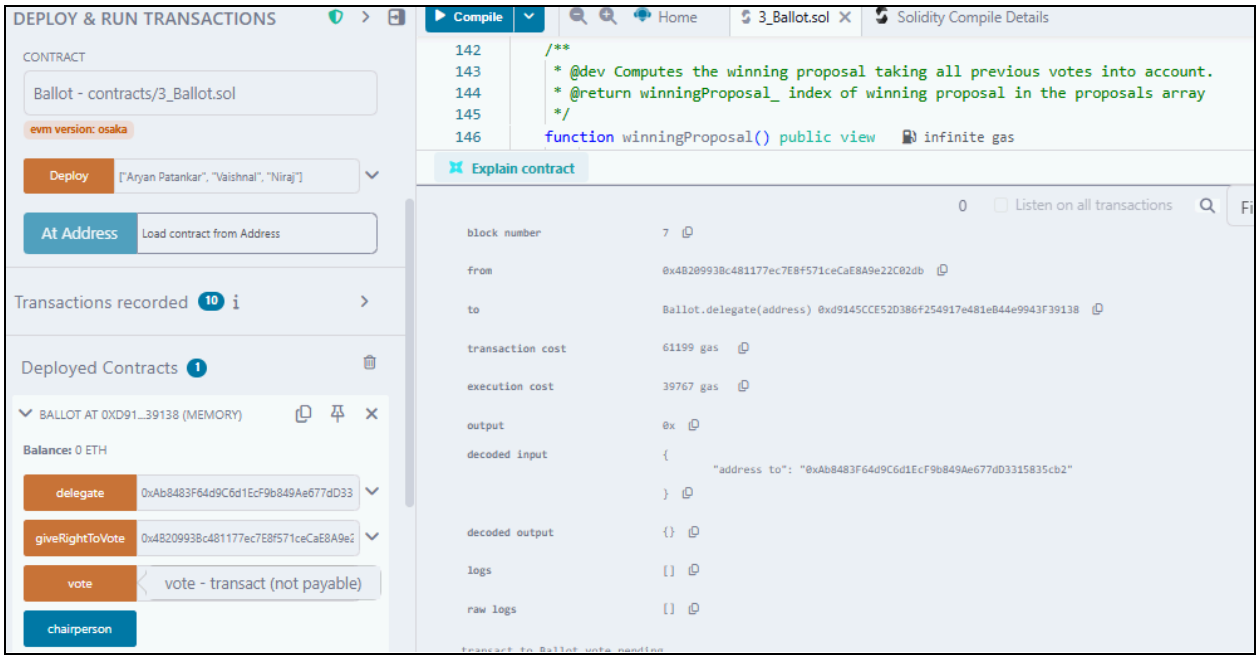
6)Selecting the account which was given the right to vote and then writing the proposal candidate's index to vote.



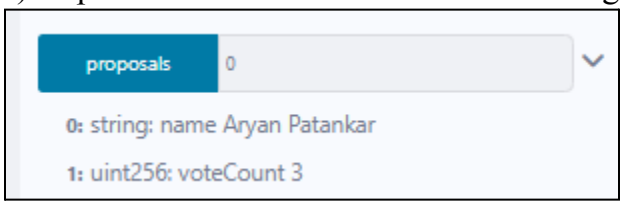
7)We can view the Voter's Information

8)Selecting the account of the delegator and then writing the address of the voter to be delegated to.

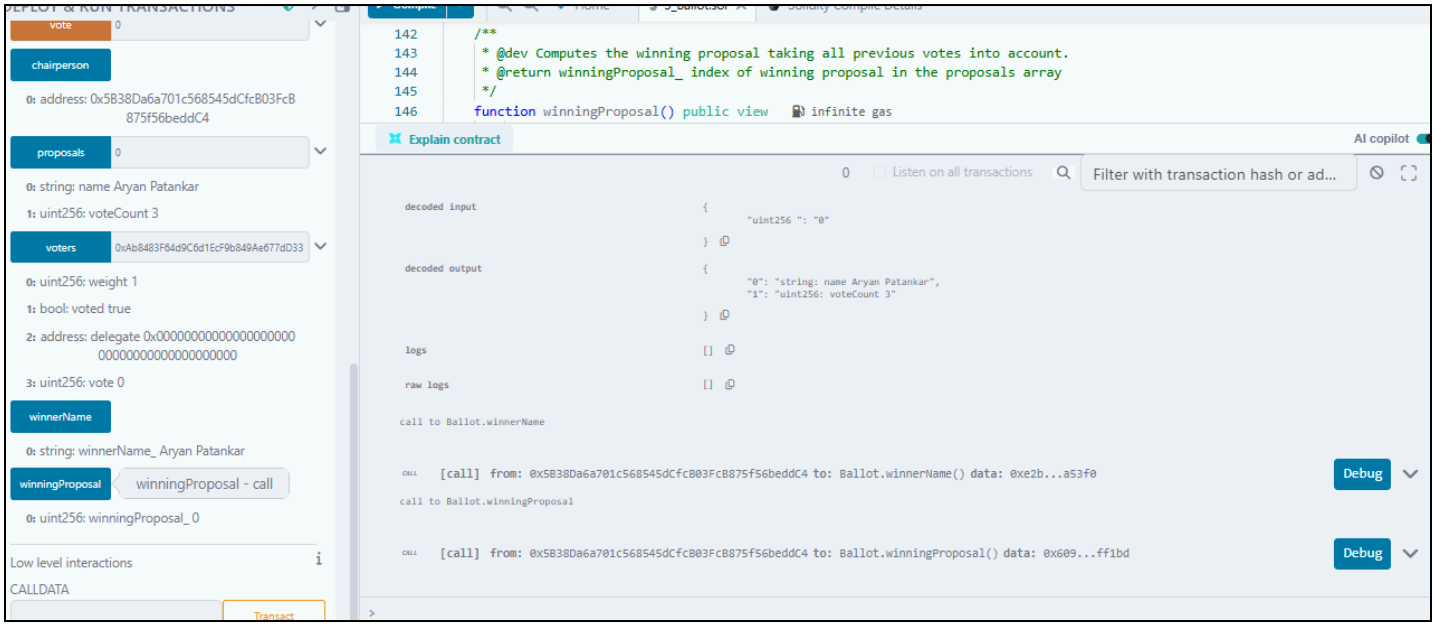




9)Proposal Candidate's Information before giving the Delegate's vote



10)In this final screenshot we can see that after the delegation's vote the weight of one vote of the one voter who was delegated is the number of people who delegated the voter as their voter (Default weight of any voter is 1)



Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.6

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim:Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation**: Executes actions automatically when conditions are satisfied.
- **Transparency**: The contract code and transactions are visible on the blockchain.
- **Security**: Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective**: Removes intermediaries and reduces transaction costs.
- **Trustless System**: Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Implementation:

The project "**Land Ownership Registry System"** focuses on automating land records and ownership transfers using Ethereum-based Smart Contracts.

To ensure the system works, we will implement two primary contracts:

12. **LandRegistry.sol**: Manages the database of lands and their owners.

13. **LandTransfer.sol**: Handles the logic of transferring ownership from a Seller to a Buyer.

Code:-

1. **LandRegistry.sol**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract LandRegistry {
    struct Land {
        uint id;
        string location;
        uint area;
        address owner;
        bool isRegistered;
    }

    mapping(uint => Land) public lands;
    address public admin;

    constructor() {
        admin = msg.sender;
    }

    function registerLand(uint _id, string memory _location, uint _area, address _owner)
public {
        require(msg.sender == admin, "Only admin can register land");
        require(!lands[_id].isRegistered, "Land already registered");

        lands[_id] = Land(_id, _location, _area, _owner, true);
    }

    function updateOwner(uint _id, address _newOwner) public {
        // This function will be called by the Transfer contract
        lands[_id].owner = _newOwner;
```

```
    }

    function getLand(uint _id) public view returns (uint, string memory, uint, address) {
        Land memory l = lands[_id];
        return (l.id, l.location, l.area, l.owner);
    }
}
```

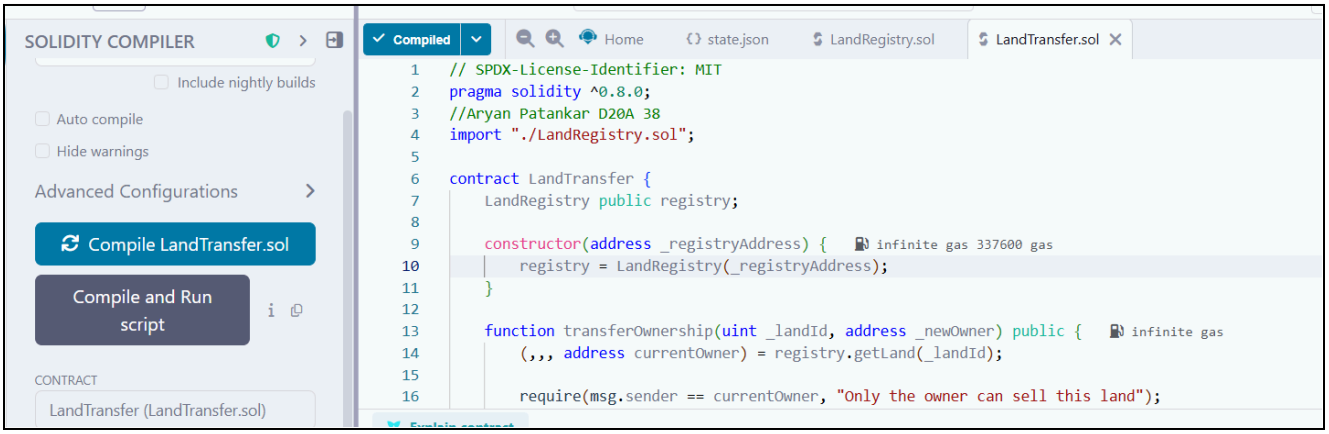
2.**LandTransfer.sol**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./LandRegistry.sol";

contract LandTransfer {
    LandRegistry public registry;

    constructor(address _registryAddress) {
        registry = LandRegistry(_registryAddress);
    }

    function transferOwnership(uint _landId, address _newOwner) public {
        (,,, address currentOwner) = registry.getLand(_landId);

        require(msg.sender == currentOwner, "Only the owner can sell this land");

        registry.updateOwner(_landId, _newOwner);
    }
}
```

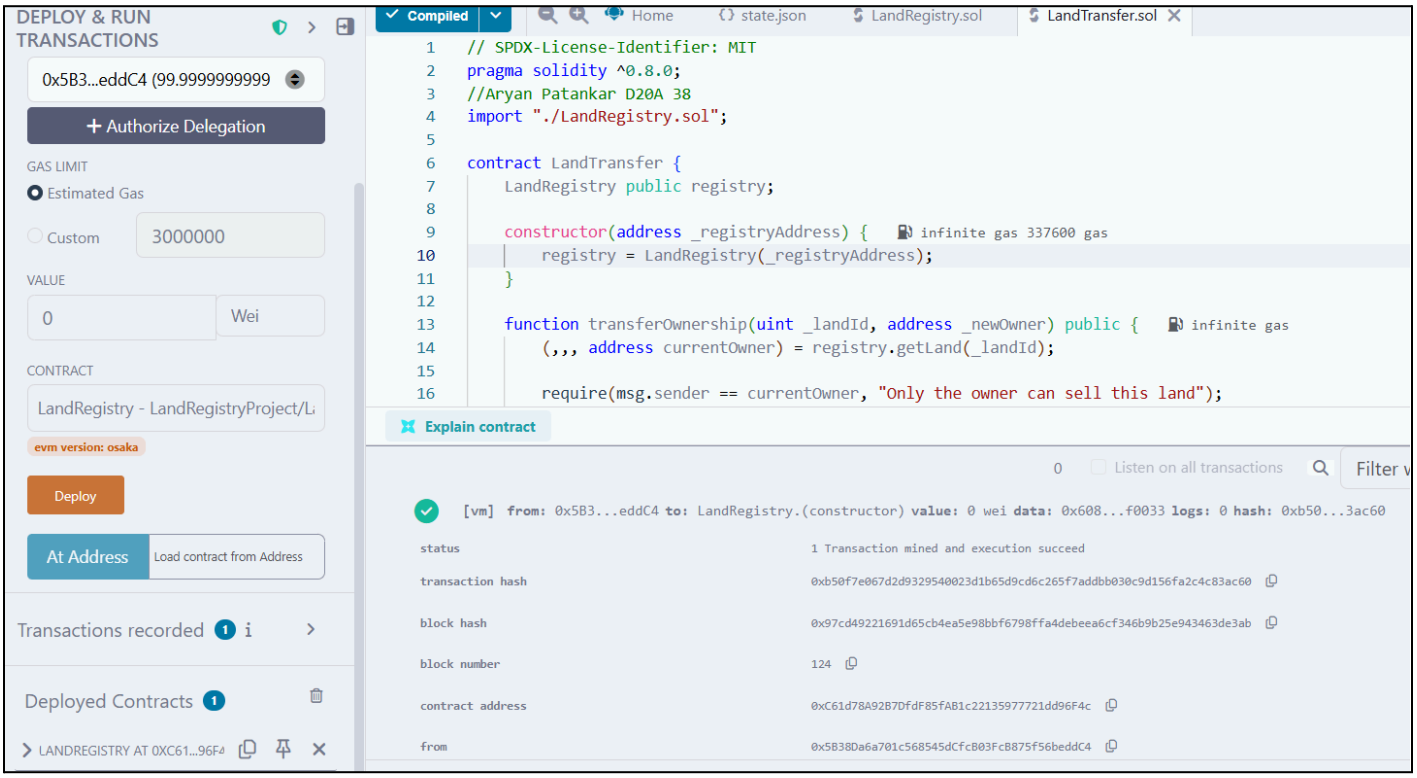
Step 1: Set up the Files

- Go to Remix IDE.
- In the File Explorer, create a new folder named LandRegistryProject.
- Create two files: LandRegistry.sol and LandTransfer.sol.

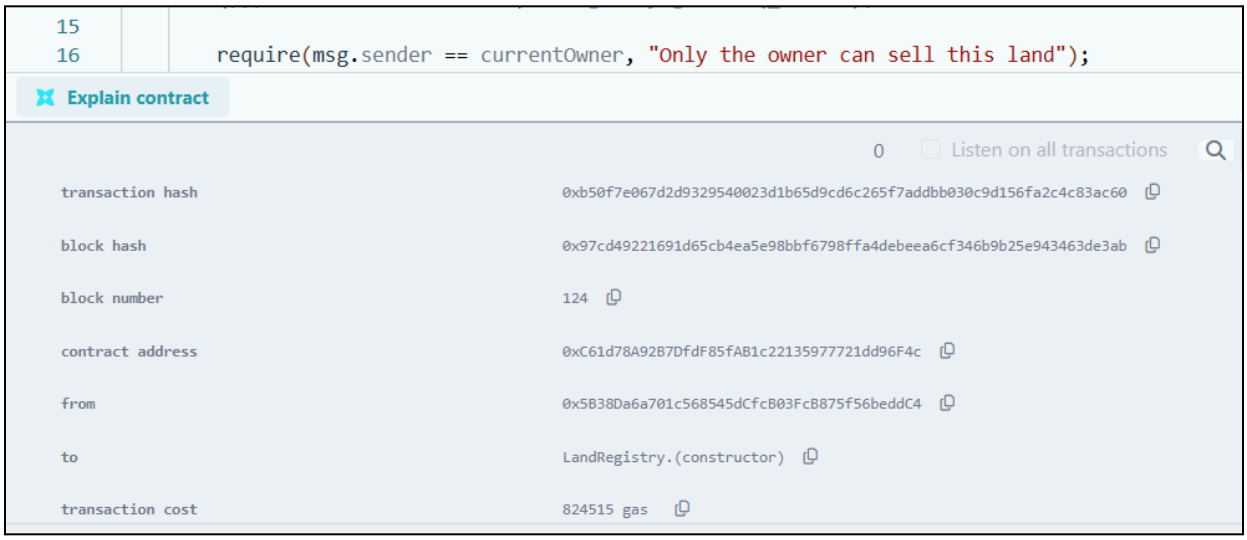
Step 2: Compile both the contracts



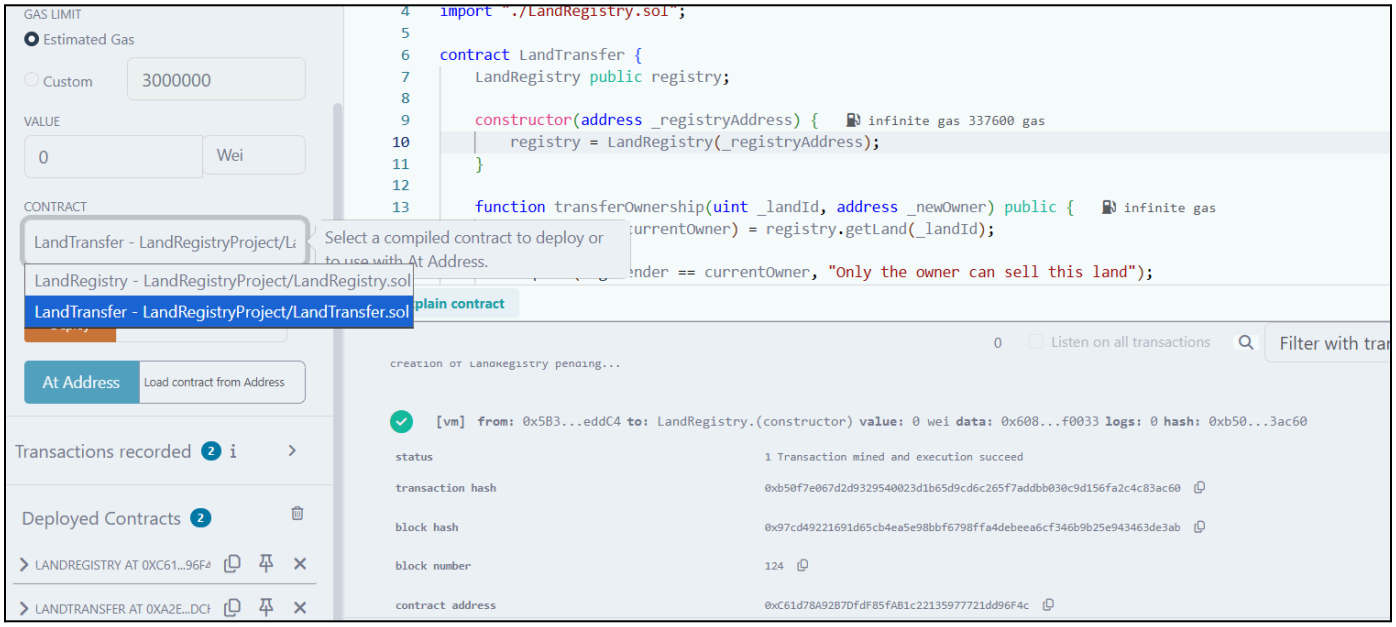
Step 3:Deploy the LandRegistry contract



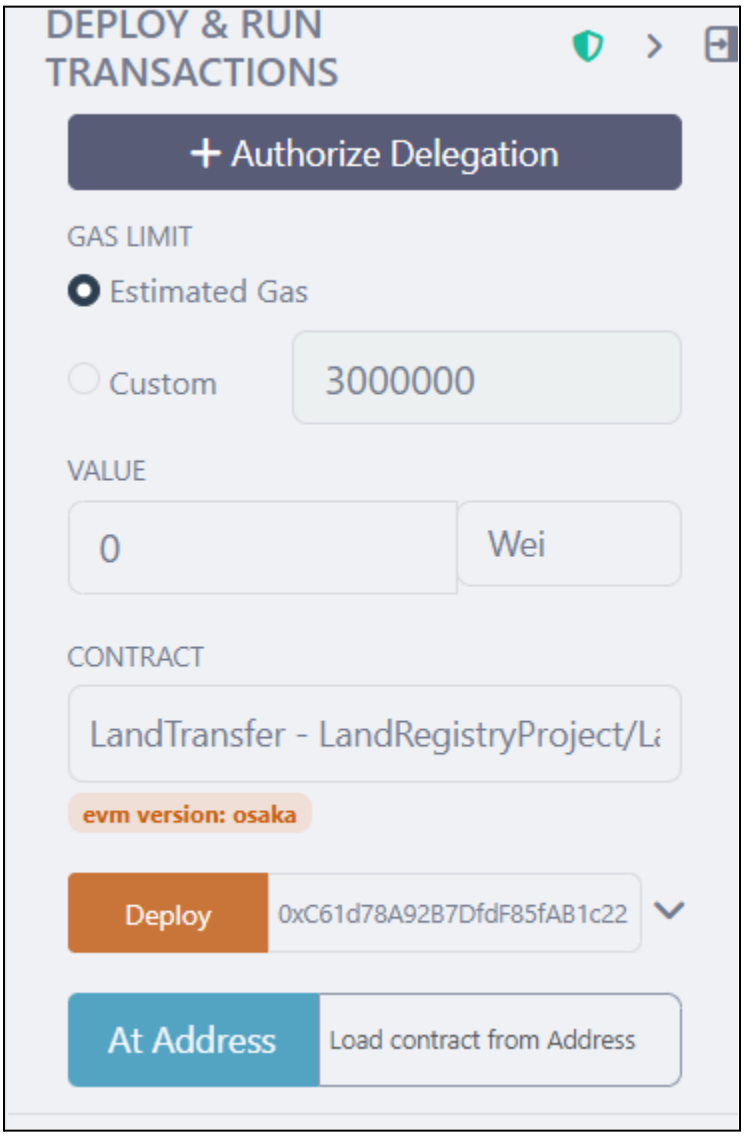
Step 4:Copy the contract address of the deployed LandRegistry contract



Step 5:Select LandTransfer here in the dropdown



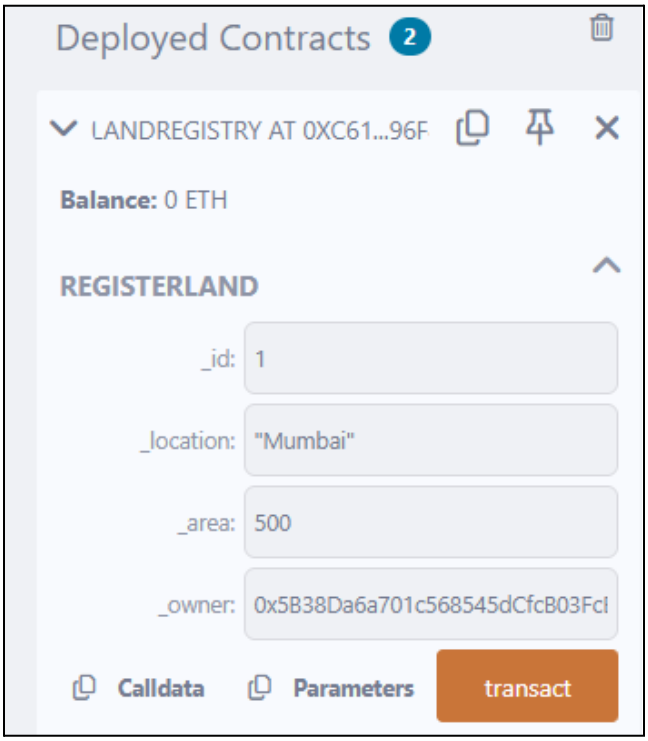
Step 6: **Add the contract address in place of _registryAddress field next to the deploy button,and click on deploy so as to deploy the LandTransfer contract**

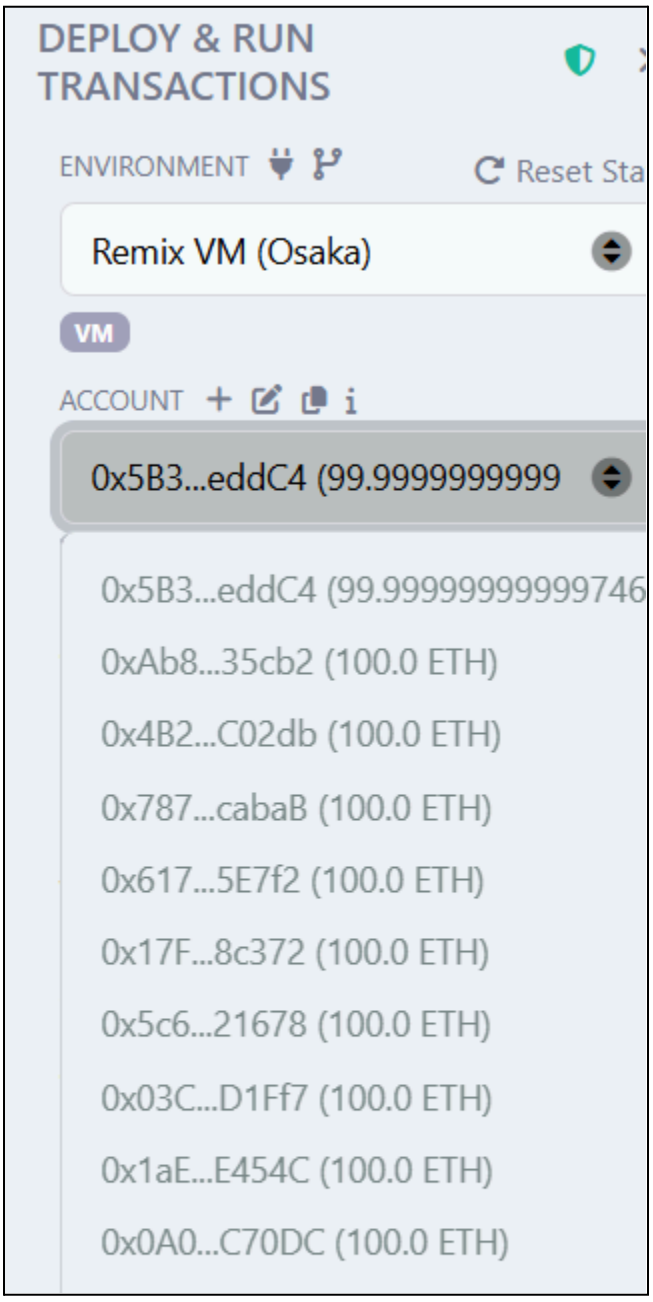


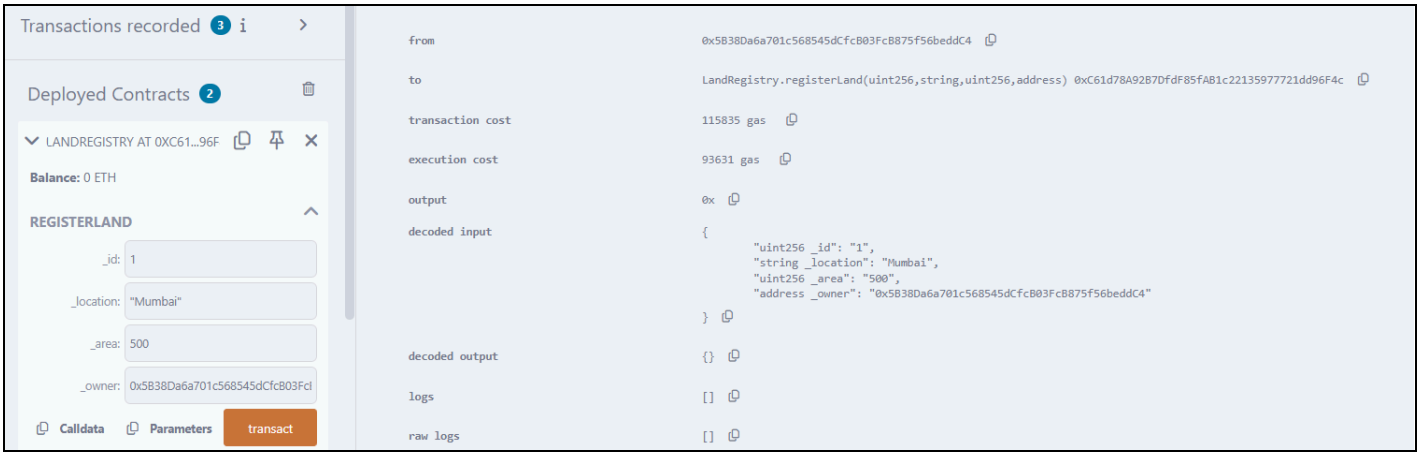
Step 7: Execute the transactions

1.Land Registration

In the deployed contracts section expand the LandRegistry contract, and add details as mentioned. Also ensure that the wallet address is added in the owner field. Once all this is done, click on transact to complete the transaction.



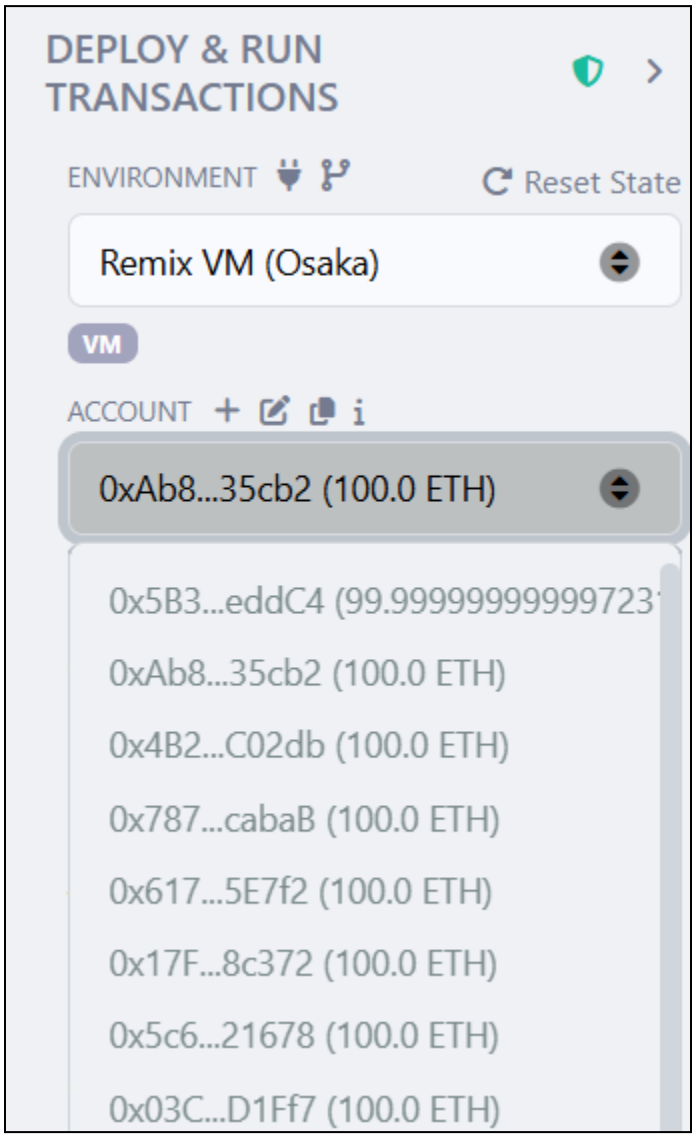

DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Reset Sta
Remix VM (Osaka)
VM
ACCOUNT
0x5B3...eddC4 (99.9999999999
0x5B3...eddC4 (99.99999999999746
0xAb8...35cb2 (100.0 ETH)
0x4B2...C02db (100.0 ETH)
0x787...cabaB (100.0 ETH)
0x617...5E7f2 (100.0 ETH)
0x17F...8c372 (100.0 ETH)
0x5c6...21678 (100.0 ETH)
0x03C...D1Ff7 (100.0 ETH)
0x1aE...E454C (100.0 ETH)
0x0A0...C70DC (100.0 ETH)



Enter id=1 in the getLand field to see the registered data

2.**Transfer Ownership**

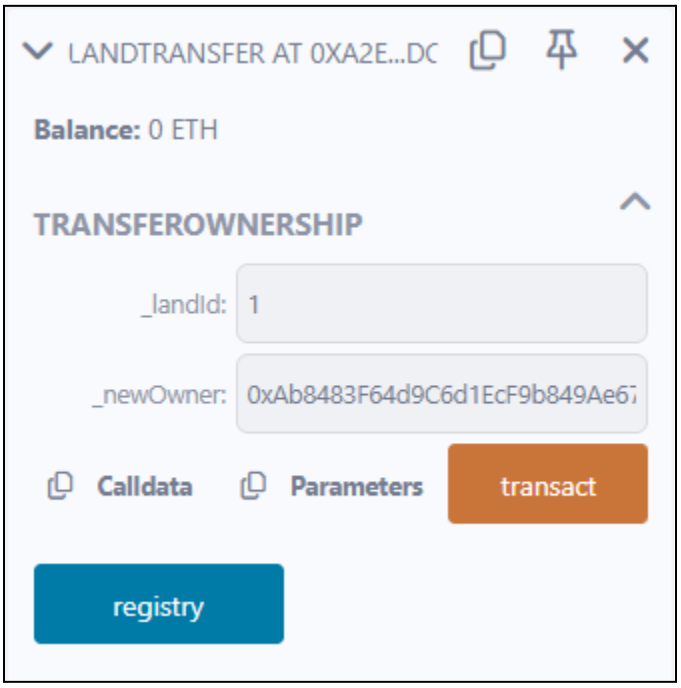
Copy a different wallet address from the account dropdown(this wallet address represents the buyer)



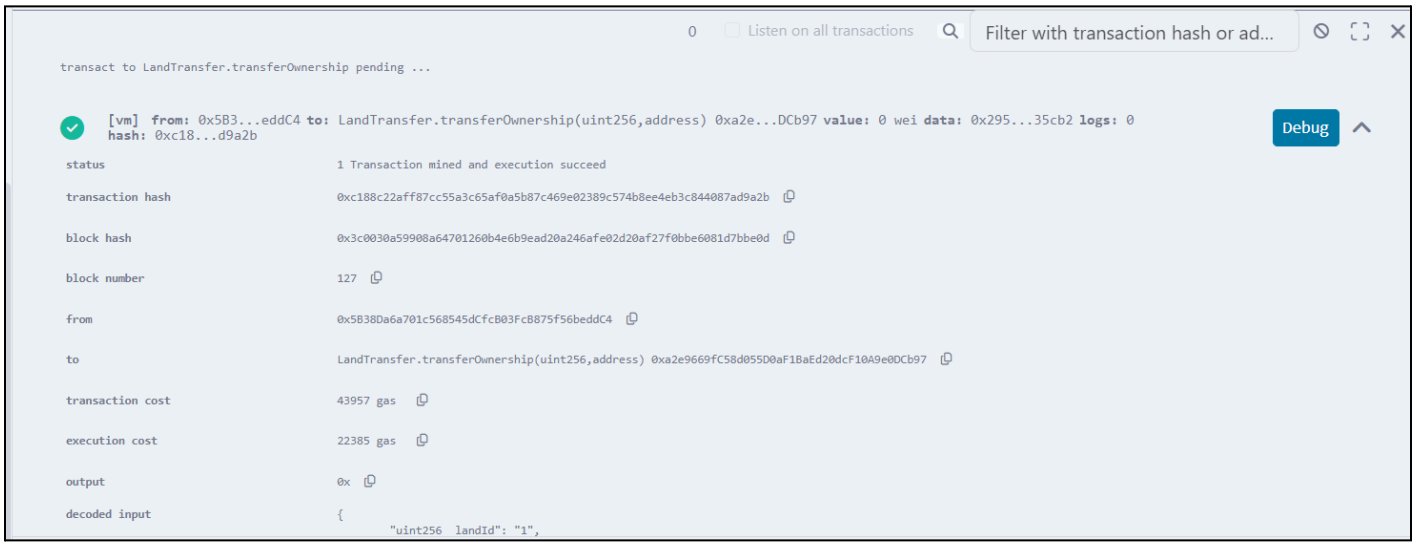
Switch back to the original wallet address(i.e the seller's wallet address) and within the transferOwnership function in the land transfer contract, enter:-

_landId: **1**

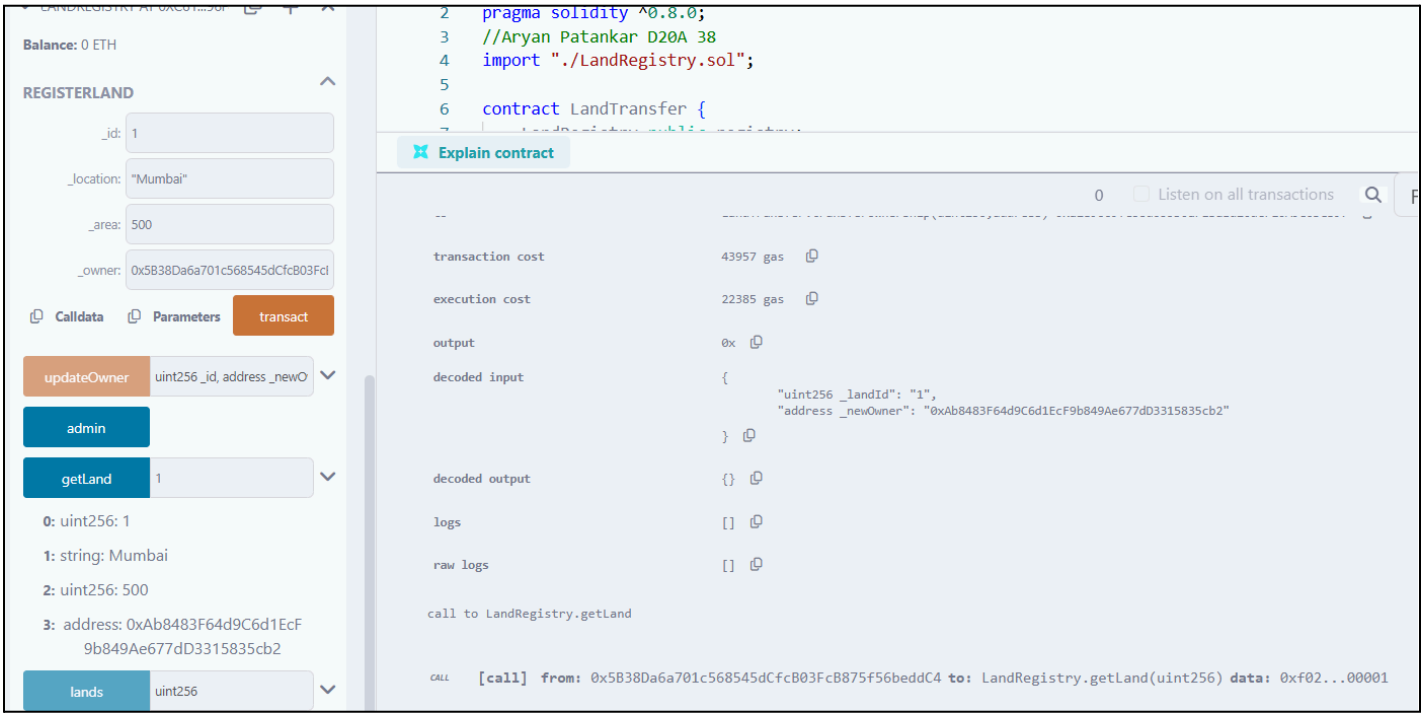
_newOwner: **[The Buyer's Address]**



Click on transact to complete the ownership transfer



Now, verify the ownership transfer by going to the LandRegistry contract. When we click on the getLand button, we see a different wallet address, indicating that the ownership transfer has taken place and is successfully verified.



Conclusion:-

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.7

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim:Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

1. What is MetaMask?

MetaMask is a software cryptocurrency wallet used to interact with the Ethereum blockchain and other EVM-compatible networks (like Polygon, BSC, and Avalanche).

It is primarily known as a **"bridge"** because it allows users to access Decentralized Applications (dApps) directly through a regular web browser (like Chrome or Firefox) without having to run a full Ethereum node.

Core Theoretical Functions:

14. **Identity Management:** It generates and manages your private keys locally on your device (not on a central server), giving you full ownership of your digital identity.
15. **Web3 Injection:** It injects a JavaScript library called **web3.js** or **ethers.js** into the websites you visit. This allows the website to "talk" to the blockchain.
16. **Transaction Signing:** When you want to perform an action on a dApp (like buying an NFT), MetaMask "pops up" to let you review and digitally sign the transaction before it is sent to the network.
17. **Network Switching:** It allows users to switch between the "Mainnet" (real money) and various "Testnets" (fake money).

2. What is a Testnet?

A **Testnet** (Test Network) is an alternative blockchain that acts as an exact replica of the main blockchain (Mainnet) but is used specifically for **experimenting, testing, and debugging.**

In blockchain theory, deploying code directly to the Mainnet is risky and expensive because every transaction costs real money (Gas fees) and bugs can lead to permanent financial loss. Testnets solve this by providing a "sandbox" environment.

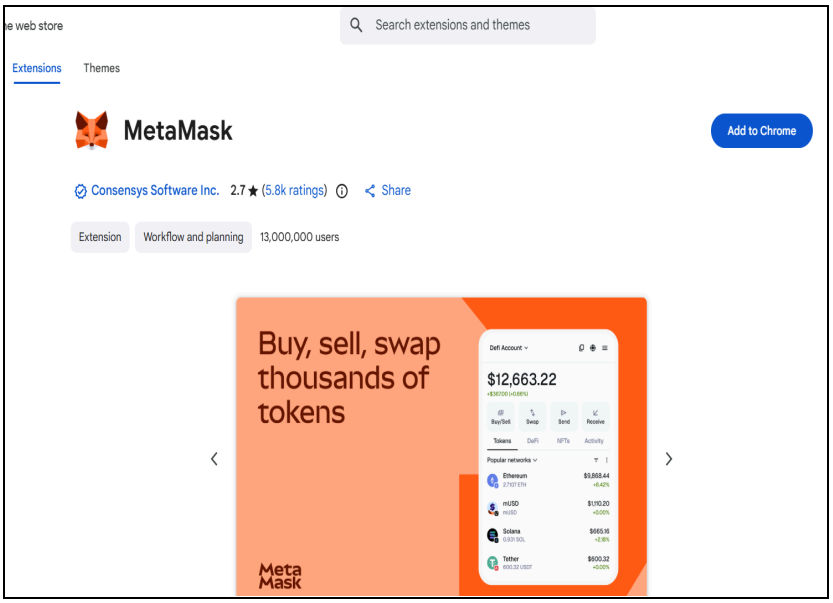
Key Characteristics:

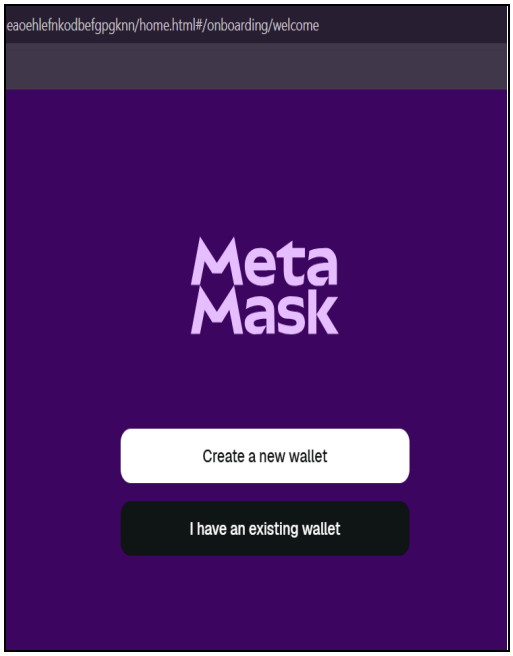
- **No Real Value:** The tokens used on a testnet (e.g., Sepolia ETH) have **zero real-world market value.** You can get them for free from "Faucets."
- **Identical Environment:** It uses the same rules, consensus mechanisms (like Proof of Stake), and smart contract logic as the Mainnet.
- **Safe Playground:** If a developer's smart contract has a bug that crashes the system or "leaks" funds, no real money is lost.
- **Public Access:** Most testnets are public, meaning anyone can join and test their applications before the official launch.

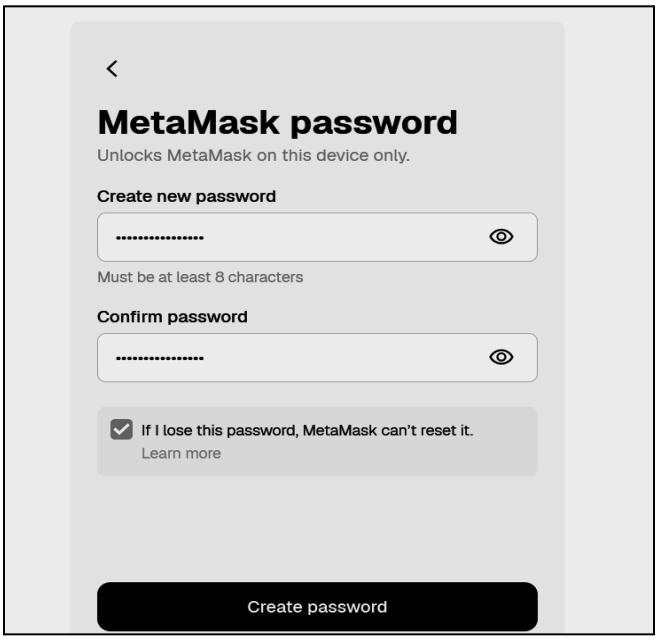
Implementation:

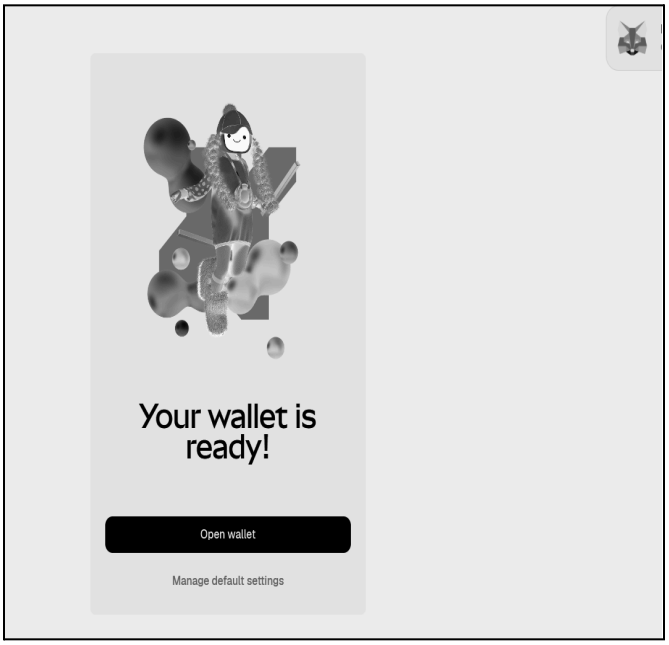
Step 1: Install Metamask

Follow this link to install the Metamask Google Chrome Extension, and also create the wallet







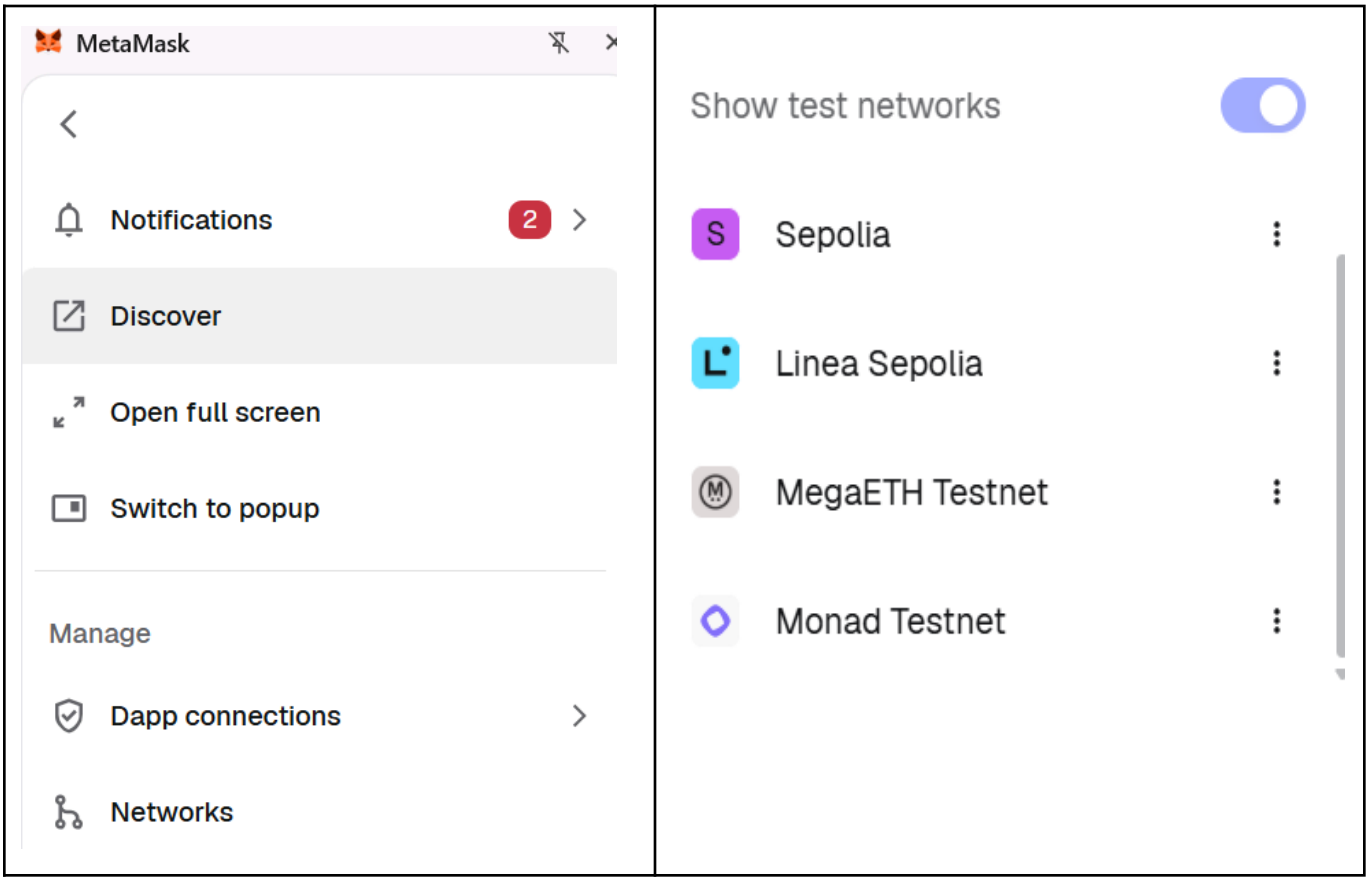


Note :

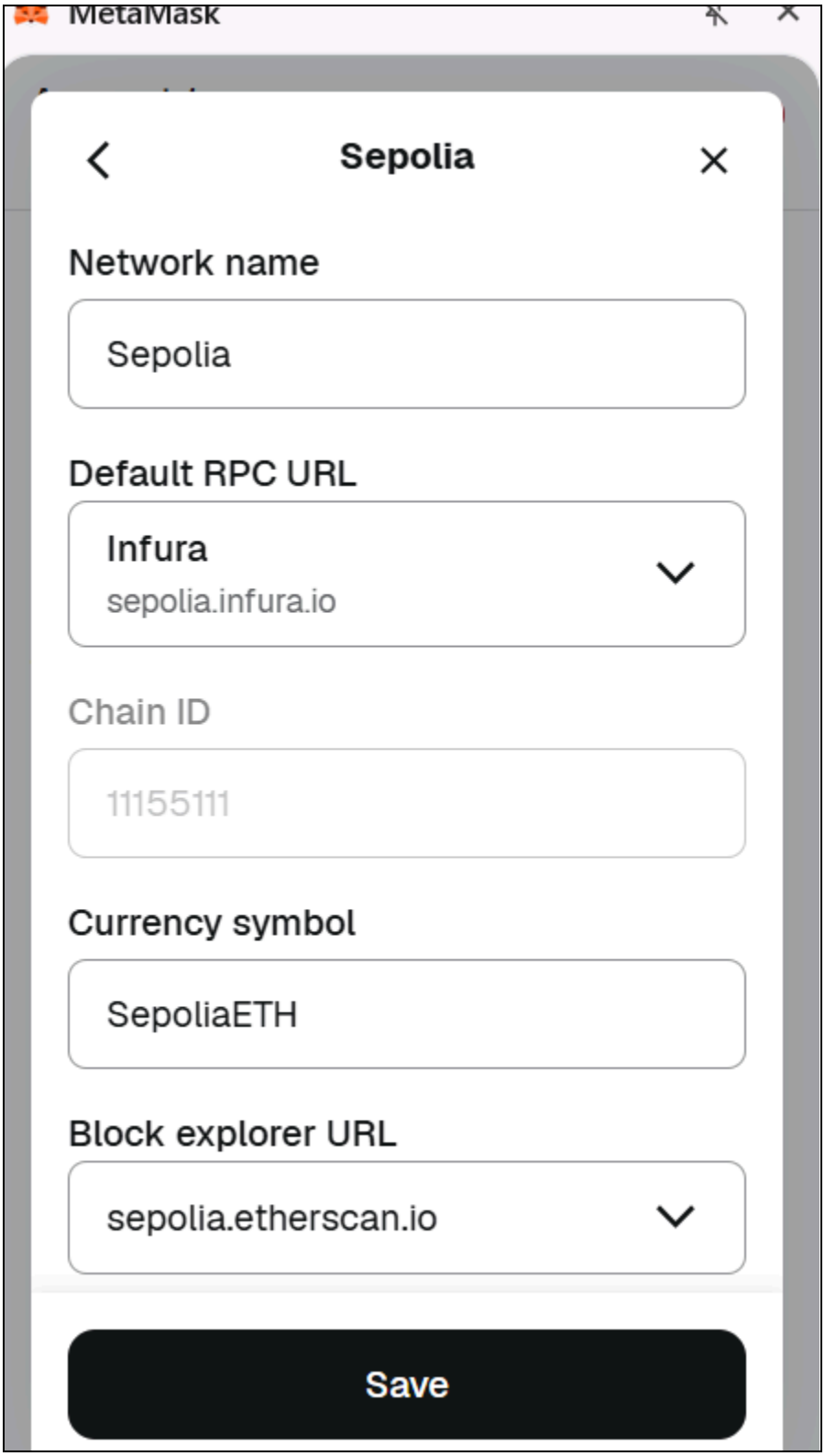
- While creation of the Metamask Wallet, you will be asked to select 12 pass phrase
- Keep the passphrase safley
- It becomes handy to recover your password

Step 2: Enable Test Networks

- By default, MetaMask shows the "Ethereum Mainnet" (real money).
- Click the network dropdown (top left) and toggle **Show test networks**.
- Select **Sepolia**.



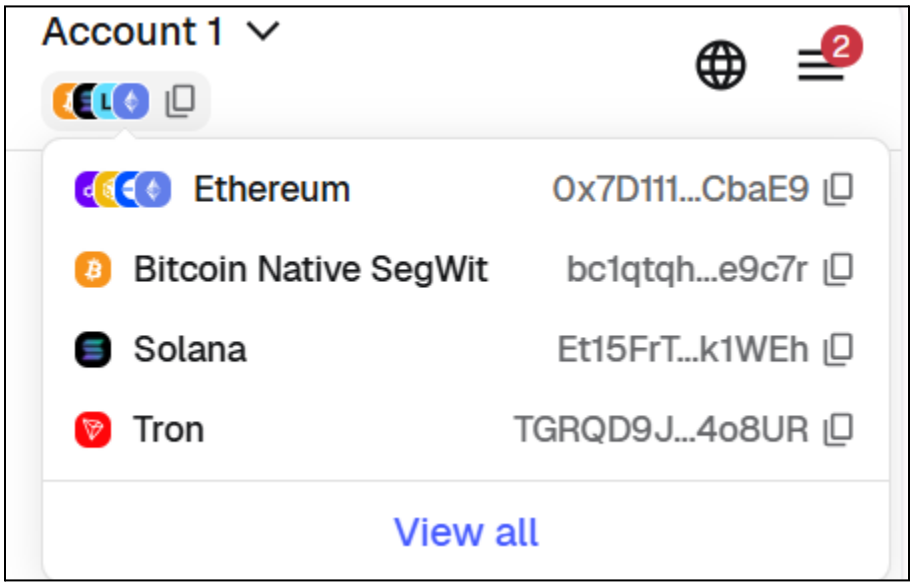
Step 3: Configuring the Sepolia Test network



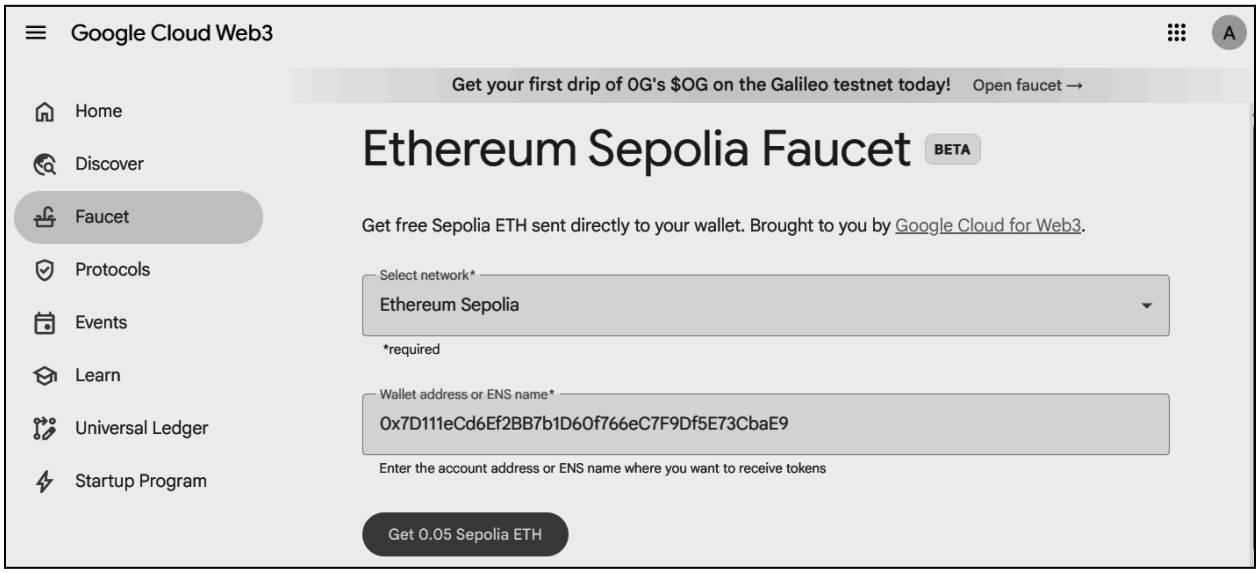
Step 4: Funding the Wallet

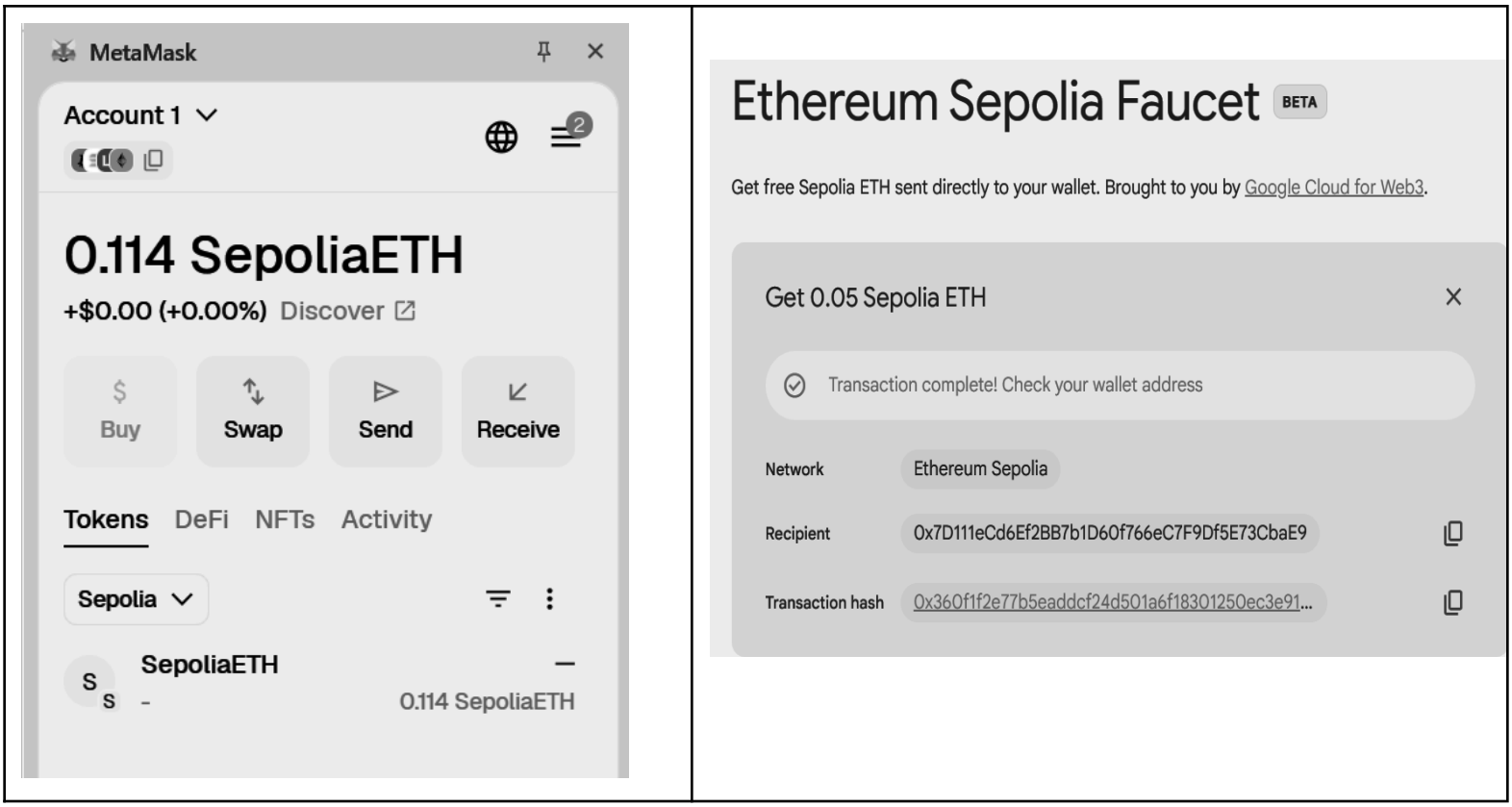
Every action on the blockchain requires a fee called **Gas**. On the Sepolia network, we use "Test ETH" which is free.

8. **Copy Your Address**: Click your account name at the top of MetaMask (it starts with `0x...`) to copy your public address.

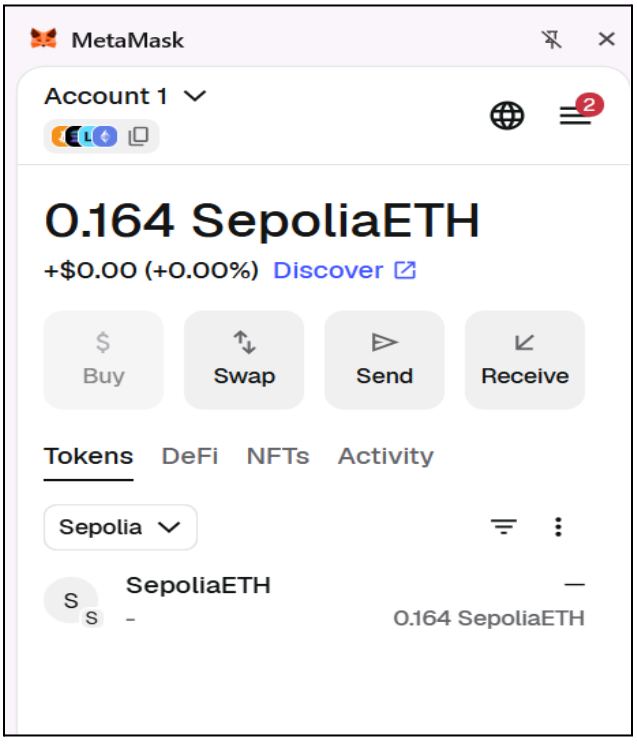


2. **Visit a Faucet**: Use the [Google Cloud Sepolia Faucet] to procure SepoliaETH by pasting the wallet address as shown



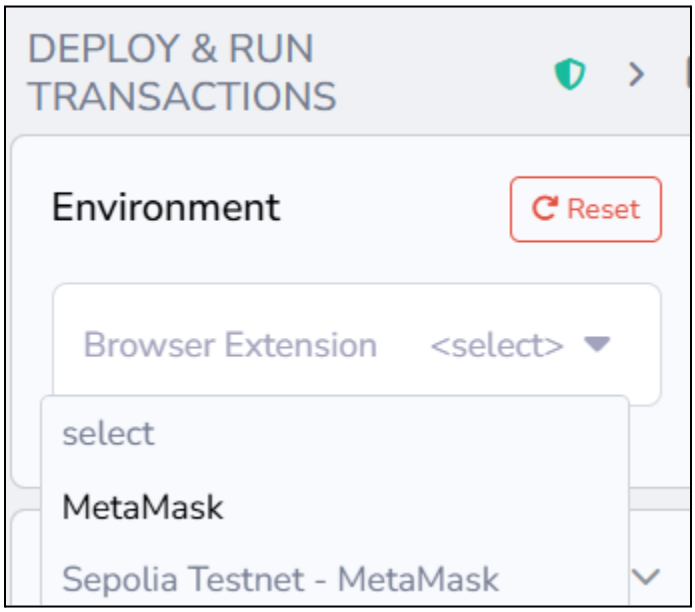


3.**Verify**: Your MetaMask balance should update within 30 seconds.

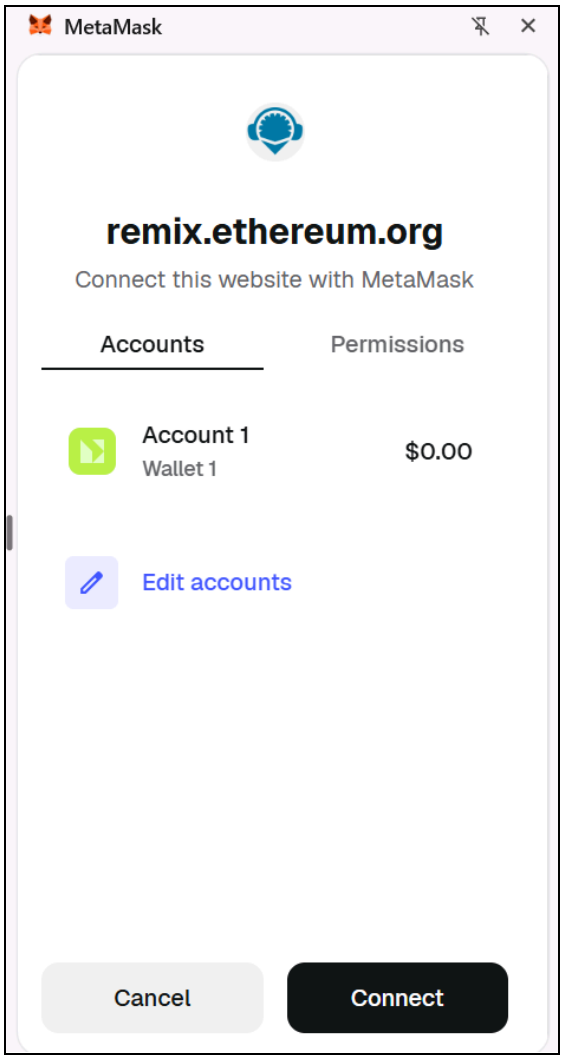


Step 5: Connect MetaMask to Remix IDE

12. Open Remix IDE.
13. Go to the **Deploy & Run Transactions** tab.
14. In the **Environment** dropdown, select **Injected Provider - MetaMask**.

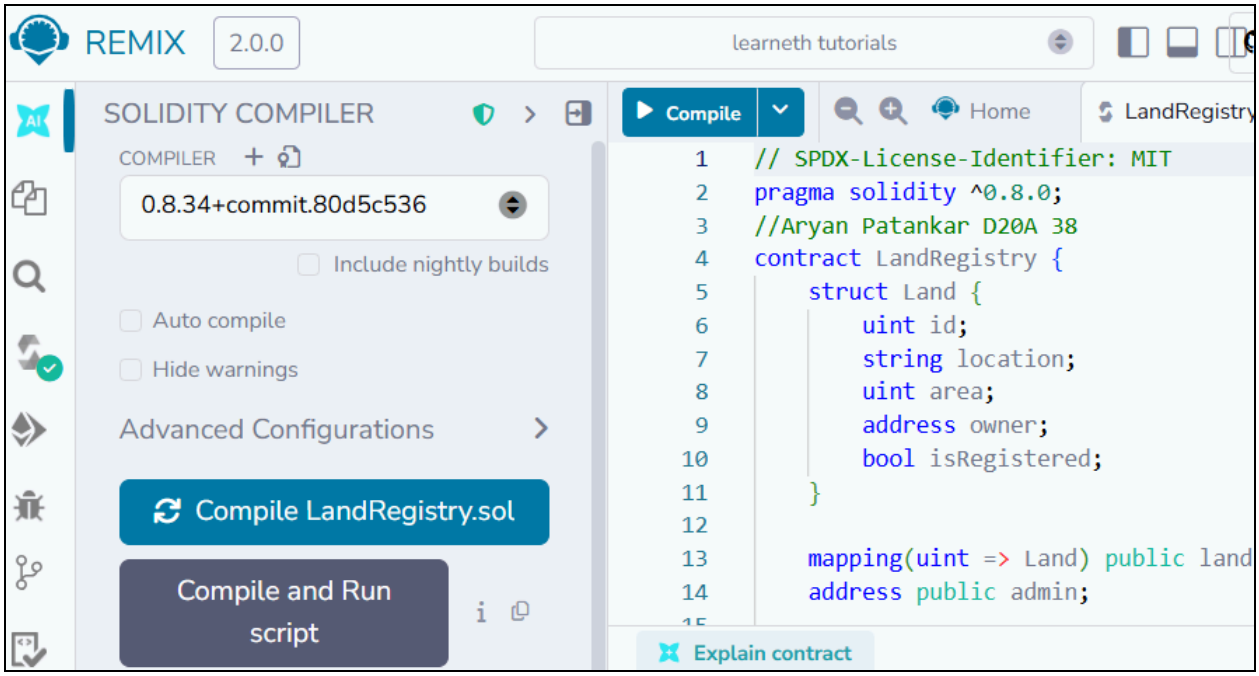


A MetaMask popup will appear; click **Connect** to link your `0x7D11...` account to Remix.



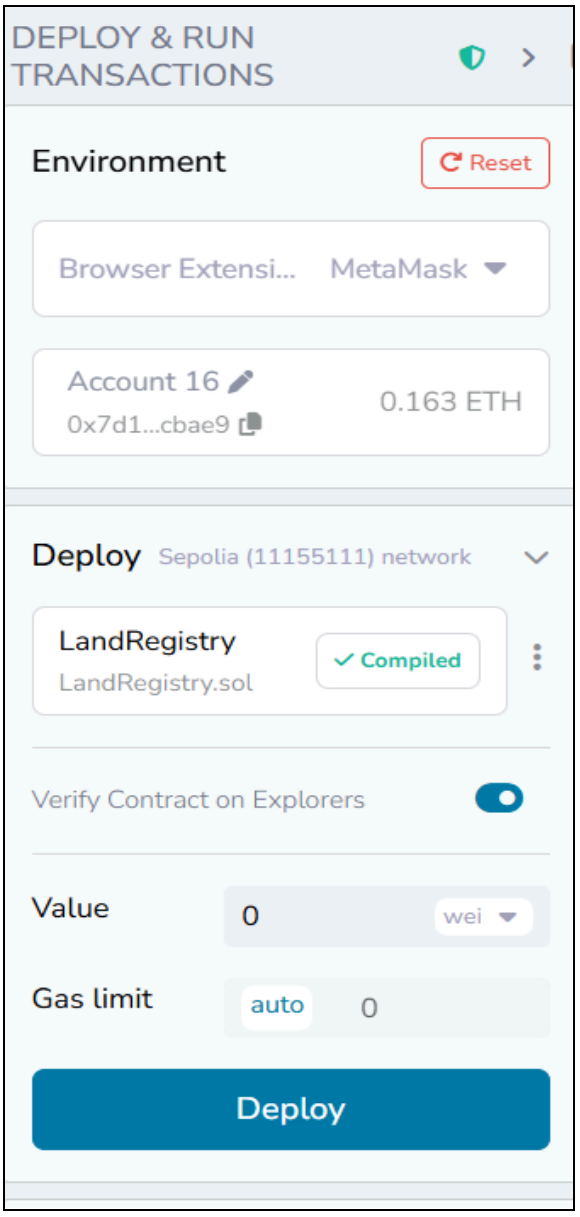
Step 6: Create and Compile the Contract

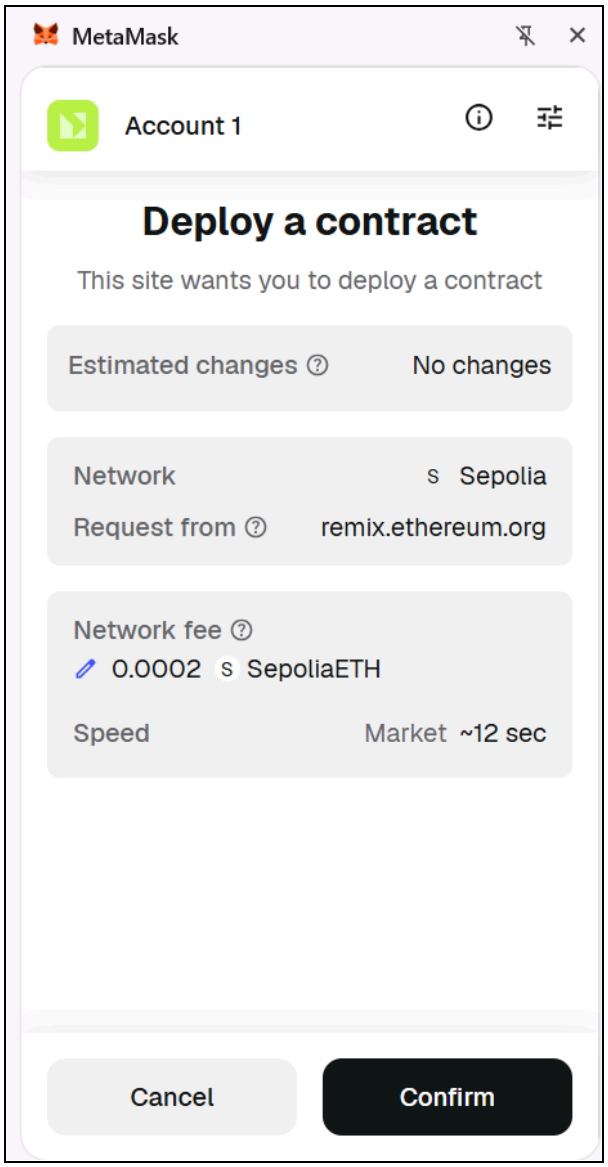
- Open your LandRegistry.sol file.
- Go to the **Solidity Compiler** tab.
- Select Compiler 0.8.0 (or the version in your code).
- Click **Compile LandRegistry.sol**.



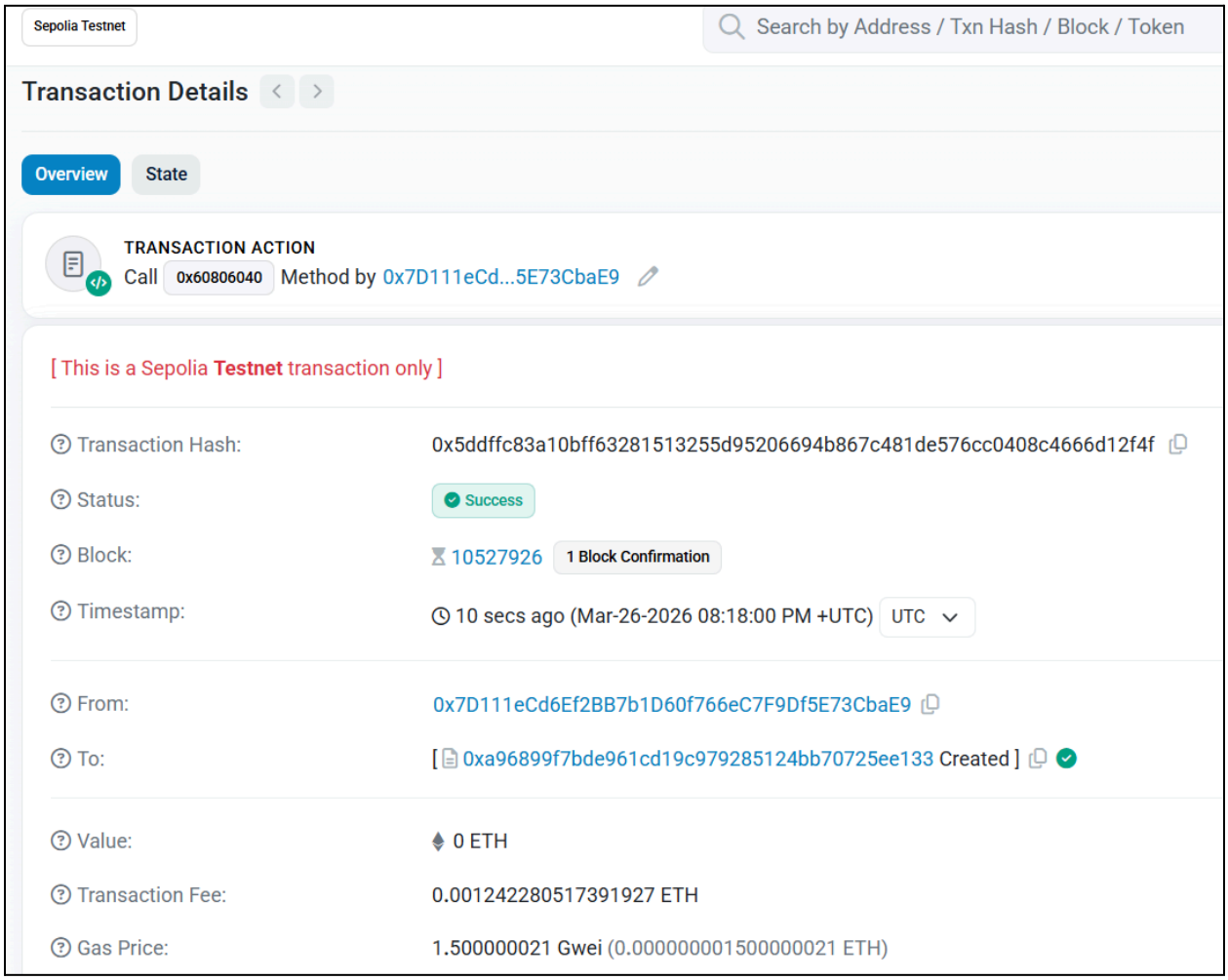
Step 7: Deploy and Verify the transaction

- Go back to the **Deploy & Run Transactions** tab.
- Ensure **Injected Provider** is still selected (you should see "Custom (11155111) network").
- Click the orange **Deploy** button.
- **MetaMask Action:** A popup will show the gas fee. Click **Confirm**.
- **Wait:** Look at the bottom terminal. Wait for the green checkmark that says "transacted."

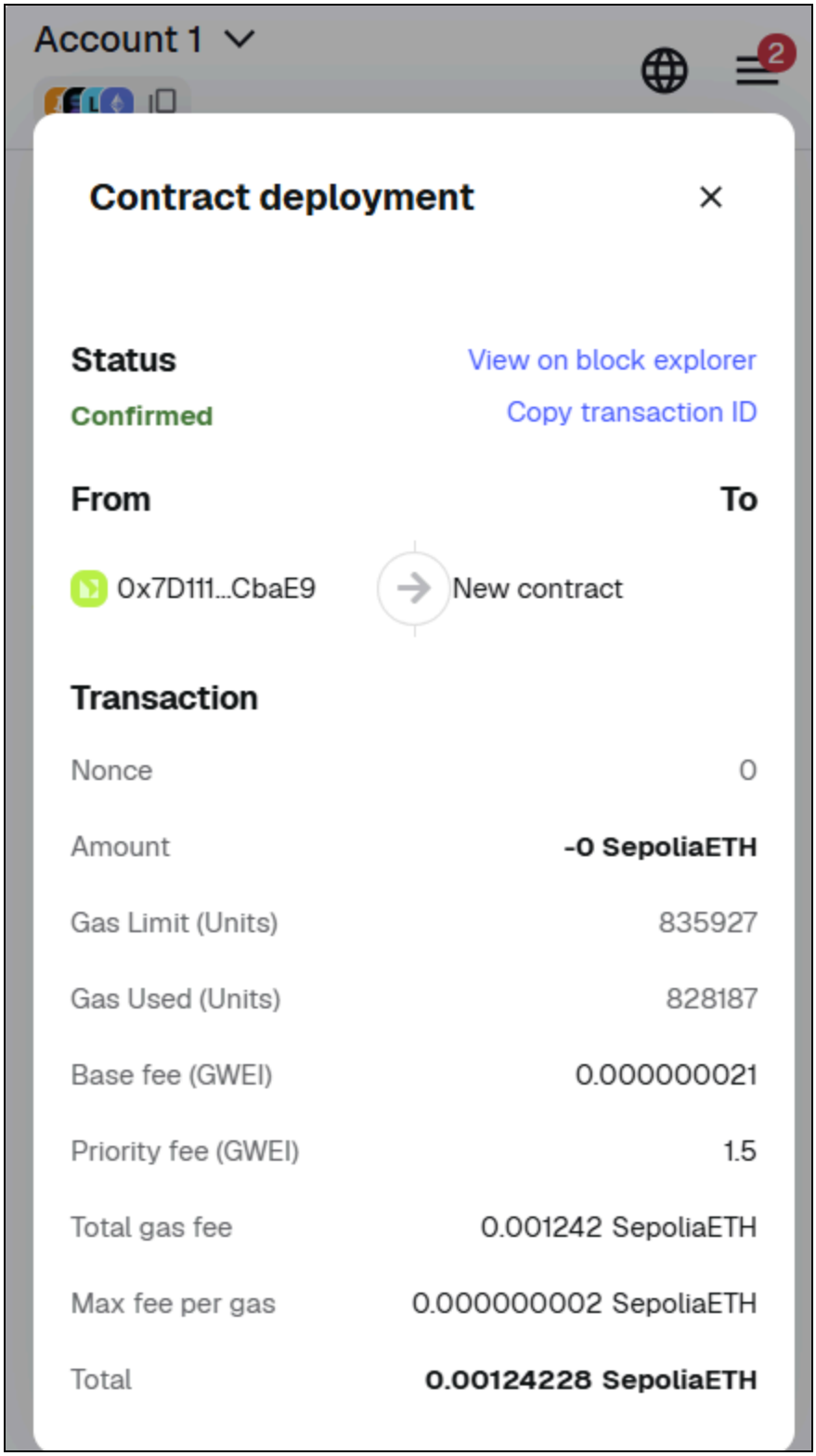


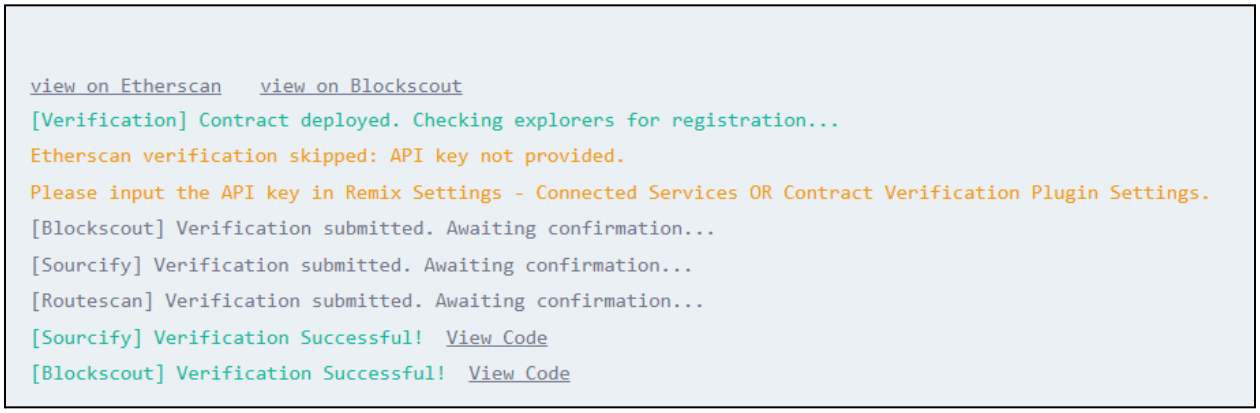


After clicking on deploy button



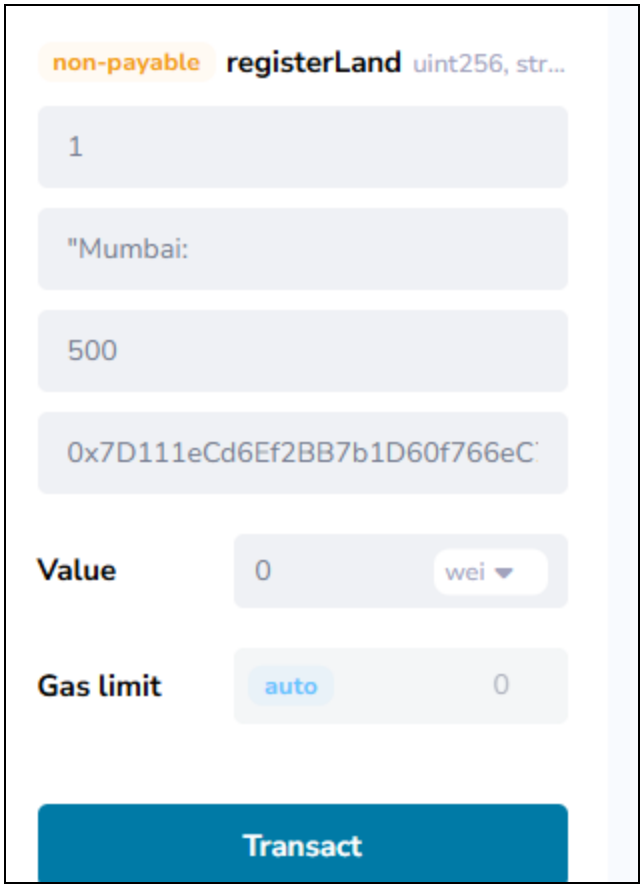
Transaction is verified on Sepolia Etherscan

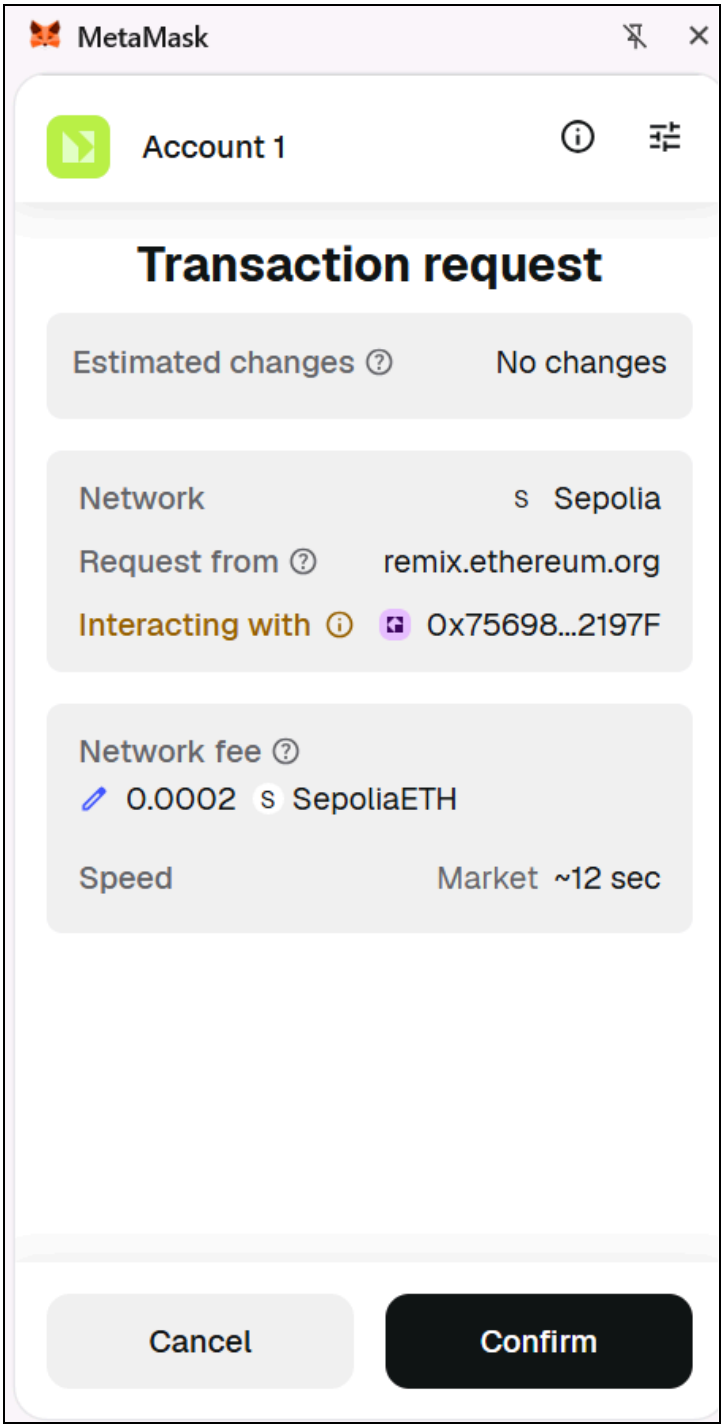

Account 1
2
Contract deployment
Status
View on block explorer
Confirmed
Copy transaction ID
From
To
0x7D111...CbaE9
New contract
Transaction
Nonce
0
Amount
-0 SepoliaETH
Gas Limit (Units)
835927
Gas Used (Units)
828187
Base fee (GWEI)
0.000000021
Priority fee (GWEI)
1.5
Total gas fee
0.001242 SepoliaETH
Max fee per gas
0.000000002 SepoliaETH
Total
0.00124228 SepoliaETH



Step 8: Interact with the Contract

- In Remix, under "Deployed Contracts," expand LandRegistry.
- Run registerLand with ID 1, Location "Mumbai", and your address.
- **MetaMask Action:** You must click **Confirm** in MetaMask again because this action changes the blockchain state.


non-payable
registerLand
uint256, str...
1
"Mumbai:
500
0x7D111eCd6Ef2BB7b1D60f766eC
Value
0
wei
Gas limit
auto
0
Transact


MetaMask
Account 1
Transaction request
Estimated changes
No changes
Network
Sepolia
Request from
remix.ethereum.org
Interacting with
0x75698...2197F
Network fee
0.0002
SepoliaETH
Speed
Market
~12 sec
Cancel
Confirm

Contract interaction

Status

View on block explorer

Confirmed

Copy transaction ID

From

0x7D111...CbaE9

To

0x75698...2197F

Transaction

Nonce	2
Amount	**-0 SepoliaETH**
Gas Limit (Units)	118966
Gas Used (Units)	115835
Base fee (GWEI)	0.000000028
Priority fee (GWEI)	1.5
Total gas fee	0.000174 SepoliaETH
Max fee per gas	0.000000002 SepoliaETH
Total	**0.00017375 SepoliaETH**

Conclusion:-

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.8

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

1. Introduction to Blockchain

Blockchain is a distributed, decentralized digital ledger that records transactions across multiple nodes in a network. Each transaction is stored in a block, and blocks are linked together using cryptographic hashes, forming a secure and immutable chain.

Key features:

- Decentralization – No central authority controls the system
- Immutability – Data cannot be altered once recorded
- Transparency – Transactions are visible to all participants
- Security – Uses cryptographic techniques

2. What is Ganache?

Ganache is a personal blockchain simulator used for developing, testing, and debugging Ethereum-based decentralized applications (DApps).

It is part of the Truffle Suite, which provides tools for smart contract development.

Ganache creates a local blockchain environment that mimics the behavior of the real Ethereum network.

3. Features of Ganache

a. Local Blockchain

Ganache runs a blockchain locally on your system, allowing developers to test without using real cryptocurrency.

b. Pre-funded Accounts

Provides multiple accounts with preloaded Ether
Useful for testing transactions without cost

c. Instant Mining

15. Transactions are mined instantly
16. No waiting time like real networks

d. Detailed Transaction Logs

4. Displays:
    - Transaction hash
    - Gas usage
    - Block number
    - Events

e. Smart Contract Testing

Supports deployment and testing of smart contracts written in Solidity.

f. Customizable Settings

- Gas limit
- Gas price
- Number of accounts
- Network ID

4. Architecture of Ganache

Ganache simulates the Ethereum architecture:

a. Accounts Layer

- Contains multiple Ethereum accounts
- Each account has:
    - Address
    - Private key
    - Balance

b. Blockchain Layer

- Stores blocks and transactions
- Each block contains:
    - Block number
    - Timestamp
    - Transactions

c. Virtual Machine

Ganache uses the **Ethereum Virtual Machine** to execute smart contracts.

d. RPC Server

18. Provides JSON-RPC API
19. Allows interaction using tools like Web3.js

5. Working of Ganache

Ganache starts a local blockchain network
It generates multiple accounts with Ether
Developers deploy smart contracts
Transactions are executed
Ganache mines blocks instantly
Results are displayed in logs

6. Advantages of Using Ganache

No real Ether required
Faster testing (instant mining)
Easy debugging with detailed logs
Safe environment (no risk of real loss)
Ideal for beginners and developers

7. Limitations of Ganache

Not a real decentralized network
Does not simulate real-world latency
Limited scalability testing
Not suitable for production deployment

8. Applications of Ganache

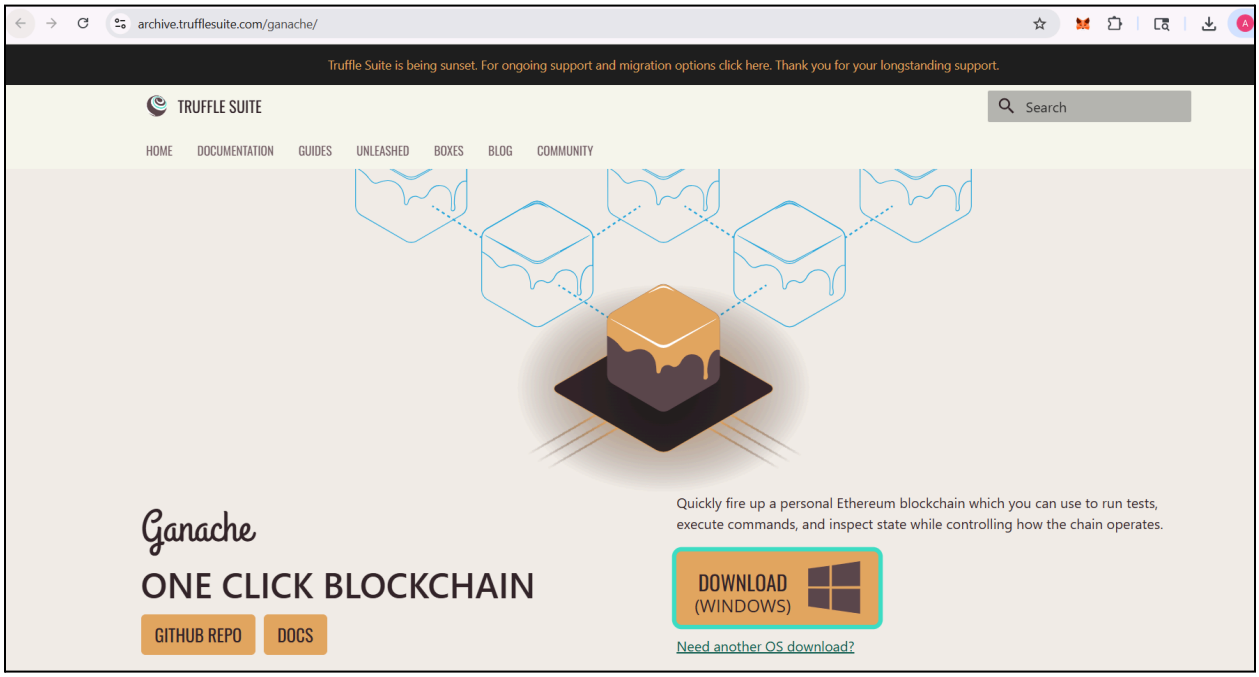
- Smart contract development
- Testing decentralized applications (DApps)
- Learning Ethereum blockchain concepts

- Debugging transactions

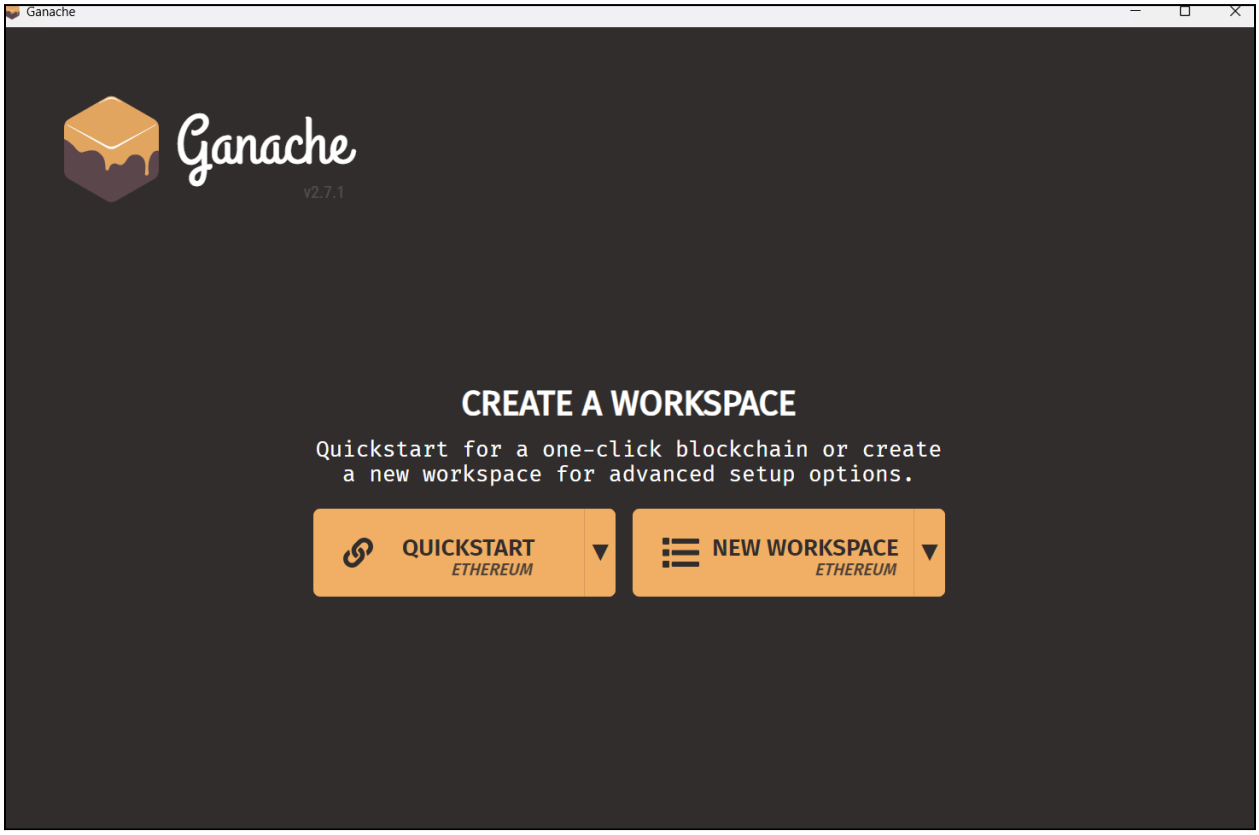
Implementation:-

1. Set Up Ganache (The Local Blockchain)

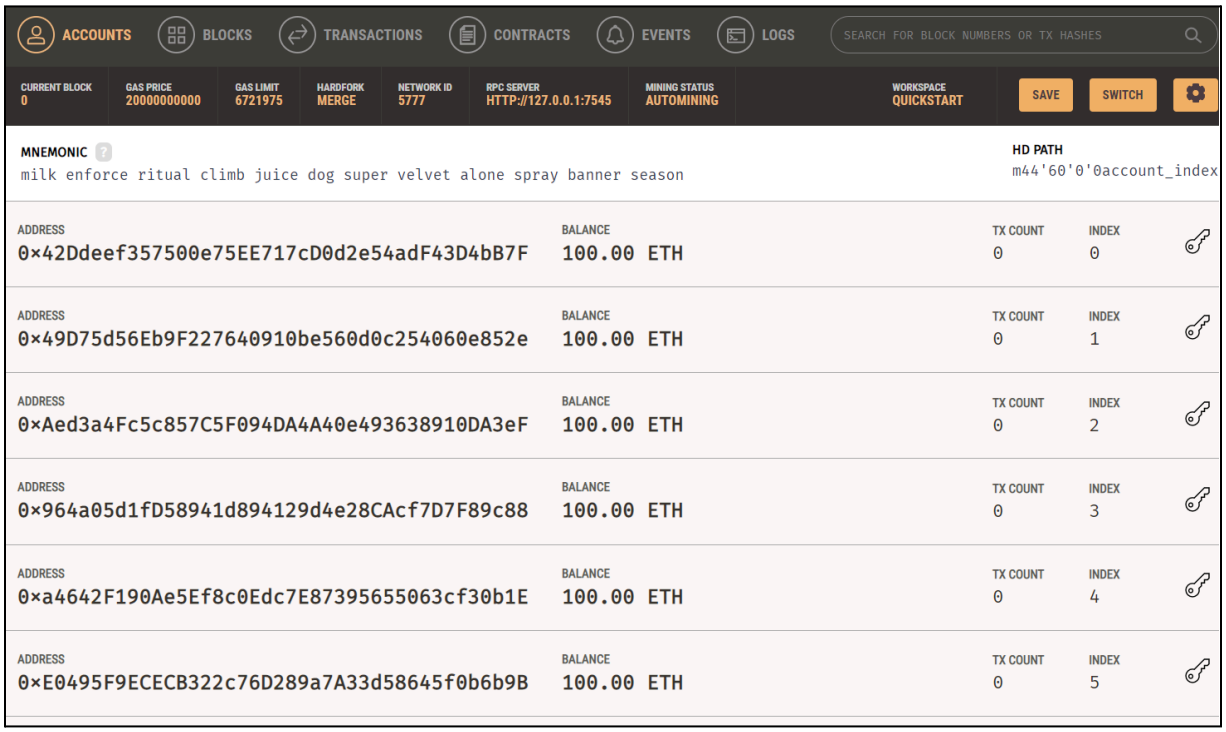
- **Download & Install**: Go to the [Truffle Suite website] and install Ganache.



- **Launch**: Open Ganache and select **"Quickstart"**.



- **Identify Details**: You will see a list of 10 addresses, each with 100 ETH.
    - Look at the top bar for **RPC Server**. It usually says `HTTP://127.0.0.1:7545`.
    - Look at the **Network ID**. It is usually `5777`.

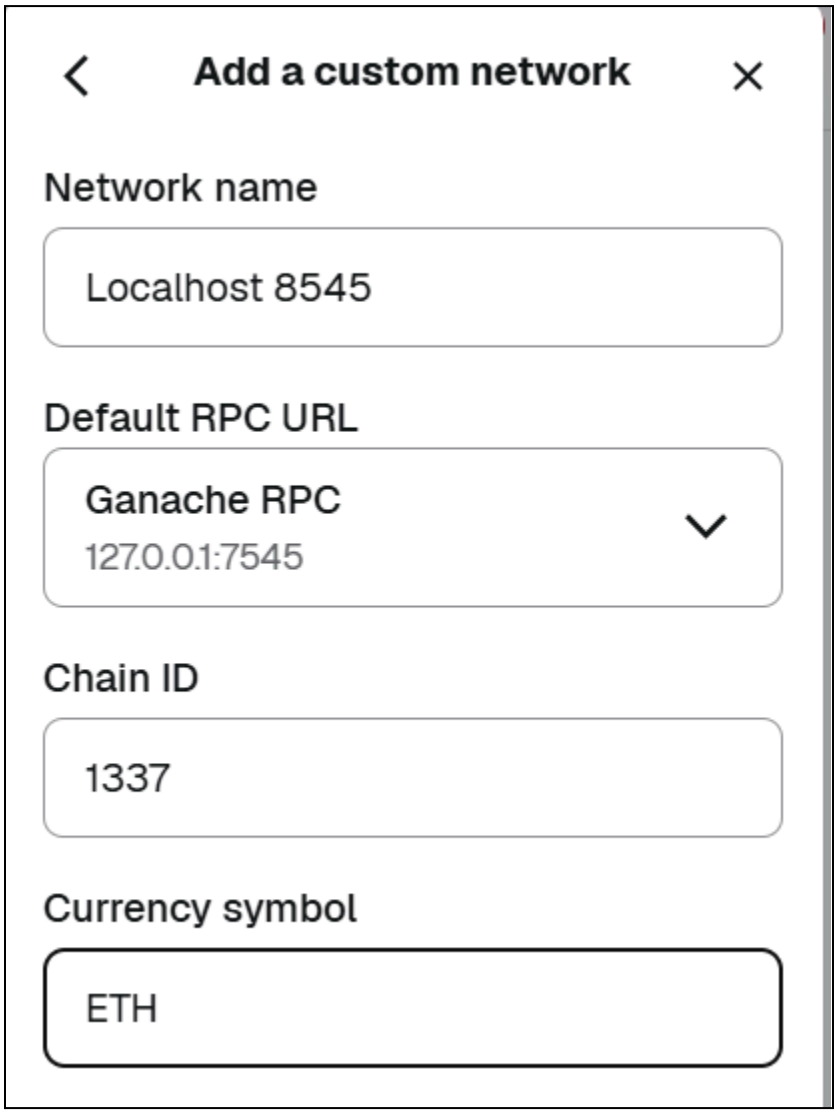


2. Connect Ganache to MetaMask

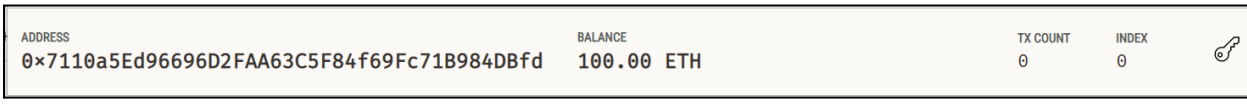
MetaMask doesn't know about your local Ganache "server" yet, so we have to add it manually.

9. Open **MetaMask** -> Click the **Network Dropdown** (top left) -> **Add Network**.
10. Select **"Add a network manually"** at the bottom.
11. Enter these details:
    - **Network Name**: Localhost 8545
    - **New RPC URL**: `http://127.0.0.1:7545`
    - **Chain ID**: `1377`

- **Currency Symbol**: ETH



12. **Import Account**:
    - In Ganache, click the **Key Icon** on the far right of the first address.



    - Copy the **Private Key**.

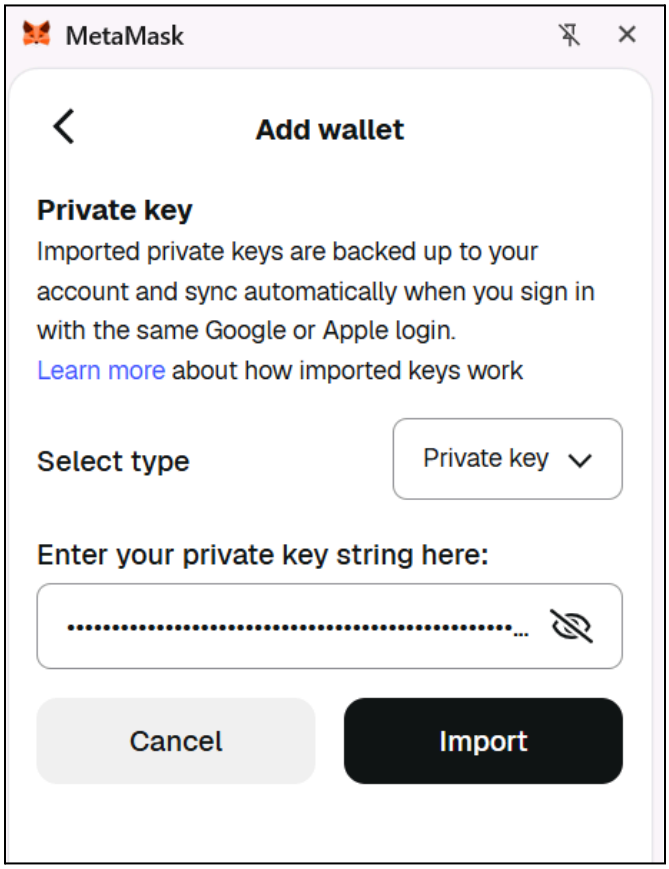
ACCOUNT ADDRESS

0×7110a5Ed96696D2FAA63C5F84f69Fc71B984DBfd

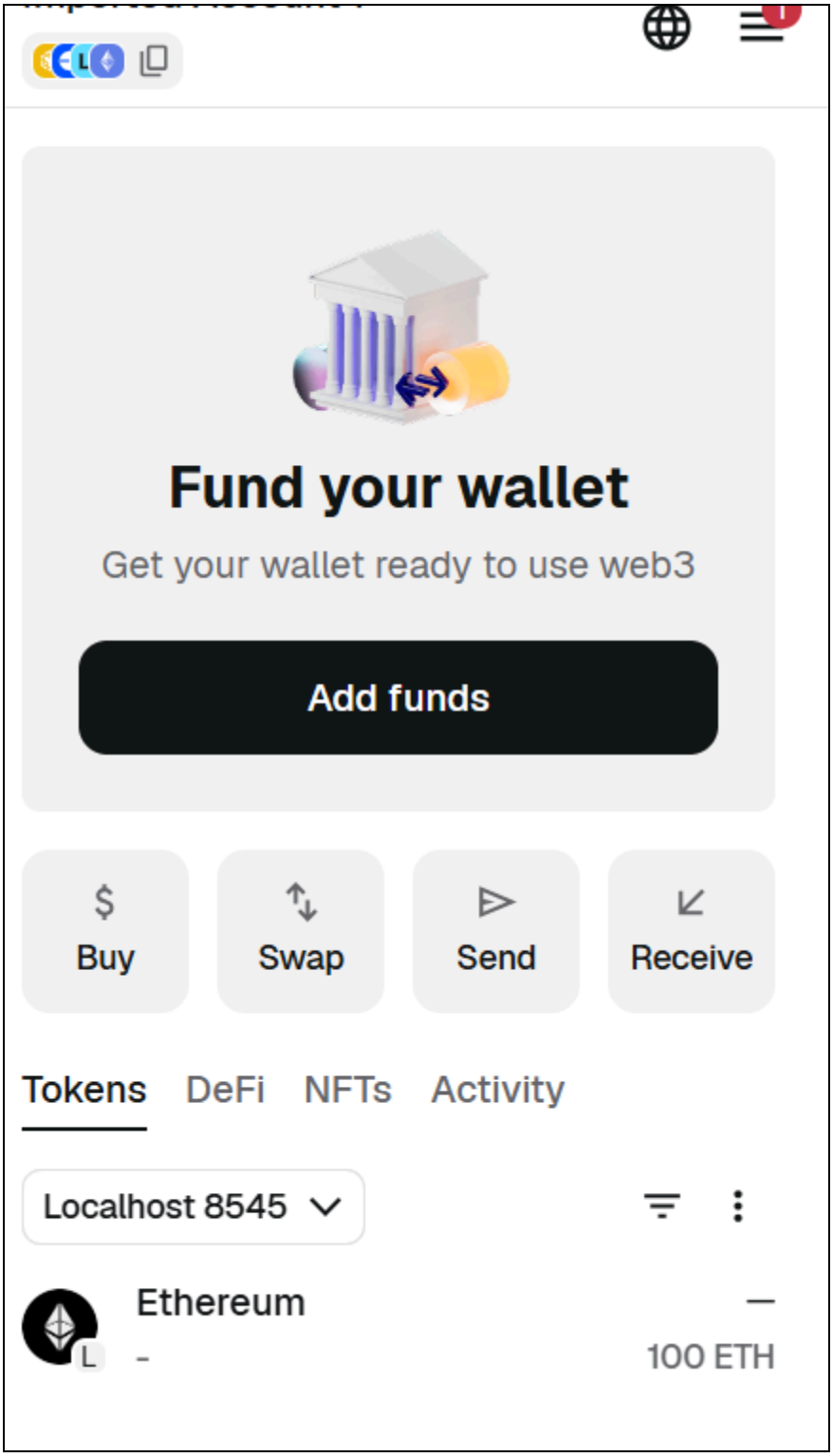
PRIVATE KEY

0×f4a7a11cb7e2aa839bae7d439672da8684ee9d061f2b809eb0c0da4700e845

- In MetaMask, click the Account Circle -> **Import Account** -> Paste the Private Key.

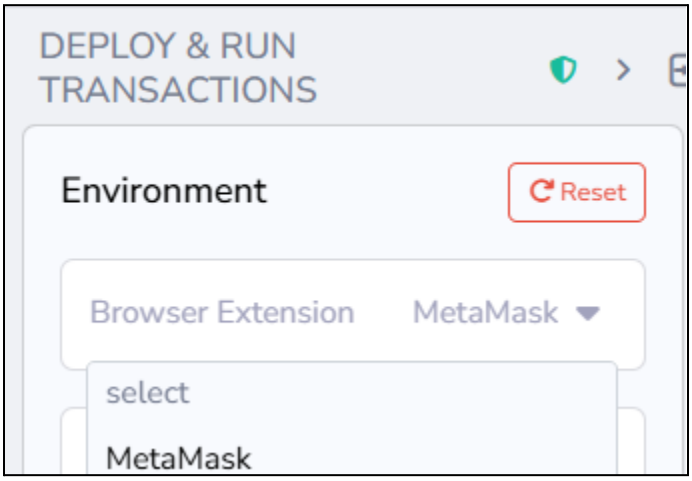


- You should now see **100 ETH** in MetaMask!

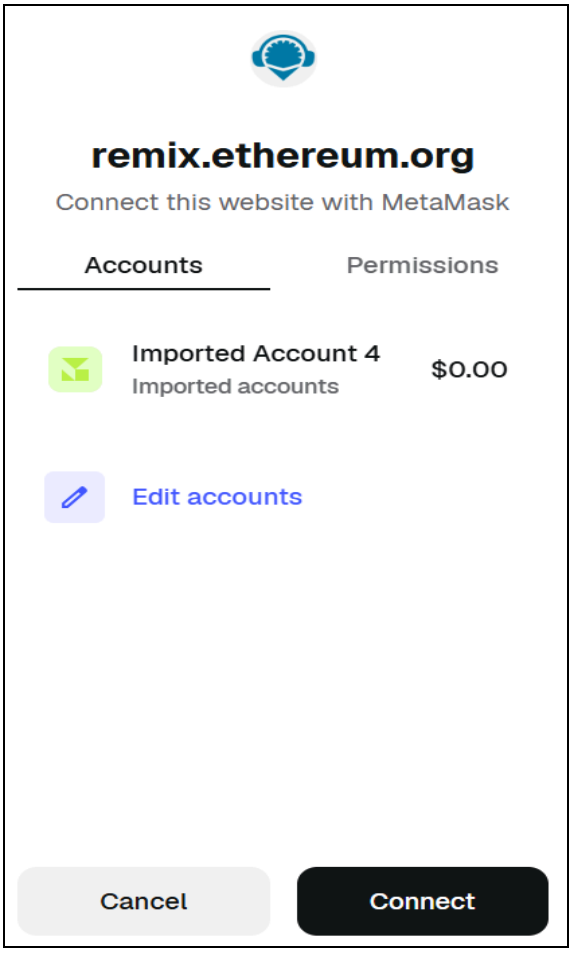


3. Connect Remix to MetaMask (Injected Provider)

In Remix, go to the **Deploy & Run Transactions** tab.
Set **Environment** to **Injected Provider - MetaMask**.

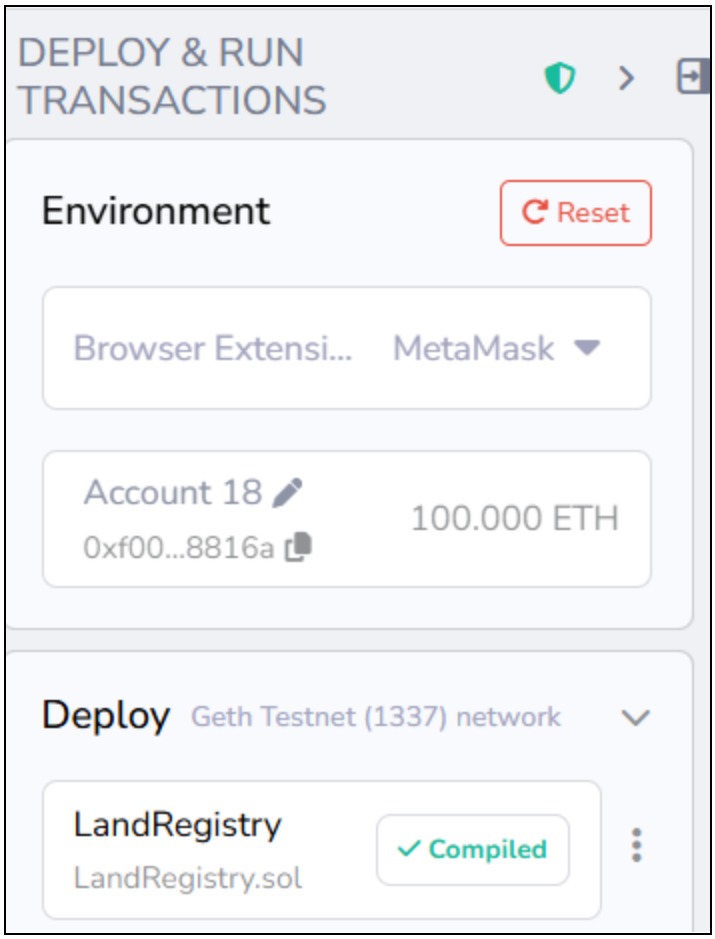


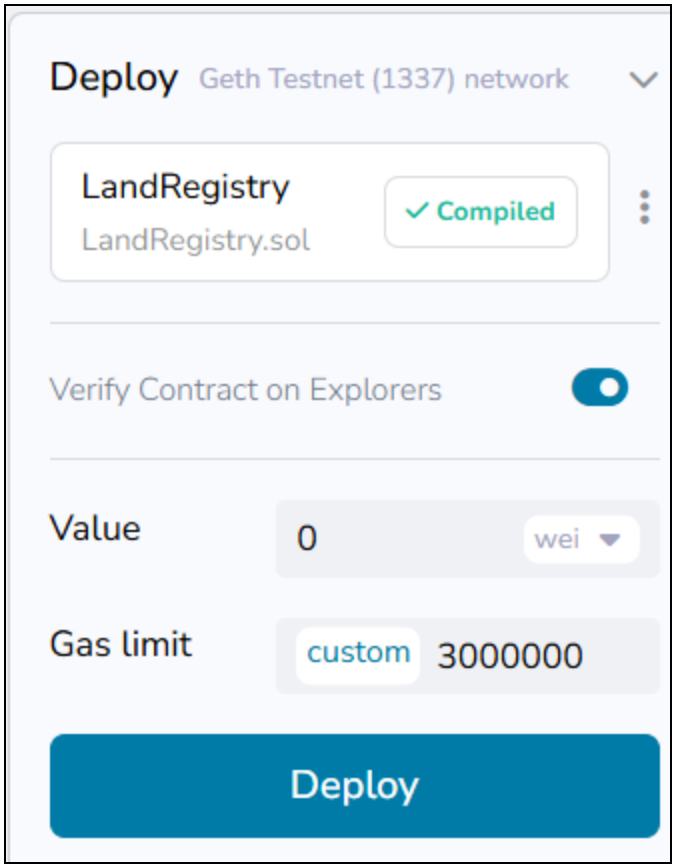
MetaMask will ask to connect to "Ganache Local." Click **Confirm**.



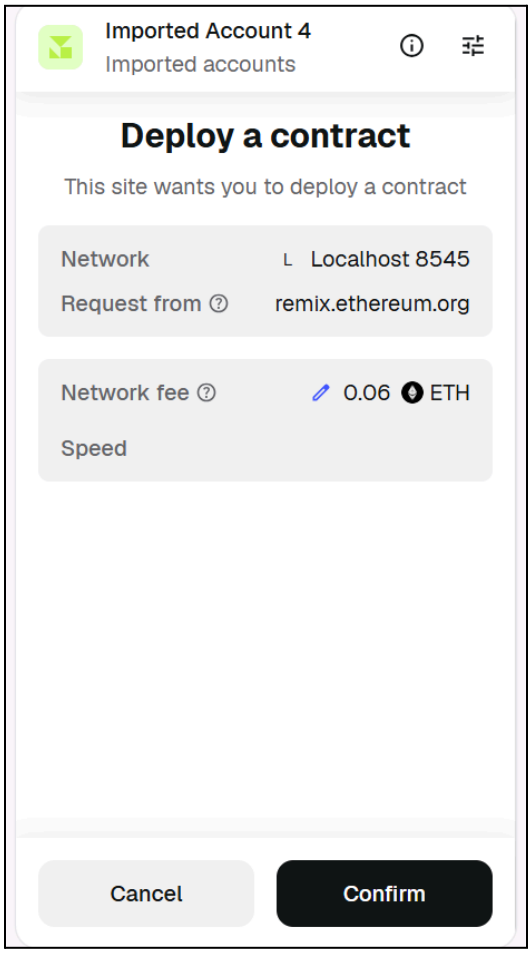
4. Compile and Deploy

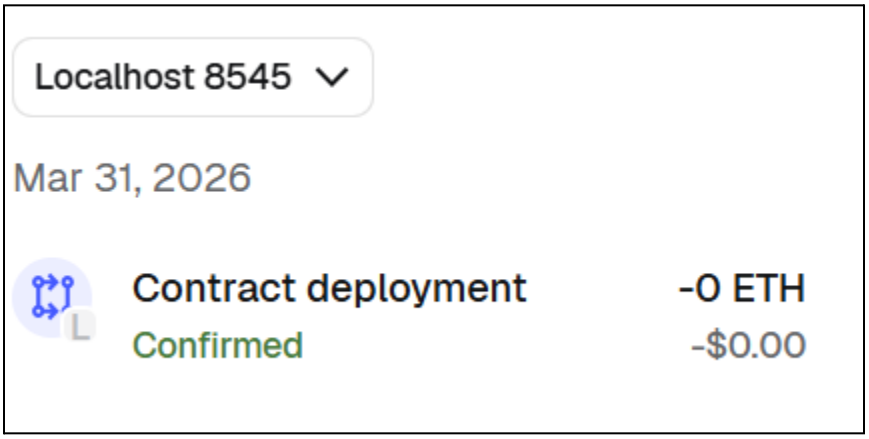
Compile your `LandRegistry.sol`.
In the Deploy tab, click **Deploy**.


DEPLOY & RUN TRANSACTIONS
Environment
Reset
Browser Extensi...
MetaMask
Account 18
0xf00...8816a
100.000 ETH
Deploy
Geth Testnet (1337) network
LandRegistry
LandRegistry.sol
Compiled


Deploy
Geth Testnet (1337) network
LandRegistry
LandRegistry.sol
Compiled
Verify Contract on Explorers
Value
0
wei
Gas limit
custom
3000000
Deploy

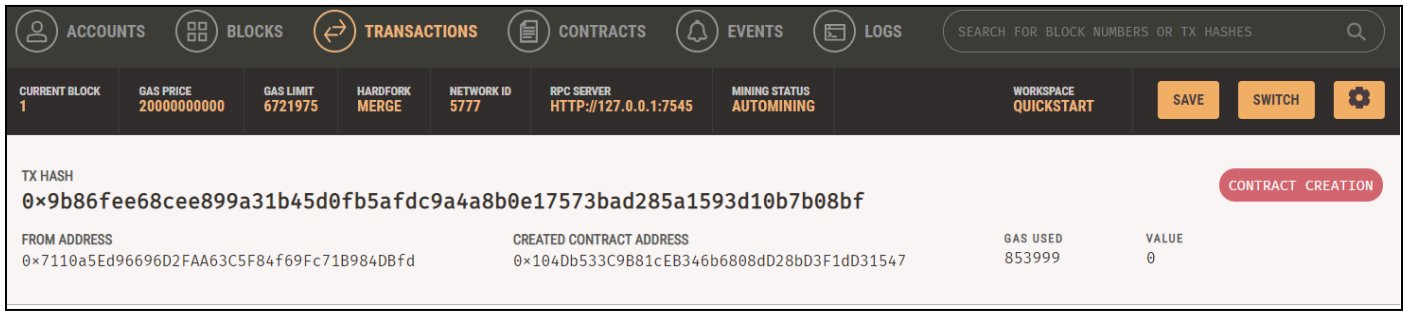
MetaMask will pop up. Since this is your local Ganache, the transaction will be **instant** and the gas cost will be deducted from your 100 ETH.

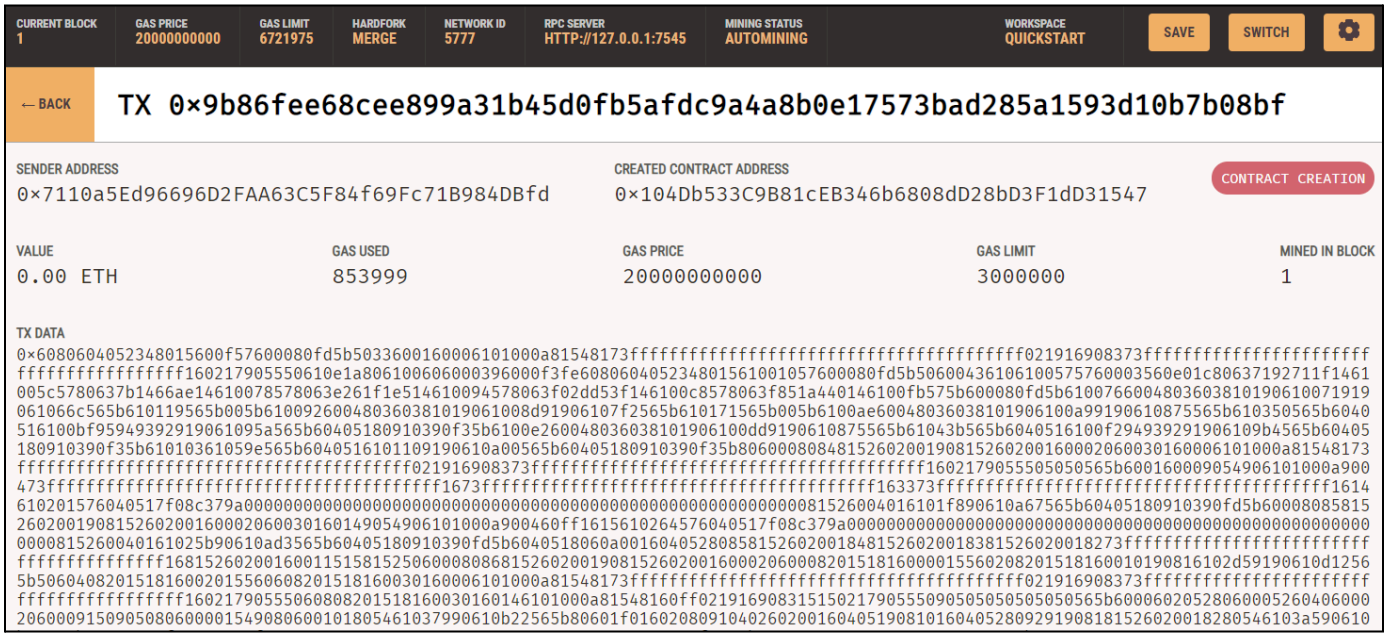




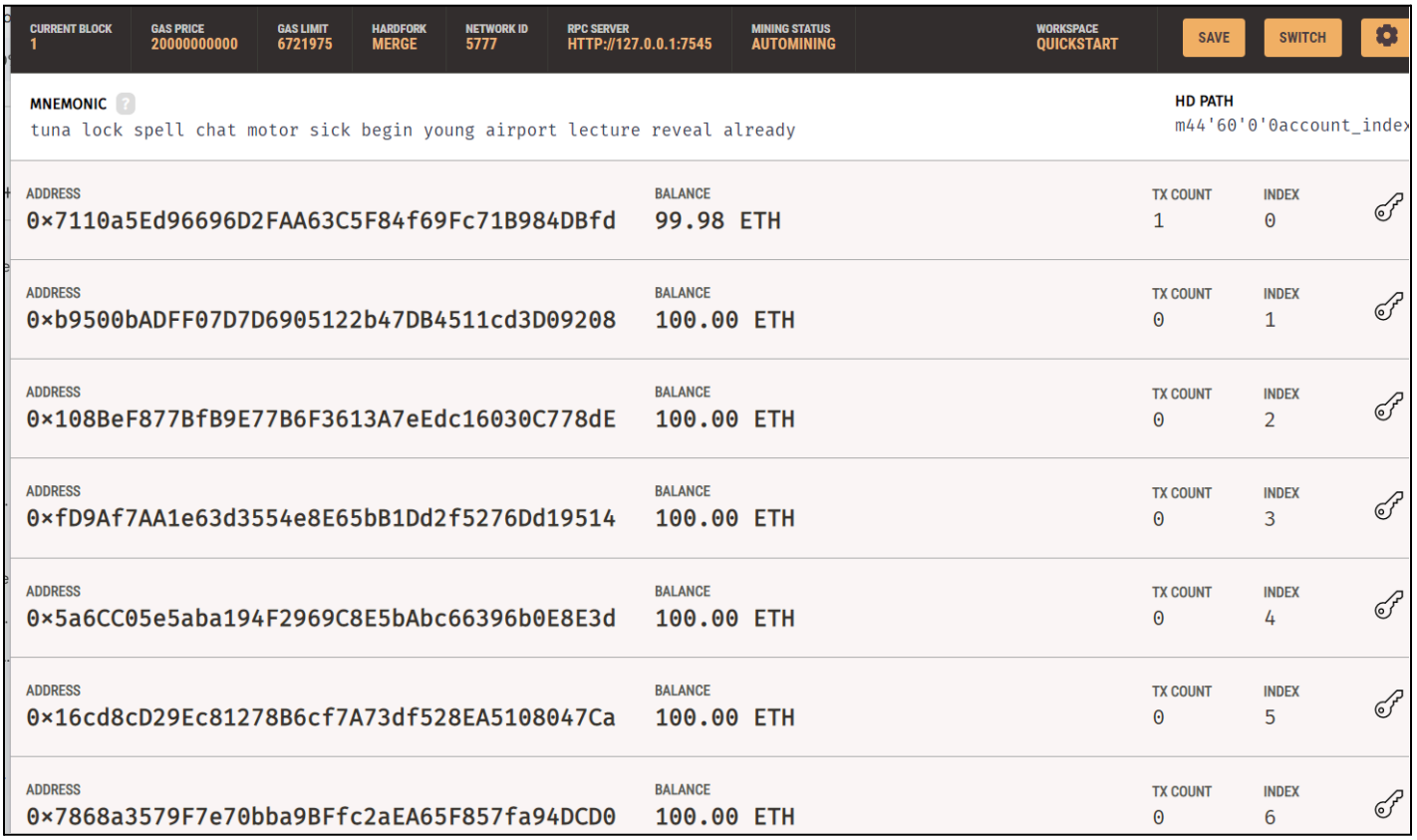
5. Check Transaction Details in Ganache

Open the **Ganache Desktop App**.
Click the **Transactions** tab at the top.
You will see a "Contract Creation" transaction. Click it to see the gas used, the "TX Hash," and the block number





In the **Accounts** tab, we can see that the some balance has been deducted from the first address as part of the transaction that has taken place



Conclusion:-

The experiment successfully demonstrated the deployment of a LandRegistry smart contract on a local blockchain using Ganache, MetaMask, and Remix IDE. By configuring a custom RPC network and aligning the Chain ID, a secure bridge was established to authorize a pre-funded account for transaction gas. This process verified the contract's compilation and its successful migration to a private ledger, confirming the integrity of the decentralized data storage. The setup provides a cost-effective, local environment for testing full-scale blockchain application logic.

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT NO.9

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim: To implement a Private Ethereum Blockchain using Geth.

Theory:
Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (DApps)**. Unlike public blockchains such as the Ethereum mainnet, a **Private Ethereum Blockchain** is restricted to specific participants, making it ideal for **development, testing, and enterprise applications** where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through **JSON-RPC, command-line, or JavaScript consoles**.

Setting up a **Private Ethereum Network** involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

- **Choosing a Network ID:**
- Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
- **Choosing a Consensus Algorithm:**

- Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation.
- **Creating a Genesis Block:**
- The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
- **Initializing the Geth Database:**
- The geth init command initializes the blockchain using the **genesis.json** file, preparing the data directory for node operation.
- **Setting up Networking:**
- Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
- **Running the Member Nodes:**
- Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
- **Running a Signer (in Clique):**
- In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Note: Download the genesis file and edit the account details, including the public keys of all participating peers in the network.

Step 1: Installing Geth on Ubuntu

sudo apt update

```
ubuntu@ubuntu:~/Desktop$ sudo apt update
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Get:4 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease [17.5 kB]
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:6 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [1,519 kB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [436 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [1,303 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [233 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [1,616 kB
]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [271 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/universe i386 Packages [599 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [852 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [163 kB]
Get:15 http://in.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [602 kB]
Get:16 http://in.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [293 kB]
Get:17 http://in.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [1,644 k
B]
Get:18 http://in.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [274 kB]
Get:19 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,060 kB]
Get:20 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe i386 Packages [698 kB]
Get:21 http://in.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [241 kB]
```

Sudo dpkg –configure -a

```
ubuntu@ubuntu:~/Desktop$ sudo dpkg --configure -a
```

sudo apt install software-properties-common

```
ubuntu@ubuntu:~/Desktop$ sudo apt install software-properties-common
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-software-properties software-properties-gtk ubuntu-advantage-tools
  ubuntu-pro-client
Recommended packages:
  ubuntu-pro-client-l10n
The following NEW packages will be installed:
  ubuntu-pro-client
The following packages will be upgraded:
  python3-software-properties software-properties-common software-properties-gtk
  ubuntu-advantage-tools
4 upgraded, 1 newly installed, 0 to remove and 231 not upgraded.
Need to get 322 kB of archives.
After this operation, 1,339 kB disk space will be freed.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 ubuntu-advantage-tools a
ll 31.2~22.04 [10.8 kB]
```

sudo add-apt-repository -y ppa:ethereum/ethereum

```
ubuntu@ubuntu:~/Desktop$ sudo add-apt-repository -y ppa:ethereum/ethereum
Repository: 'deb https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu/ jammy main'
More info: https://launchpad.net/~ethereum/+archive/ubuntu/ethereum
Adding repository.
Found existing deb entry in /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Adding deb entry to /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Found existing deb-src entry in /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.list
Adding disabled deb-src entry to /etc/apt/sources.list.d/ethereum-ubuntu-ethereum-jammy.lis
t
Adding key to /etc/apt/trusted.gpg.d/ethereum-ubuntu-ethereum.gpg with fingerprint 2A518C81
9BE37D2C2031944D1C52189C923F6CA9
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:5 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease
Reading package lists... Done
```

sudo apt update

```
ubuntu@ubuntu:~/Desktop$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 https://ppa.launchpadcontent.net/ethereum/ethereum/ubuntu jammy InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
231 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ubuntu:~/Desktop$
```

sudo apt install geth

```
ubuntu@ubuntu:~/Desktop$ sudo apt install geth
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
geth is already the newest version (1.13.14+build29502+jammy).
geth set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 231 not upgraded.
```

Step 2: Check the version of Geth on the Terminal

geth version

```
ubuntu@ubuntu:~/Desktop$ geth version
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Architecture: amd64
Go Version: go1.21.6
Operating System: linux
GOPATH=
GOROOT=
ubuntu@ubuntu:~/Desktop$
```

3. Step - 3: Create a Private Ethereum Network

1. Create a folder named, private_ethereum_setup

```
ubuntu@ubuntu:~/Desktop$ mkdir private_ethereum_setup
```

2. Create 2 subfolders named node1 and node2 in the folder private_ethereum_setup

```
ubuntu@ubuntu:~/Desktop$ cd private_ethereum_setup/
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ mkdir node1 node2
```

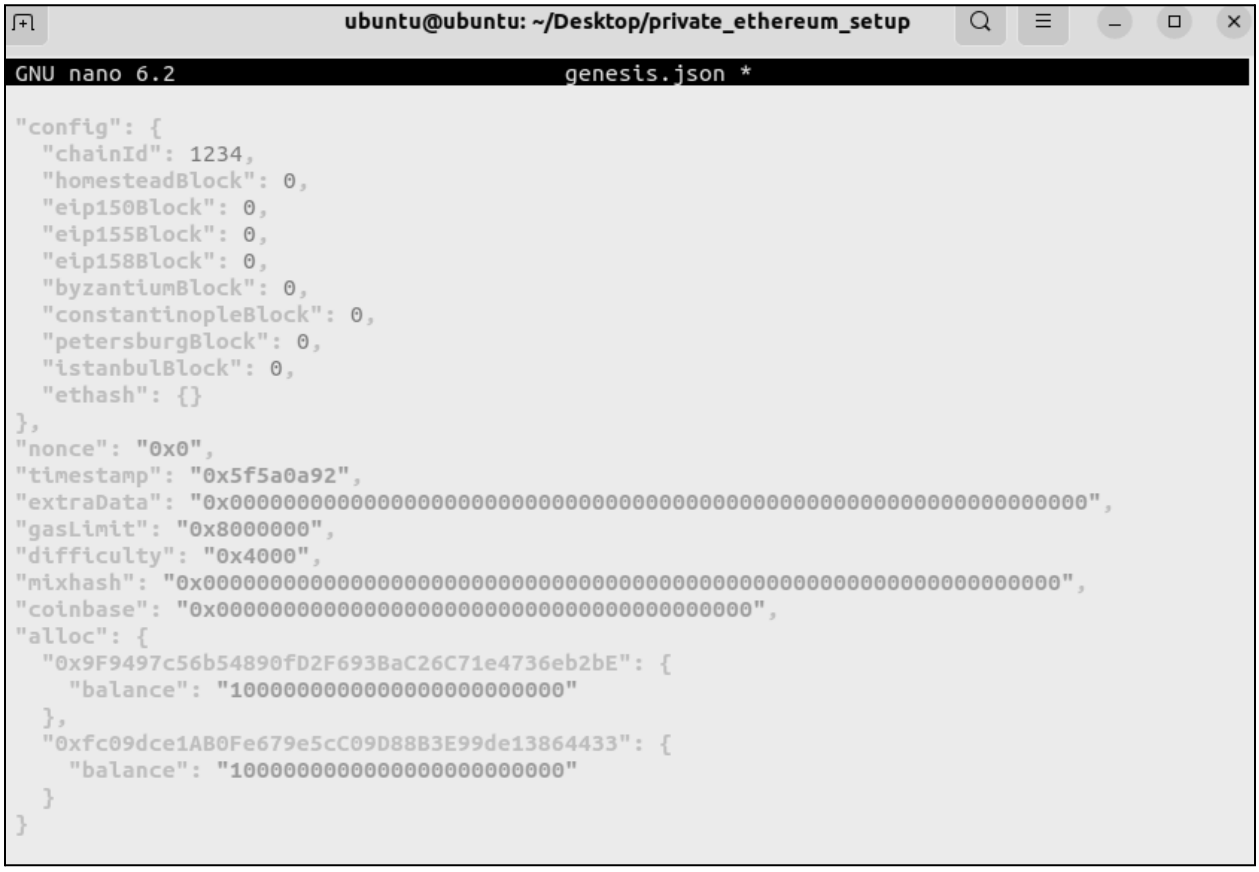
3. Create 2 accounts in the folder corresponding to node1 and node2

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 account new
INFO [03-31|10:20:39.436] Maximum peer count                       ETH=50 total=50
INFO [03-31|10:20:39.438] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.
comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this passwor
d.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE
Path of the secret key file: node1/keystore/UTC--2024-03-31T04-50-51.743374765Z--9f9497c56b548
90fd2f693bac26c71e4736eb2be

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ S
```

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 account new
INFO [03-31|10:21:31.739] Maximum peer count                       ETH=50 total=50
INFO [03-31|10:21:31.741] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.
comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this passwor
d.
Password:
Repeat password:

Your new key was generated

Public address of the key:   0xfc09dce1AB0Fe679e5cC09D88B3E99de13864433
Path of the secret key file: node2/keystore/UTC--2024-03-31T04-51-35.934981574Z--fc09dce1ab0fe
679e5cc09d88b3e99de13864433

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

ubuntu@ubuntu:~/Desktop/private_ethereum_setup$
```

4. Create a genesis.json file in the folder, private_ethereum_setup

nano genesis.json



5. Initialize the nodes with the genesis file

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 init genesis.json
INFO [03-31|10:33:21.842] Maximum peer count                       ETH=50 total=50
INFO [03-31|10:33:21.843] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.
comm: no such file or directory"
INFO [03-31|10:33:21.847] Set global gas cap                       cap=50,000,000
INFO [03-31|10:33:21.850] Initializing the KZG library             backend=gokzg
INFO [03-31|10:33:21.919] Defaulting to pebble as the backing database
INFO [03-31|10:33:21.919] Allocated cache and file handles         database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/chaindata cache=16.00MiB handles=16
INFO [03-31|10:33:21.942] Opened ancient database                  database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/chaindata/ancient/chain readonly=false
INFO [03-31|10:33:21.943] State schema set to default              scheme=hash
INFO [03-31|10:33:21.943] Writing custom genesis block
INFO [03-31|10:33:21.945] Persisted trie from memory database      nodes=4 size=480.00B time="
609.399µs" gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:21.953] Successfully wrote genesis state         database=chaindata hash=c63
8fa..fe1846
INFO [03-31|10:33:21.953] Defaulting to pebble as the backing database
INFO [03-31|10:33:21.953] Allocated cache and file handles         database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/lightchaindata cache=16.00MiB handles=16
INFO [03-31|10:33:21.962] Opened ancient database                  database=/home/ubuntu/Deskt
op/private_ethereum_setup/node1/geth/lightchaindata/ancient/chain readonly=false
INFO [03-31|10:33:21.962] State schema set to default              scheme=hash
INFO [03-31|10:33:21.963] Writing custom genesis block
INFO [03-31|10:33:21.964] Persisted trie from memory database      nodes=4 size=480.00B time="
811.122µs" gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:21.969] Successfully wrote genesis state         database=lightchaindata has
h=c638fa..fe1846
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ S
```

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 init genesis.json
INFO [03-31|10:33:56.477] Maximum peer count                       ETH=50 total=50
INFO [03-31|10:33:56.479] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.
comm: no such file or directory"
INFO [03-31|10:33:56.485] Set global gas cap                       cap=50,000,000
INFO [03-31|10:33:56.486] Initializing the KZG library             backend=gokzg
INFO [03-31|10:33:56.571] Defaulting to pebble as the backing database
INFO [03-31|10:33:56.571] Allocated cache and file handles         database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/chaindata cache=16.00MiB handles=16
INFO [03-31|10:33:56.583] Opened ancient database                  database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/chaindata/ancient/chain readonly=false
INFO [03-31|10:33:56.583] State schema set to default              scheme=hash
INFO [03-31|10:33:56.583] Writing custom genesis block
INFO [03-31|10:33:56.584] Persisted trie from memory database      nodes=4 size=480.00B time="
695.266µs" gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:56.590] Successfully wrote genesis state         database=chaindata hash=c63
8fa..fe1846
INFO [03-31|10:33:56.590] Defaulting to pebble as the backing database
INFO [03-31|10:33:56.590] Allocated cache and file handles         database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/lightchaindata cache=16.00MiB handles=16
INFO [03-31|10:33:56.602] Opened ancient database                  database=/home/ubuntu/Deskt
op/private_ethereum_setup/node2/geth/lightchaindata/ancient/chain readonly=false
INFO [03-31|10:33:56.602] State schema set to default              scheme=hash
INFO [03-31|10:33:56.602] Writing custom genesis block
INFO [03-31|10:33:56.603] Persisted trie from memory database      nodes=4 size=480.00B time="
549.872µs" gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|10:33:56.609] Successfully wrote genesis state         database=lightchaindata has
h=c638fa..fe1846
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$
```

6. For configuring the boot node

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ sudo apt-get install bootnode
[sudo] password for ubuntu:
Sorry, try again.
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bootnode is already the newest version (1.13.14+build29502+jammy).
bootnode set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 231 not upgraded.
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$
```

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -genkey boot.key
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -nodekey boot.key
enode://fbbb379450b32a560c1adbc01f89cd7bf7434c64d3d61140e35f01e4dc6872e079e610606a8127a161a0ed
9d7be778636b22552290f7aaabad2e8232ed823eee@127.0.0.1:0?discport=30301
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [03-31|10:35:18.634] New local node record                    seq=1,711,861,518,633 id=ba
197519c6e384da ip=<nil> udp=0 tcp=0
```

4. Step - 4: Establish a Peer-Peer Connection between the nodes along with the bootnode

1. On the first Terminal, Use boot.key to run the boot node

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ bootnode -verbosity 5 -nodekey boot.key -addr
127.0.0.1:30305
enode://fbbb379450b32a560c1adbc01f89cd7bf7434c64d3d61140e35f01e4dc6872e079e610606a8127a161a0ed
9d7be778636b22552290f7aaabad2e8232ed823eee@127.0.0.1:0?discport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [03-31|10:40:06.489] New local node record                    seq=1,711,861,806,489 id=ba
197519c6e384da ip=<nil> udp=0 tcp=0
```

2. On the second Terminal, Run Node 1

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node1 --port 30306 --bootnodes enode://f7aba85ba369
923bffd3438b4c8fde6b1f02b1c23ea0aac825ed7eac38e6230e5cadcf868e73b0e28710f4c9f685ca71a86a4911461637ae9ab2bd852939b7
7f@127.0.0.1:0?discport=30305  --networkid 1234 --unlock 0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE --password "./
password.txt" --authrpc.port 8551 --mine --miner.etherbase 0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE
INFO [03-31|11:25:31.763] Maximum peer count                       ETH=50 LES=0 total=50
INFO [03-31|11:25:31.765] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file o
r directory"
INFO [03-31|11:25:31.769] Set global gas cap                       cap=50,000,000
INFO [03-31|11:25:31.771] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [03-31|11:25:31.771] Using pebble as the backing database
INFO [03-31|11:25:31.771] Allocated cache and file handles         database=/home/ubuntu/Desktop/private_ethereum_
setup/node1/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|11:25:31.785] Opened ancient database                  database=/home/ubuntu/Desktop/private_ethereum_
setup/node1/geth/chaindata/ancient/chain readonly=false
INFO [03-31|11:25:31.792] Disk storage enabled for ethash caches   dir=/home/ubuntu/Desktop/private_ethereum_setup
/node1/geth/ethash count=3
INFO [03-31|11:25:31.796] Disk storage enabled for ethash DAGs     dir=/home/ubuntu/.ethash count=2
INFO [03-31|11:25:31.797] Initialising Ethereum protocol           network=1234 dbversion=8
INFO [03-31|11:25:31.797]
INFO [03-31|11:25:31.797] ------------------------------------------------------------------------------------
------------------------------------------------------------
INFO [03-31|11:25:31.797] Chain ID:  1234 (unknown)
INFO [03-31|11:25:31.797] Consensus: Ethash (proof-of-work)
INFO [03-31|11:25:31.797]
INFO [03-31|11:25:31.797] Pre-Merge hard forks (block based):
INFO [03-31|11:25:31.797]  - Homestead:                    #0        (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [03-31|11:25:31.797]  - Tangerine Whistle (EIP 150): #0        (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [03-31|11:25:31.797]  - Spurious Dragon/1 (EIP 155): #0        (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [03-31|11:25:31.797]  - Spurious Dragon/2 (EIP 158): #0        (https://github.com/ethereum/execution-specs/b
```

```
ubuntu@ubuntu: ~/Desktop/private_ethereum_setup  ×   ubuntu@ubuntu: ~/Desktop/private_ethereum_setup  ×
il>
TRACE[03-31|11:27:49.599] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:50.100] << FINDNODE/v4                           id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:50.100] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:50.601] << FINDNODE/v4                           id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:50.601] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:51.102] << FINDNODE/v4                           id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:51.102] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:51.602] << FINDNODE/v4                           id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:51.602] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:51.625] << PONG/v4                               id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:51.625] >> PING/v4                               id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
DEBUG[03-31|11:27:51.625] Revalidated node                         b=16 id=1af3a1e4b66753ff checks=2
TRACE[03-31|11:27:52.103] << FINDNODE/v4                           id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:52.104] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:52.604] << FINDNODE/v4                           id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:52.604] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:53.105] << FINDNODE/v4                           id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
TRACE[03-31|11:27:53.105] >> NEIGHBORS/v4                          id=1af3a1e4b66753ff addr=127.0.0.1:30306 err=<n
il>
```

3. On the third Terminal, Run Node 2

```
ubuntu@ubuntu:~/Desktop/private_ethereum_setup$ geth --datadir node2 --port 30307 --bootnodes enode://f7aba85ba369
923bffd3438b4c8fde6b1f02b1c23ea0aac825ed7eac38e6230e5cadcf868e73b0e28710f4c9f685ca71a86a4911461637ae9ab2bd852939b7
7f@127.0.0.1:0?discport=30305  --networkid 1234 --keystore "./node2/keystore" --unlock 0xfc09dce1AB0Fe679e5cC09D88
B3E99de13864433 --password "./password.txt" console
```

```
INFO [03-31|11:56:49.157] Maximum peer count                       ETH=50 LES=0 total=50
INFO [03-31|11:56:49.159] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file o
r directory"
INFO [03-31|11:56:49.164] Set global gas cap                       cap=50,000,000
INFO [03-31|11:56:49.167] Allocated trie memory caches             clean=154.00MiB dirty=256.00MiB
INFO [03-31|11:56:49.167] Using pebble as the backing database
INFO [03-31|11:56:49.167] Allocated cache and file handles         database=/home/ubuntu/Desktop/private_ethereum_
setup/node2/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|11:56:49.192] Opened ancient database                  database=/home/ubuntu/Desktop/private_ethereum_
setup/node2/geth/chaindata/ancient/chain readonly=false
INFO [03-31|11:56:49.202] Disk storage enabled for ethash caches   dir=/home/ubuntu/Desktop/private_ethereum_setup
/node2/geth/ethash count=3
INFO [03-31|11:56:49.202] Disk storage enabled for ethash DAGs     dir=/home/ubuntu/.ethash count=2
INFO [03-31|11:56:49.203] Initialising Ethereum protocol           network=1234 dbversion=8
INFO [03-31|11:56:49.204]
INFO [03-31|11:56:49.204] ------------------------------------------------------------------------------------
------------------------------------------------------------
INFO [03-31|11:56:49.204] Chain ID:  1234 (unknown)
INFO [03-31|11:56:49.205] Consensus: Ethash (proof-of-work)
INFO [03-31|11:56:49.205]
INFO [03-31|11:56:49.205] Pre-Merge hard forks (block based):
INFO [03-31|11:56:49.205]  - Homestead:                    #0       (https://github.com/ethereum/execution-specs/b
lob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [03-31|11:56:49.205]  - Tangerine Whistle (EIP 150): #0       (https://github.com/ethereum/execution-specs/b
```

5. Step - 5 : Exploring the network by attaching JavaScript console to Node 1

1. Fetch network status

```
> net
{
  listening: true,
  peerCount: 1,
  version: "1234",
  getListening: function(callback),
  getPeerCount: function(callback),
  getVersion: function(callback)
}
```

2. To fetch the number of blocks mined

```
> web3.eth.blockNumber
0
> INFO [03-23|12:15:29.133] Looking for peers                        peercount=1
 tried=0 static=0
```

3. To check the balance of the accounts

```
> eth.getBalance(eth.accounts[0])
1.000322e+24
> INFO [03-31|12:09:54.727] Generating DAG in progress
```

4. To fetch the details of the latest mined block

```
> eth.getBlock("latest")
{
  difficulty: 141501,
  extraData: "0xd883010b06846765746888676f312e32302e33856c696e7578",
  gasLimit: 114681864,
  gasUsed: 0,
  hash: "0x971fd7f7cada2a9922fad1ea258680d8b47aeb76858c0e2e1c668088af95dd46",
  logsBloom: "0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000",
  miner: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
  mixHash: "0x04711352f8591ce3fe869586e2d889e92a2affa5f30d66d81d2394655fac5eed",
  nonce: "0x074eaadf2f84ee20",
  number: 161,
  parentHash: "0x4bb099d8e334270cdfd4f9014a81c05fec8e5b5d690236a6c3c3de27fe945a0e",
  receiptsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",
  sha3Uncles: "0x1dcc4de8dec75d7aab85b567b6ccd41ad312451b948a7413f0a142fd40d49347",
  size: 538,
  stateRoot: "0x3c5c06751cbcf16f9ce7a8ae4a7593bece60d735f93c9573d92c8082cb678f33",
  timestamp: 1711867051,
  totalDifficulty: 21944603,
  transactions: [],
  transactionsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",
  uncles: []
}
> 
```

5. To fetch the details of a specific block

```
> eth.getBlock(100)
{
  difficulty: 137447,
  extraData: "0xd883010b06846765746888676f312e32302e33856c696e7578",
  gasLimit: 121724529,
  gasUsed: 0,
  hash: "0x70d88d2e2b6adbc5f77a303047feab3860428d9cf767be8a1ed614a056901a57",
  logsBloom: "0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000",
  miner: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
  mixHash: "0xb0dd82c44a786d1a23d3cd2e1c7951e30cafe30ed469fc1af7b7b4410ac24d1c",
  nonce: "0x284e830e78c6a90e",
  number: 100,
  parentHash: "0x4ddc65426bffb820125bb925968d6268d30c933ff67991080ffbca37a9510cbc",
  receiptsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",
  sha3Uncles: "0x1dcc4de8dec75d7aab85b567b6ccd41ad312451b948a7413f0a142fd40d49347",
  size: 537,
  stateRoot: "0x5d295a7b8ada8718850a4676e054e6e2dea72b739a3b2e4a4233b8be9243dea8",
  timestamp: 1711866955,
  totalDifficulty: 13436860,
  transactions: [],
  transactionsRoot: "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",
  uncles: []
}
> INFO [03-31|12:12:07.308] Looking for peers                        peercount=1 tried=0 static=0
INFO [03-31|12:12:17.326] Looking for peers                        peercount=1 tried=0 static=0
```

6. To check the account balance of the peer machine, provide their Public Key

```
> eth.getBalance("0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE")
1.000322e+24
> INFO [03-31|12:13:17.464] Looking for peers
>
```

7. Fetch the details of the peers in the network

```
> admin.peers
[{
    caps: ["eth/66", "eth/67", "eth/68", "snap/1"],
    enode: "enode://dce739f3f2fae54667568d84488b8d055409c862e80d78d653bf086b250ddd431fa6c6342502f9ff8f99129b5ee0
55679fabc9335a9049ec7ae1b13941bb265e@127.0.0.1:30307",
    id: "ebf13aa79411d6e216f71bee7625c7391e7a2049d3853aed09ffb5566981dc27",
    name: "Geth/v1.11.6-stable-ea9e62ca/linux-amd64/go1.20.3",
    network: {
      inbound: false,
      localAddress: "127.0.0.1:36900",
      remoteAddress: "127.0.0.1:30307",
      static: false,
      trusted: false
    },
    protocols: {
      eth: {
        version: 68
      },
      snap: {
        version: 1
      }
    }
}]
```

8. Perform Transactions between peers in the network

```
> eth.sendTransaction({from:"0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE",to:"0xfc09dce1AB0Fe679e5cC09D88B3E99de1
3864433"})
WARN [03-31|12:16:19.599] Caller gas above allowance, capping      requested=114,569,871 cap=50,000,000
INFO [03-31|12:16:19.617] Setting new local account                address=0x9F9497c56b54890fD2F693BaC26C71e4736
eb2bE
INFO [03-31|12:16:19.619] Submitted transaction                    hash=0xf2c1e10c56397e2726e94bc166405b3ff0863d
e4757b479a1b6812debe56f33f from=0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE nonce=0 recipient=0xfc09dce1AB0Fe679e
5cC09D88B3E99de13864433 value=0
"0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f"
> INFO [03-31|12:16:27.814] Looking for peers                        peercount=1 tried=0 static=0
```

9. Check the balances of sender and receiver 2

```
> eth.getBalance("0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE")

1.000322e+24
```

10. To check the details of the transaction on Node 1 Terminal

```
> eth.getTransaction("0xe181e503b93f95f2e69553091fbdd8459eda0942662c68ead0c5492779cf31c7")
{
  blockHash: null,
  blockNumber: null,
  chainId: "0x4d2",
  from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
  gas: 21000,
  gasPrice: 1000000000,
  hash: "0xe181e503b93f95f2e69553091fbdd8459eda0942662c68ead0c5492779cf31c7",
  input: "0x",
  nonce: 4,
  r: "0xd7bb1c00265f84c46d3c4fbf9ed8228eb791fb0e0303f1d2098989bfed21ecd1",
  s: "0x676eb39f0310ac550aeed7bade8f2988b99e64fcbcbe743cf91d2d889298c6d9",
  to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
  transactionIndex: null,
  type: "0x0",
  v: "0x9c7",
  value: 90009000
}
> INFO [03-31|12:19:48.198] Looking for peers                        peercount=1 tried=0 static=0
> INFO [03-31|12:19:58.215] Looking for peers                        peercount=1 tried=0 static=0
```

11. To check the contents in the Mempool - Transaction Pool

```
> txpool.content
{
  pending: {
    0x9F9497c56b54890fD2F693BaC26C71e4736eb2bE: {
      0: {
        blockHash: null,
        blockNumber: null,
        chainId: "0x4d2",
        from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
        gas: "0x5208",
        gasPrice: "0x3b9aca00",
        hash: "0xf2c1e10c56397e2726e94bc166405b3ff0863de4757b479a1b6812debe56f33f",
        input: "0x",
        nonce: "0x0",
        r: "0xfe80302530833e03cb38f654004a63d2b812eeab0d15ac0c743c02959e32b17e",
        s: "0x532a68167bf51184d46643bc35d828d941fc9a75a043b4b37010a7409cb41f1c",
        to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
        transactionIndex: null,
        type: "0x0",
        v: "0x9c8",
        value: "0x0"
      },
      1: {
        blockHash: null,
        blockNumber: null,
        chainId: "0x4d2",
        from: "0x9f9497c56b54890fd2f693bac26c71e4736eb2be",
        gas: "0x5208",
        gasPrice: "0x3b9aca00",
        hash: "0xdde43d6b52e779b78d33a4e5c311ce24b71fc7c7434c93464d662ce20f81ec3c",
        input: "0x",
        nonce: "0x1",
        r: "0xa1abb8a85a39d5da37ac14f8275a2eb5034e8182bd4cfa7b3523d631020188f3",
        s: "0xfb96f3fb07299f86289f4a30805b0872a36b091f9c9c65e577de70adb609e11",
        to: "0xfc09dce1ab0fe679e5cc09d88b3e99de13864433",
        transactionIndex: null,
```

12. To check the status of the Mempool - Transaction Pool

```
> txpool.status
{
  pending: 5,
  queued: 0
}
> INFO [03-31|12:21:38.389] Looking for peers                        peercount=1 tried=0 static=0
```

Conclusion:-

The experiment successfully implemented a **Private Ethereum Blockchain** using **Geth** and **Clique (PoA)** consensus. By initializing a custom **genesis block**, an isolated network was established for zero-cost testing. This setup proved that authorized signers can effectively validate transactions without the computational overhead of public networks, providing a secure environment for enterprise DApp development.

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

EXPERIMENT NO.10

Name: Aryan Anil Patankar
Class:D20A
Roll No:38
Batch:B

Aim: Explore the Cryptocurrency Landscape

Theory:

This experiment explores the cryptocurrency landscape by combining real-world market data analysis with blockchain technology research. The focus asset for this analysis is **Chainlink (LINK)**, and data was collected from three reputable sources as per the lab requirement:

20. **Financial Data (APIs)** – Services like **CoinGecko** and **CoinMarketCap** provided structured data on market capitalization, circulating supply, and historical 24-hour trading volumes, which are essential for calculating liquidity and asset dominance.
21. **News & Social Platforms** – Platforms such as **CryptoPanic** (news aggregator) and **LunarCrush** (social sentiment) were used to track how public perception and breaking news correlate with price volatility.
22. **Regulatory & Institutional Portals** – Websites like **The Block Pro** or official government portals (e.g., SEC, EU MiCA updates) provided data on legal frameworks and institutional entry.

Cryptocurrency Landscape

- **Definition** – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
- **Market Diversity** – Thousands of cryptocurrencies exist, each serving unique roles:

    - **Store of Value (SoV)** – Bitcoin (BTC), characterized by digital scarcity.
    - **Utility Tokens** – Chainlink (LINK), Ether (ETH), used to pay for network services.
    - **Stablecoins** – USDT, USDC, pegged to fiat (USD) to provide price stability.
    - **Governance Tokens** – UNI, AAVE, granting holders voting rights over protocols.
- **Volatility** – Prices are highly sensitive to market sentiment, technological developments, and "macro" regulatory drivers.
- **Data Analysis Importance** – Integrating financial, social, and regulatory data is essential for a holistic understanding of asset performance and market health.

Data Collection Process

The program used the CoinGecko API to automate historical data fetching for Chainlink. The retrieved data was processed with Pandas to compute:

- **Daily Returns** – Percentage change between consecutive days to measure volatility.
- **Moving Averages (MA7, MA30)** – Short-term and long-term trend indicators.
- **Cumulative Returns** – Growth factor over time.
- **Rolling Volatility** – Short-term risk measurement based on daily return fluctuations.

Visualization & Dashboard

Using Matplotlib, a six-panel dashboard was created to display:

17. **Price Trend** – Daily price movement over the selected period.
18. **Price vs Volume** – Correlation between trading activity and price changes.
19. **Daily Returns Distribution** – Statistical spread of daily gains/losses.
20. **Moving Averages** – Technical analysis trend lines for price movement.
21. **Cumulative Returns** – Overall growth performance.
22. **Rolling Volatility** – Market risk trends over time.

Technological Analysis

Beyond market data, the study covered advancements in blockchain technology, including:

- **Consensus Mechanisms** – The shift toward Proof of Stake (PoS) for sustainability and Proof of History (PoH) for faster processing.
- **Scalability Solutions** – Implementation of Layer-2 Rollups (Optimistic and ZK-Rollups) and Sharding to solve the "Blockchain Trilemma."
- **Interoperability Protocols** – Tools like Chainlink CCIP and Polkadot's Relay Chain enabling seamless cross-chain data and value exchange.
- **Privacy Features** – Zero-Knowledge Proofs (ZKP) allowing verifiable transactions without revealing sensitive underlying data.
- **Technical Roadmaps** – Ethereum's focus on "The Surge" (100k+ TPS) and Solana's "Firedancer" client for institutional-grade reliability.

Market Trends & Adoption

13. **Merchant Acceptance** – Significant surge in interest with major processors like Stripe and Shopify integrating stablecoins for global settlement.
14. **Wallet Growth** – Increasing downloads of non-custodial wallets (MetaMask, Phantom), indicating a shift toward direct on-chain interaction.
15. **Network Activity** – Analysis of transaction volumes vs. active addresses to distinguish between "whale" activity and organic retail growth.
16. **Integration with Traditional Finance** – The rise of Real-World Asset (RWA) Tokenization, bringing traditional assets like bonds and real estate on-chain.
17. **Regulatory Developments** – The implementation of frameworks like the EU's MiCA and U.S. FIT21 Act, providing a clear licensing roadmap for the industry

Utilizing Api for analysis and visualization :
Code:

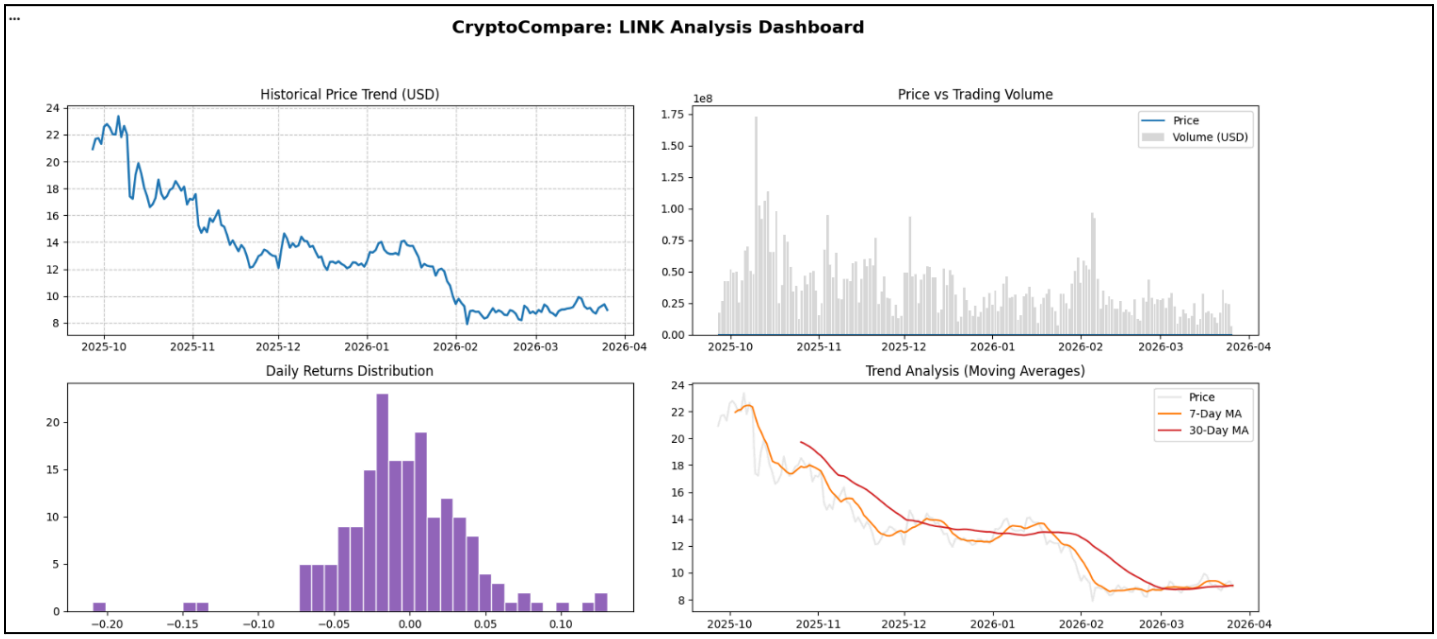
```
import requests
import matplotlib.pyplot as plt
```

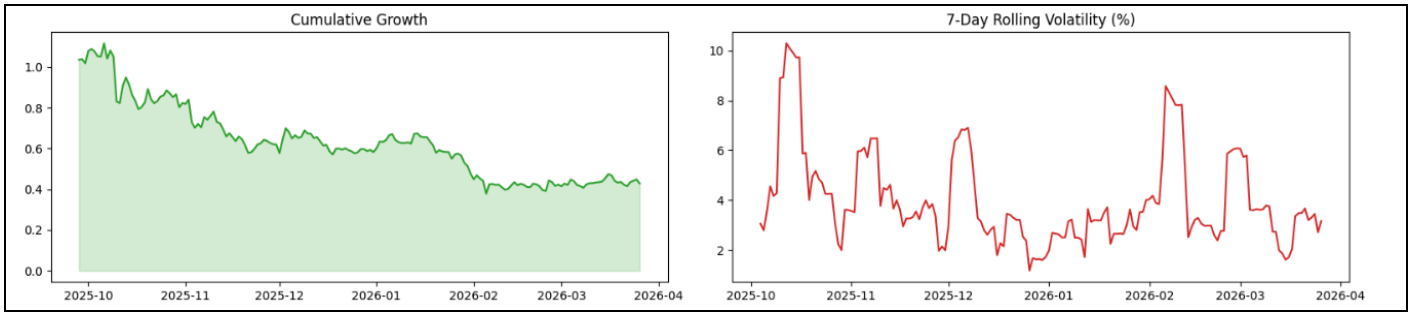
```
import pandas as pd
import datetime

# ---------- CONFIGURATION ----------
# Replace 'YOUR_API_KEY' with the key you got from CryptoCompare
API_KEY = 'YOUR_API_KEY'
SYMBOL = 'LINK'
COMPARISON_SYMBOL = 'USD'
LIMIT = 180  # Number of days

# ---------- Fetch Data Function ----------
def fetch_cryptocompare_data(symbol, comparison_symbol, limit, api_key):
    url = f"https://min-api.cryptocompare.com/data/v2/histoday"
    params = {
        "fsym": symbol,
        "tsym": comparison_symbol,
        "limit": limit,
        "api_key": api_key
    }

    response = requests.get(url, params=params)
    data = response.json()

    if data['Response'] == 'Success':
        # CryptoCompare returns OHLCV data in 'Data' -> 'Data'
        return data['Data']['Data']
    else:
        print(f"Error fetching data: {data.get('Message')}")
        return None

# ---------- Dashboard Function ----------
def cryptocompare_dashboard(symbol, comparison_symbol, limit, api_key):
    raw_data = fetch_cryptocompare_data(symbol, comparison_symbol, limit,
api_key)
    if not raw_data: return

    # Create DataFrame
    df = pd.DataFrame(raw_data)

    # Convert 'time' (unix timestamp) to datetime
```

```
    df['timestamp'] = pd.to_datetime(df['time'], unit='s')
    df.set_index('timestamp', inplace=True)

    # Standardize column names to match your previous logic
    # 'close' is used as the daily price
    df['price'] = df['close']
    df['volume'] = df['volumeto'] # Total volume in USD (volumeto)

    # Calculate metrics
    df['returns'] = df['price'].pct_change()
    df['MA7'] = df['price'].rolling(window=7).mean()
    df['MA30'] = df['price'].rolling(window=30).mean()
    df['cumulative'] = (1 + df['returns']).cumprod()
    df['volatility'] = df['returns'].rolling(window=7).std() * 100

    # ---------- Plotting ----------
    fig, axes = plt.subplots(3, 2, figsize=(16, 12))
    fig.suptitle(f'CryptoCompare: {symbol} Analysis Dashboard',
fontsize=16, fontweight='bold')

    # 1. Price Trend
    axes[0, 0].plot(df.index, df['price'], color='tab:blue', linewidth=2)
    axes[0, 0].set_title('Historical Price Trend (USD)')
    axes[0, 0].grid(True, linestyle='--', alpha=0.7)

    # 2. Price vs Volume
    axes[0, 1].plot(df.index, df['price'], color='tab:blue',
label='Price')
    axes[0, 1].bar(df.index, df['volume'], color='gray', alpha=0.3,
label='Volume (USD)')
    axes[0, 1].set_title('Price vs Trading Volume')
    axes[0, 1].legend()

    # 3. Daily Returns Distribution
    axes[1, 0].hist(df['returns'].dropna(), bins=40, color='tab:purple',
edgecolor='white')
    axes[1, 0].set_title('Daily Returns Distribution')

    # 4. Moving Averages
    axes[1, 1].plot(df.index, df['price'], color='lightgray', alpha=0.5,
```

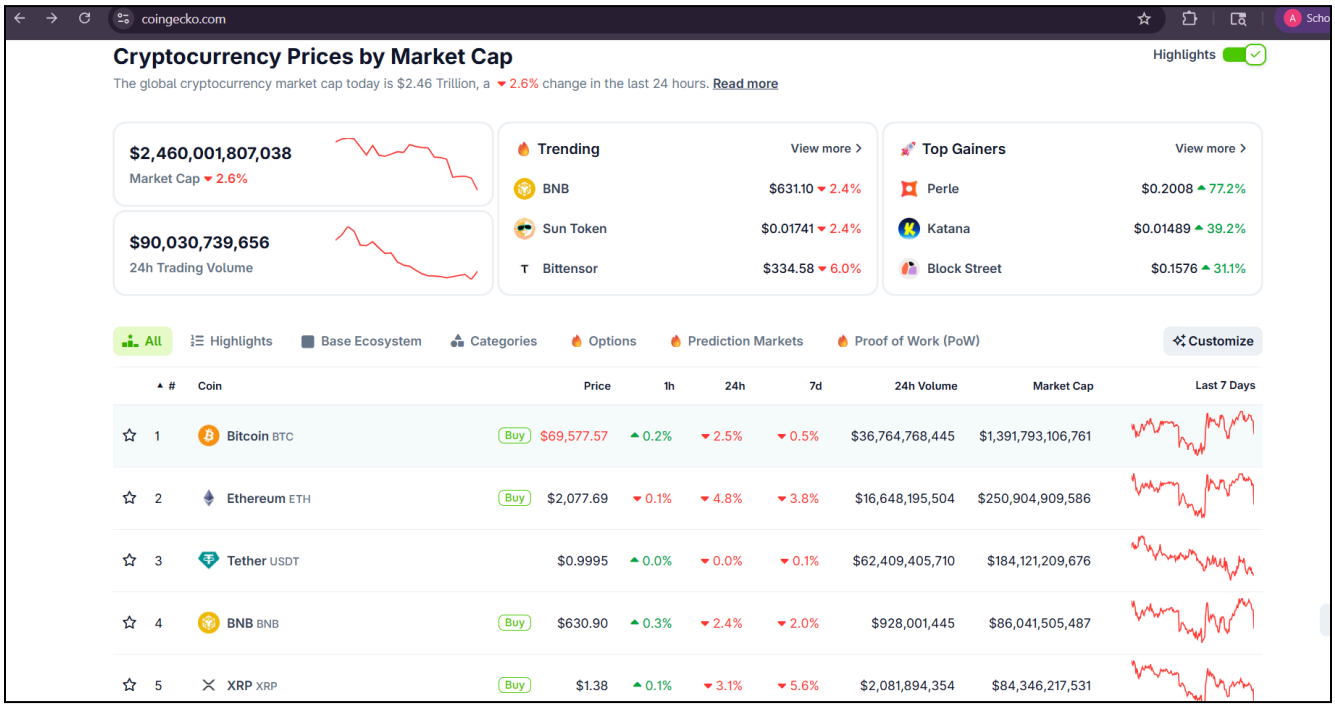
```
label='Price')
    axes[1, 1].plot(df.index, df['MA7'], color='tab:orange', label='7-Day
MA')
    axes[1, 1].plot(df.index, df['MA30'], color='tab:red', label='30-Day
MA')
    axes[1, 1].set_title('Trend Analysis (Moving Averages)')
    axes[1, 1].legend()

    # 5. Cumulative Returns
    axes[2, 0].fill_between(df.index, df['cumulative'], color='tab:green',
alpha=0.2)
    axes[2, 0].plot(df.index, df['cumulative'], color='tab:green')
    axes[2, 0].set_title('Cumulative Growth')

    # 6. Volatility
    axes[2, 1].plot(df.index, df['volatility'], color='tab:red')
    axes[2, 1].set_title('7-Day Rolling Volatility (%)')

    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

# ---------- Run ----------
cryptocompare_dashboard(SYMBOL, COMPARISON_SYMBOL, LIMIT, API_KEY)
```

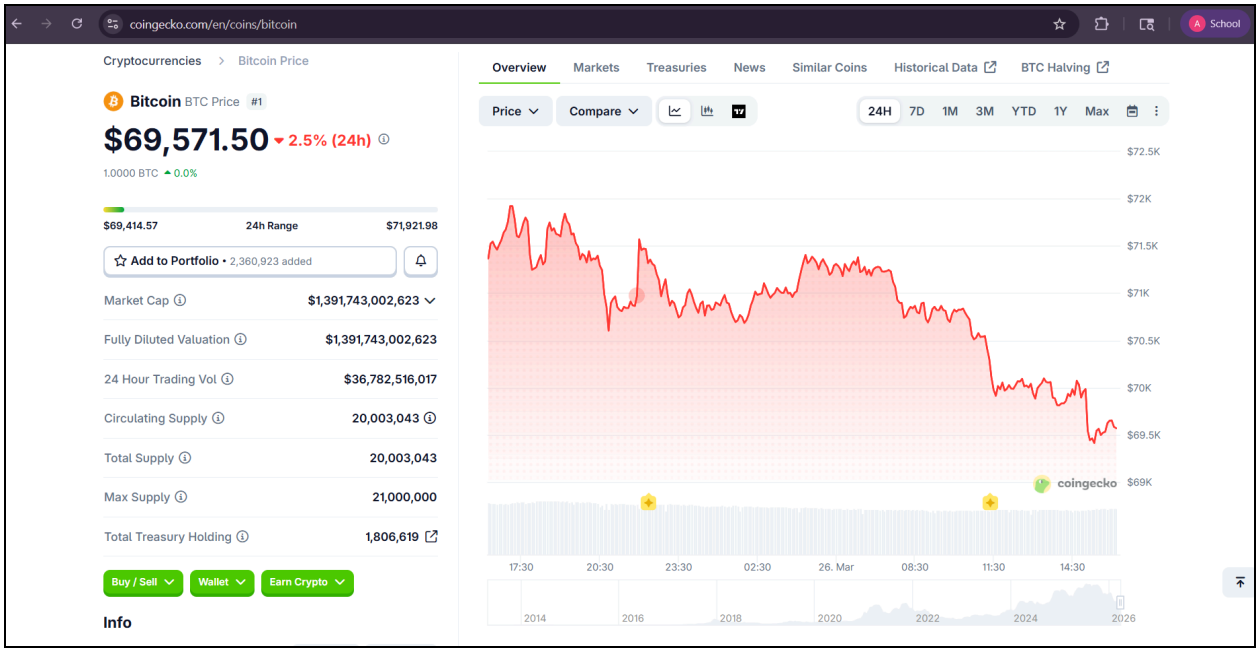
Output:-



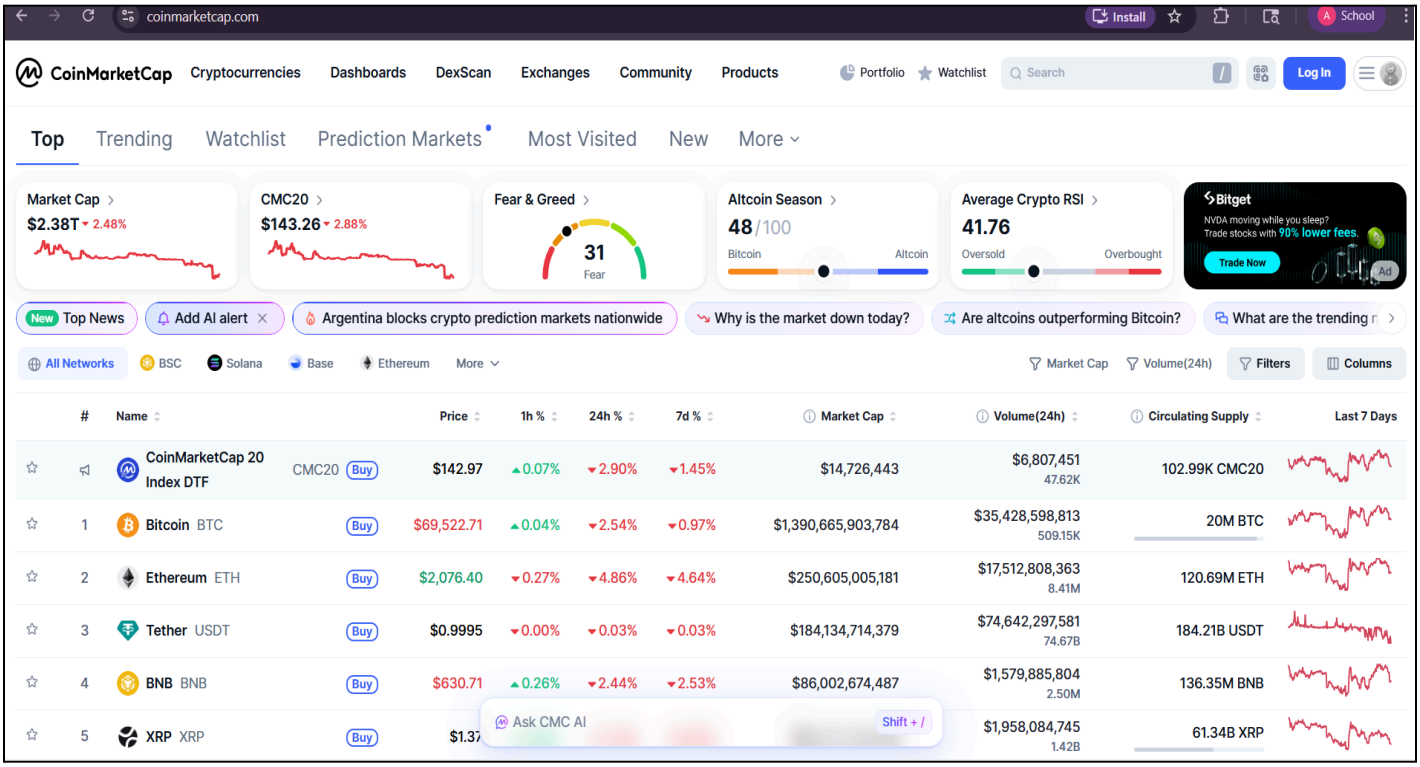


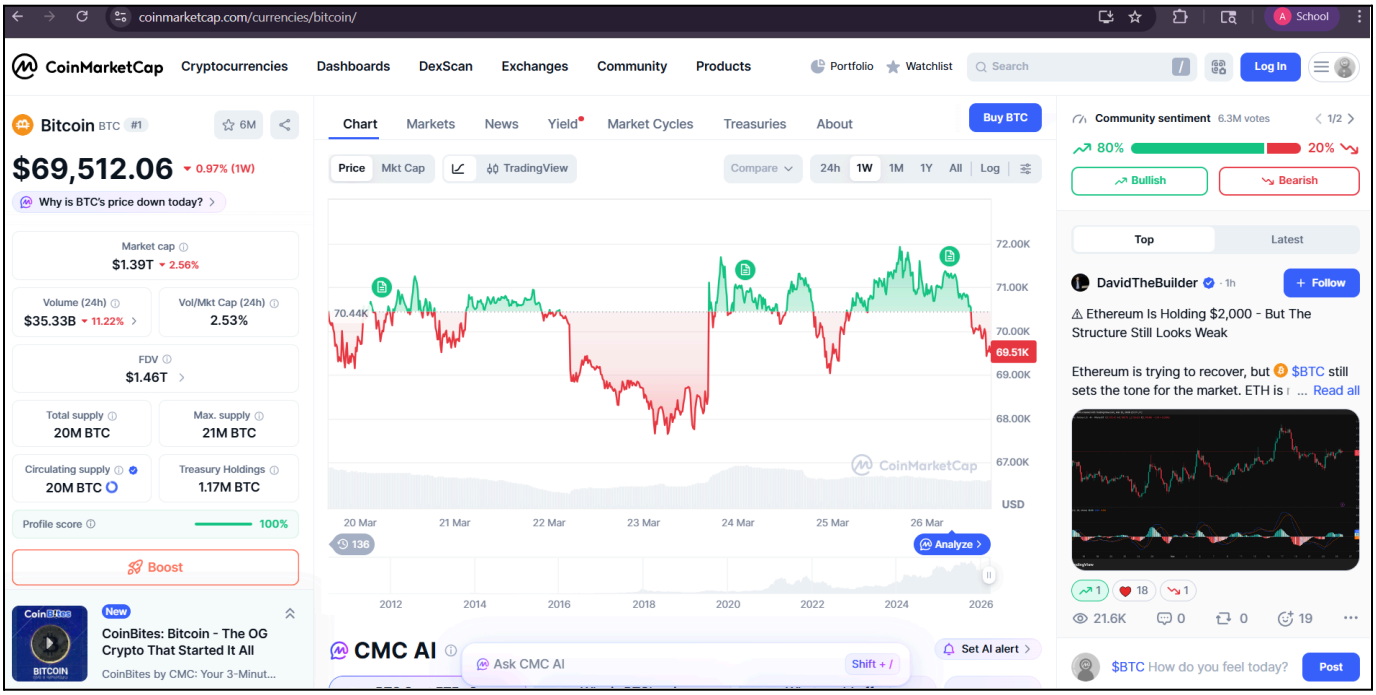
Coingecko Website





CoinMarketCap website





Conclusion:-

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

EXPERIMENT NO.11

Name: Aryan Patankar
Class:D20A
Roll No:38
Batch:B

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Prerequisites:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

Procedure:

Step 1: Update and Install dependencies

Command:
sudo apt update
sudo apt upgrade
-y

```
Aryan@Aryan:~$ sudo apt update
sudo apt upgrade -y
sudo apt install jq curl git nodejs npm -y
[sudo] password for vaishnal:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1579 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [253 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7
Get:10 http://security.ubuntu.com/ubuntu noble-scurity/universe amd64 Packages [1170 k
```

Step 2: Clone Fabric

Sample Command:

git clone
https://github.com/hyperledger/fabric-samples cd
fabric-samples

```
Aryan@Aryan:~$ git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects:    100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161),   reused 109 (delta 109), pack-reused 153380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 390.00 KiB/s, done.
Resolving deltas: 100% (8809/8809), done.
```

Step 3: Install Fabric Command:

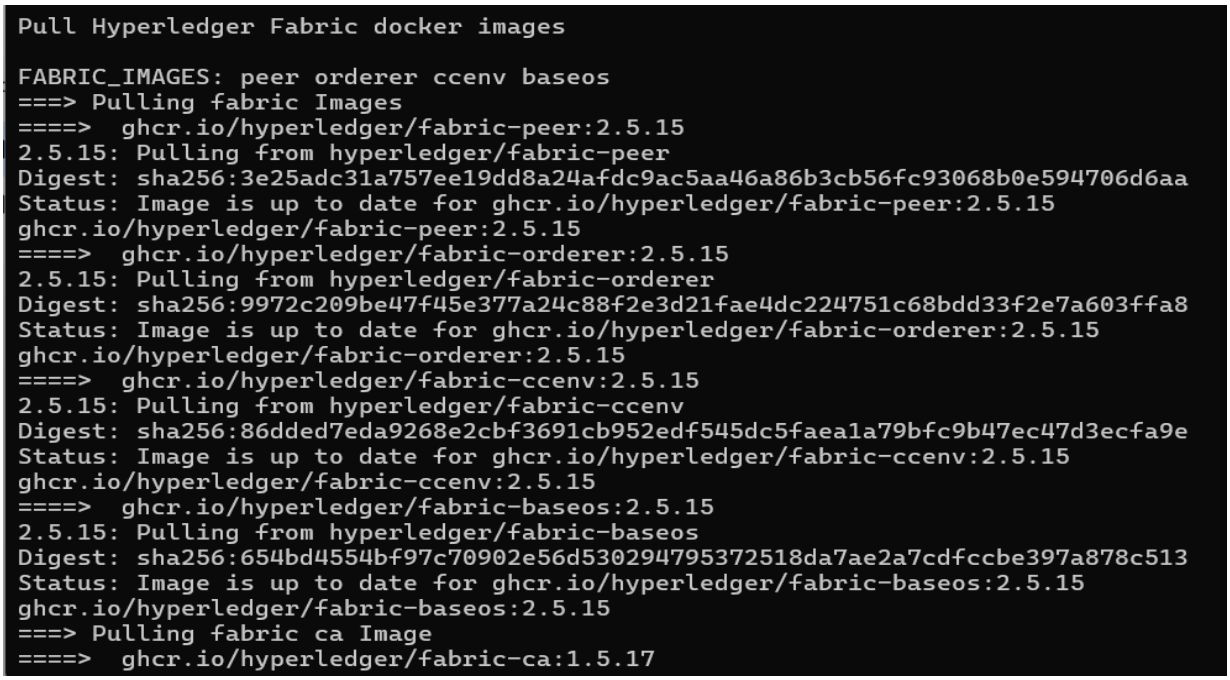
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-f
abr ic.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

```
Aryan@Aryan:~/fabric-samples$ curl --SSLO https://raw.githurohubusercontent.com/hyperledger/fabric/main/scripts/instal
-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent v
rsions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.1
.tar.gz
===> Will unpack to: /home/Aryan/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0   487k      0  0:04:13  0:04:13 --:--:--  397k
```

Step 4: Verify Installation Command:

docker images | grep fabric

```
Aryan@Aryan:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --f
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB         123MB
ghcr.io/hyperledger/fabric-peer1.5.17       453a0a727e82        381MB         123MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92        1GB           2555MB
hyperledger/fabric-ccenv:latest             86dded7eda92        1GB           255MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
```



Step 5: Set Environment

Variables Command:

export PATH=$PATH:$HOME/fabric-samples/bin

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version

```
Aryan@Aryan:~/fabric-samples$ export PATH=$PATH $HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
Aryan@Aryan:~/fabric-samples$ peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.20.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

Step 6: Start Network Command:

cd ~/fabric-samples/test-network

./network.sh up createChannel

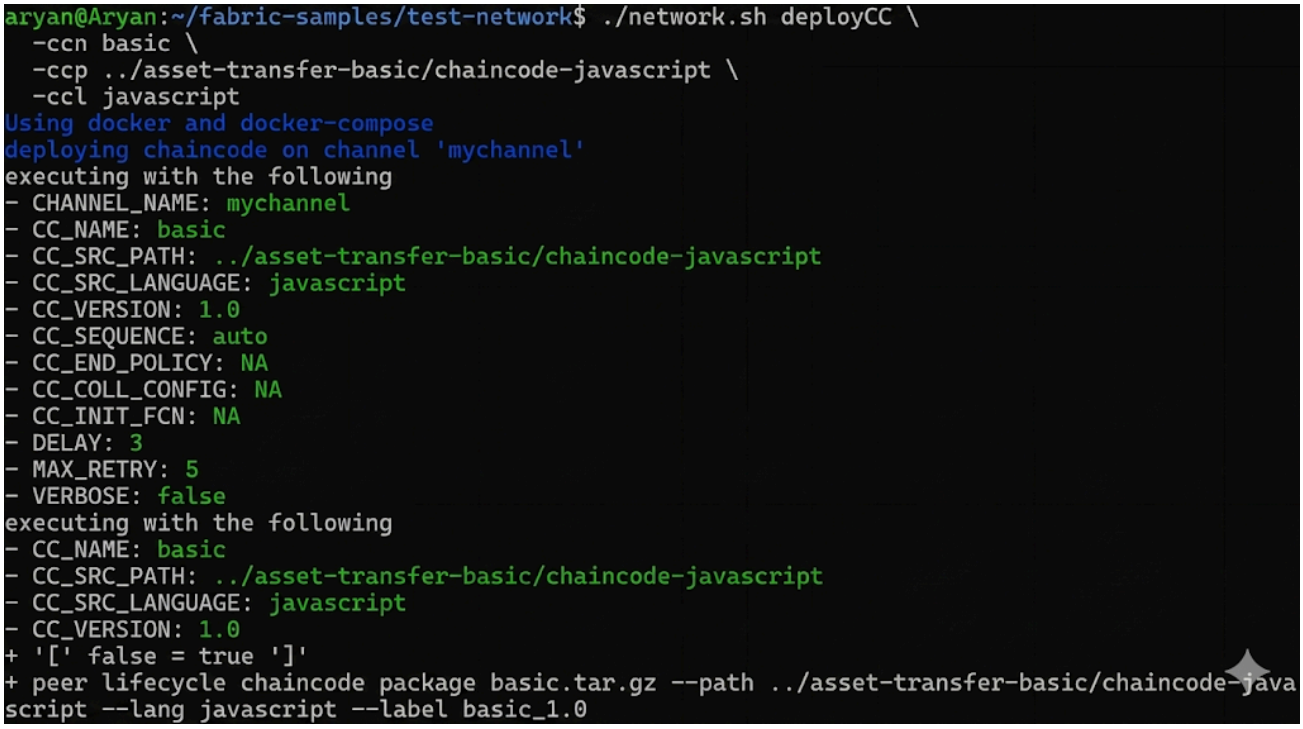
```
aryan@Aryan:~/fabric-samples$ cd ~/fabric-samples/test-network
./network.sh up createChannel
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and us
ldb with crypto from 'cryptogen'
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/aryan/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizati
Generating CCP files for Org1 and Org2
[+] up 7/7
 ✔Network fabric_test                    Created
 ✔Volume compose_peer0.org1.example.com  Created
```

Step 7: Deploy Chaincode Command:

./network.sh deployCC \

-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript

```
++ configtxlator proto_encode --input /home/vaishnal/fabric-samples/test-network/channel-artifacts/config_update_in_en
lope.json --type common.Envelope --output /home/vaishnal/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.
2026-04-09 17:04:04.975 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-09 17:04:04.990 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
vaishnal@Vaishnal:~/fabric-samples/test-network$
vaishnal@Vaishnal:~/fabric-samples/test-network$
```



Step 8: Set Peer

Environment Command:

export CORE_PEER_TLS_ENABLED=true

export

CORE_PEER_LOCALMSPID="Org1MSP"

export

CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.exampl

e.com/msp

export CORE_PEER_ADDRESS=localhost:7051

```
aryan@Aryan:~/fabric-samples/test-network$ export CORE_
PEER_TLS_ENABLEED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/te
est-netwwork/organizations/peer/orincas/
export org1.example.com/peers/peerr/peer0.org1.example.c
om/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/set
test-network/organzations/pix  /deer-orgercar(ia,peeroor
ganizations,org10.com/peer
peerOrganzations/org1.example.com/ussr@org1.example.com/
                              users/Admin@org1.example.com/
```

Step 9: Initialize Ledger Command:

peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
"$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers
/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
-c '{"function":"InitLedger","Args":[]}'

```
Aryan@Aryan:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsca
t.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
-c '{"function":"InitLedger","Args":[]}'
2026-04-09 17:26:44.205 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
Aryan@Aryan:~/fabric-samples/test-network$
```

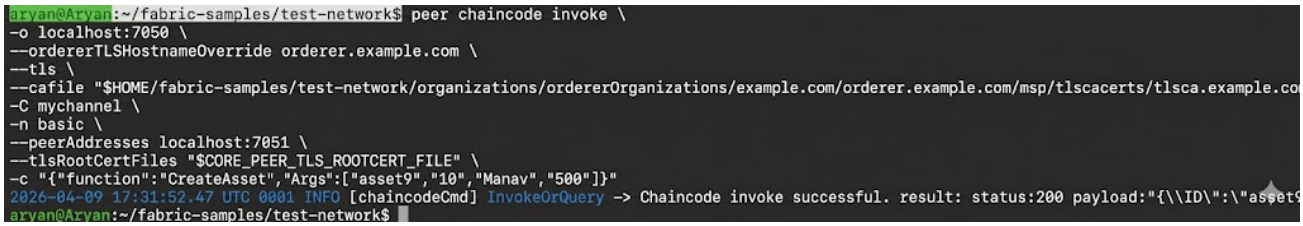
Step 10: Query Assets Command:

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

Step 11: Create Asset Command:

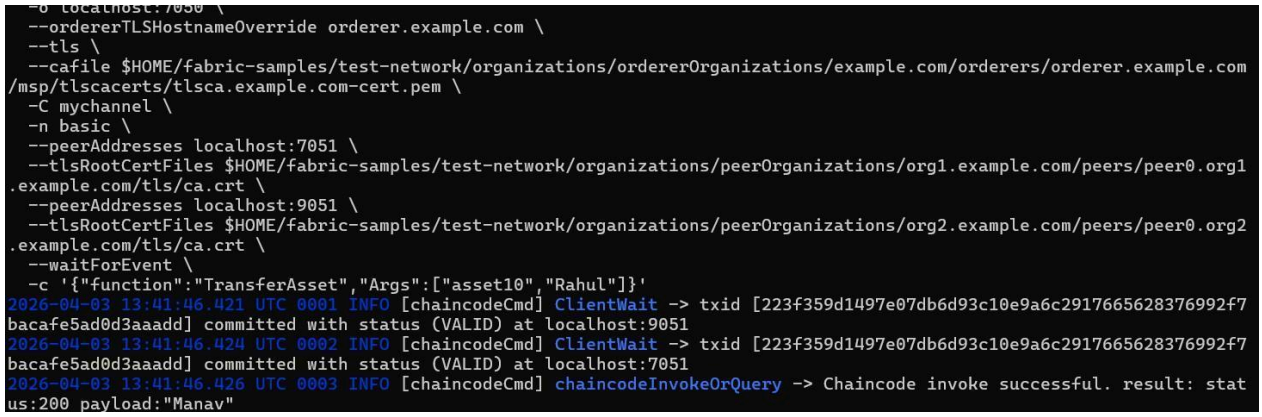
peer chaincode invoke \

-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
"$HOME/fabric-samples/test-network/organizations/ordererOrganizations
/ex
ample.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.co
m-c ert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
-c '{"function":"CreateAsset","Args":["asset9","blue","10","Manav","500"]}'



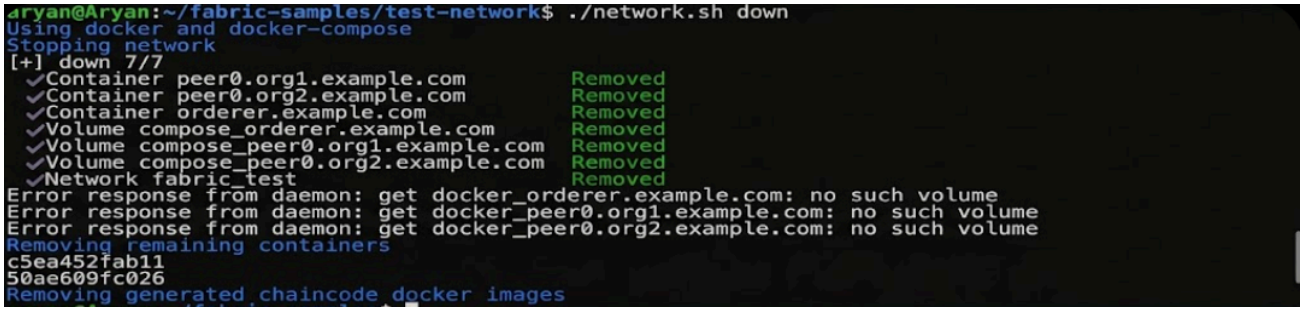
Step 12: Transfer Asset Command:

peer chaincode invoke ... TransferAsset



Step 13: Stop Network Command:

./network.sh down



Result:

The Hyperledger Fabric network was successfully implemented.

Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name **Manav**, confirming correct execution.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 12

Aim: Mini Project

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 13

Aim: Assignment 1

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 14

Aim: Assignment 2

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain it significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization,and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 15

Aim: Assignment 3

Roll No.	38
Name	Aryan Patankar
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance.Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency.(Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted